Decide, Don’t Generate

Decide, Don’t Generate

Jev, System One Models, and the Decision Layer of Agentic AI

Sridhar Mukkandi

First edition

Decide, Don’t Generate: Jev, System One Models, and the Decision Layer of Agentic AI

Copyright © 2026 Sridhar Mukkandi. All rights reserved.

No part of this book may be reproduced, stored or transmitted in any form without written permission from the author, except for brief quotations in reviews and articles. The code in this book is released under the MIT licence at github.com/Mukkandi-Sridhar/decide-dont-generate.

Independently published. First edition, 2026.

Jev is a product of TypeSafe AI. This book is independent: the author is not affiliated with TypeSafe AI, and the book is not endorsed or sponsored by it. All trademarks belong to their respective owners.

About the numbers in this book. The author did not have access to the Jev API while writing. Every Jev response, chart and table in this book was produced by `jev-mock-synthetic`, a deterministic stand-in that copies Jev’s public interface but not its behaviour. Such material is labelled “Synthetic — not measured on real Jev”. Claims made by the vendor are labelled “vendor-reported” and cited. The security data is synthetic too: Kestrel Logistics, Harbor Pharma and everyone in them are invented.

The information in this book is provided without warranty. The author assumes no responsibility for errors or omissions, or for damages resulting from its use. Nothing in this book is legal, security-compliance or medical advice.

Cover design and typesetting by the author. Text set in Source Serif 4, Inter and JetBrains Mono, all under the SIL Open Font License.

Contact: sridhar.authorhub@gmail.com

Table of contents

Preface vii

How to read this book viii

Prologue: two nights ix

1 What “learning” means 1

2 Probability, and how a machine learns it 7

3 Calibration: when 0.8 really means 80% 13

4 From probabilities to actions 19

5 How modern AI models work 25

6 RAG, agents, and where they break 31

7 Fast and slow: System One models and Jev 37

8 Asking the right question: choice, score, noul 43

9 Testing a model’s calibration yourself 48

10 The bake-off: six ways to make a decision 54

11 Act, review, or escalate 60

12 Decision patterns, and a hybrid agent 66

13 One team, start to finish, and beyond 74

14 From model to service 81

15 What changes now 88

Appendices 95

Glossary 95

Cheat sheets 99

Bibliography 102

Index 105

About the Author 107

Preface

This is a book about deciding.

Not about AI in general, though you'll learn a lot of that on the way. It's about one narrow, very common job: looking at a situation and choosing what to do. Is this email phishing? Which team gets this ticket? Is this payment safe? Should this agent call that tool?

Software makes millions of these decisions every day, and more and more of them are handed to large language models. LLMs are superb at reading and writing. But a decision isn't a paragraph. It's a choice between a few options, made without being sure, with a price for getting it wrong. To make one well you need three things:

- a **probability you can trust**, one that means what it says;
- a **threshold set by what mistakes cost**, not by habit;
- a clear rule for **when to hand the case to a person**.

Most people building with AI today have none of the three. This book gives you all of them, from first principles, in plain language.

In 2026 a company called TypeSafe AI released Jev, which it calls the first **System One model**: a model that never writes, and only answers typed questions with probabilities, in one fast pass (TypeSafe AI 2026). You'll learn to test whether Jev does what its makers claim. But the idea behind it outlives any one product: some decisions belong in a separate, fast, measurable layer of an AI system, and that layer needs different skills from the rest.

Three rules I followed

No invented measurements. I had no access to Jev's API while writing, so every Jev number comes from a stand-in that copies its public interface, and is marked *synthetic* every time. These numbers show you how to reason and test; they're not claims about the real model.

Vendor claims are labelled. When I report what TypeSafe says about Jev, I say it's *vendor-reported* and cite the source.

Claims go in piles. Everything said about Jev is sorted into what I could check, what the vendor reports, and what nobody outside the company knows yet. Chapter 9 explains how. Use the same three piles for anything you read about new models, this book included.

How to read this book

New to machine learning? Read it in order. Chapters 1 to 4 are the foundation: what a probability means, how to check one, and how to turn it into an action. Chapters 5 and 6 are a short tour of neural networks, LLMs and agents.

Already build with LLMs? Read Chapters 3 and 4 anyway; everything after depends on them. Then go to Chapter 7.

Lead a team or a product? Read Chapters 1 to 4, then 7, 10, 11, 13 and 15. You'll be able to ask the four questions that matter: What does a mistake cost? Where's the threshold? Can we trust the probabilities? Who checks the warnings?

The boxes. A **Key idea** box holds one sentence worth remembering a year from now; each chapter ends with **Keep these**, its key ideas in one list. **Where this breaks** shows the limits of what the chapter just taught: read these most of all. **Set the threshold** asks you to make a small decision with real costs, with the answer in brackets.

The numbers. The running example is a made-up shipping company, Kestrel Logistics, and its security team. Its alerts come from a program that knows the true answer for every one, so we can measure exactly how many attacks a policy misses. Every such number is an *example of a method*, not a fact about the world, and is marked **synthetic**. Every Jev answer comes from a stand-in that copies Jev's public interface. What TypeSafe AI says about Jev is marked *vendor-reported*. The code behind every figure is in the companion repository, github.com/Mukkandi-Sridhar/decide-dont-generate; you don't need it to follow the book.

Prologue: two nights

Picture a shipping company's security team. Its computers raise a warning, called an **alert**, whenever something looks suspicious: a sign-in from a strange country, an email with an odd link, a laptop talking to an unknown server. Someone has to look at each alert and decide: real attack, or nothing?

Tonight one person is on duty. Checking an alert properly takes about twelve minutes, so they can check about five an hour.

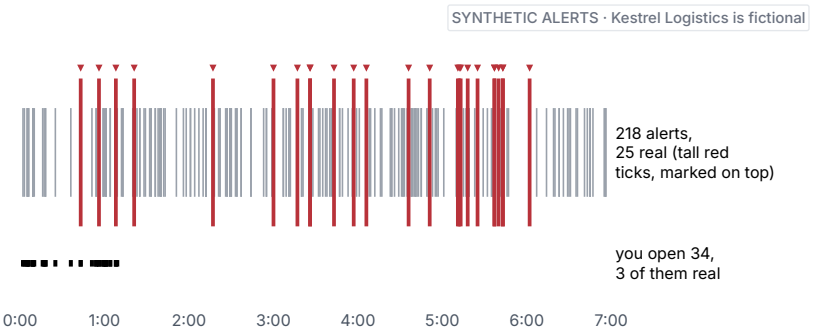


Figure 1. One night, midnight to 7 a.m. Each line is an alert; the real attacks are red, taller and marked on top. Working in the order they arrive, the person on duty checks the ones on the left. The rest wait for the morning.

Figure 1 shows the whole night. 218 alerts arrived between midnight and seven. 25 of them were real attacks. Working through them in the order they arrived, the person on duty checked 34, and only 3 of those were real.

They made no mistakes. Every alert they checked, they judged correctly. The problem was the order. The queue decided what they looked at, and a queue only knows what arrived first.

Now picture a second night.

A model reads every alert the moment it arrives. It writes nothing. It answers one question with a number: *how likely is this to be real?* And that number can be trusted: of the alerts it calls "one in ten", about one in ten really are attacks.

Alerts it's almost sure are harmless get closed, with a random few kept aside to check later. Alerts it's fairly sure are dangerous wake someone up at once. Everything in between goes to the person on duty, most likely first. The person still makes the decisions that matter, on the right alerts.

That second night is what this book builds. It needs little new technology and a lot of clear thinking: what a probability really is and how to tell if one can be trusted; what each mistake costs, and where that puts the line between “act” and “ignore”; and when a machine should decide and when it should hand the case to a person. The same thinking runs a bank’s fraud check, a hospital’s waiting list and every AI agent choosing its next step. The security team is just the example, because its mistakes are easy to count.

Next: before a machine can decide anything, it has to learn. What does that actually mean?

CHAPTER 1

What “learning” means

The question of whether a computer can think is no more interesting than the question of whether a submarine can swim.

— Edsger W. Dijkstra, 1984

AI, machine learning, deep learning, LLMs, RAG, agents: it can feel like seven subjects to learn. It’s one story. Each step exists because the one before it hit a problem it couldn’t solve. This chapter tells that story, and shows where this book fits in it.

BY THE END OF THIS CHAPTER

- Tell a computer following rules from a computer learning.
- Say what a *model* is in one sentence.
- Place AI, machine learning, LLMs, RAG, agents and Jev on one map.

Rules, and where they run out

Inside a thermostat is a line that says, more or less: *if the temperature is above 30°C, turn on the fan.* That’s a computer following a rule. It works nothing out.

Rules are underrated. They’re fast, cheap, and when one goes wrong you can read why. When a rule does the job, use it.

The trouble starts when the world changes faster than you can write rules. A security team wants to catch *phishing*, emails that trick staff into typing a password into a fake page. Someone writes: *flag any email that mentions “password”.* On Monday it flags the IT team’s own reset emails. On Wednesday an attacker writes “passw0rd”. On Friday the attacker drops the word entirely and sends a link to a fake shared document. The rule list grows, and it’s always one trick behind.

So turn the job around. Give the computer thousands of emails where the answer is already known, *this one was phishing, this one was fine*, and let it find the pattern itself.

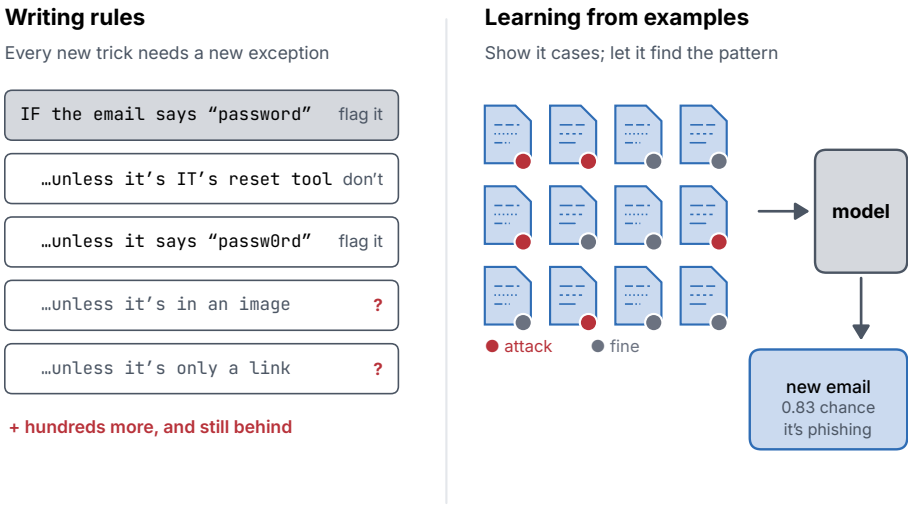


Figure 1.1. Two ways to catch phishing. On the left, every new trick forces a new exception, and the list never catches up. On the right, we stop writing rules and show the computer labelled examples instead.

● KEY IDEA

Machine learning is getting a computer to work out the rules itself, from examples where the answer is already known.

What learning leaves behind is a **model**: something that takes an input and gives an answer. Picture a machine with many knobs. Learning turns the knobs until the machine gets the examples right as often as possible; then you use it on new cases.

● KEY IDEA

A model is a function whose settings came from examples rather than from a programmer.

A machine learns the wrong thing

The book’s running example is a made-up shipping company, Kestrel Logistics. Its security team receives 20,000 warnings, or **alerts**, in four weeks, about 714 a day. Only 7.6% are real attacks. (A program invents the alerts, so we know the true answer for every one and can check every model exactly.)

Each alert has a *threat score* from 0 to 1. An obvious rule: *flag it if the score is at least t.* A number like *t* that splits cases in two is a **threshold**. Which *t*? Let the machine try every value, count the mistakes each makes, and keep the best. That's learning in its smallest form.

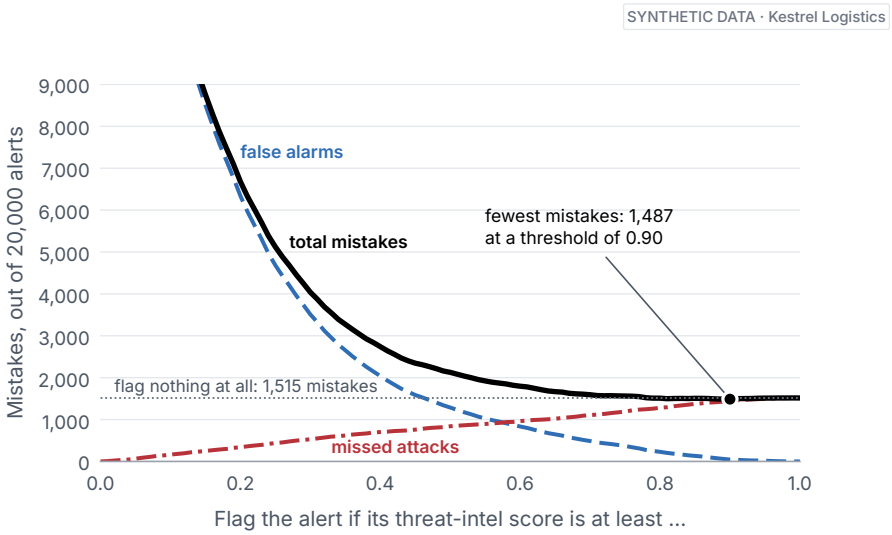


Figure 1.2. Every possible threshold and the mistakes it makes. Low thresholds bury the team in false alarms; high ones miss attacks. The total is lowest at the far right, just below the dotted line for "flag nothing at all".

It picks 0.90, flagging only the very worst scores, and makes 1,487 mistakes. A "rule" that never flags anything makes 1,515, barely more. Of 1,515 real attacks, the winner catches 73. Harmless alerts outnumber attacks twelve to one, so every false alarm hurts the count more than a caught attack helps it (Figure 1.2).

● KEY IDEA

A model learns exactly what you ask for: ask the wrong question, and it learns the wrong thing.

Two things went wrong. We counted every mistake as equal, though missing an attack costs far more than a false alarm (Chapter 4 fixes that). And we forced the answer to be *yes* or *no*.

The answer isn't yes or no

Ask instead: *among alerts with a given score, how many were attacks?*

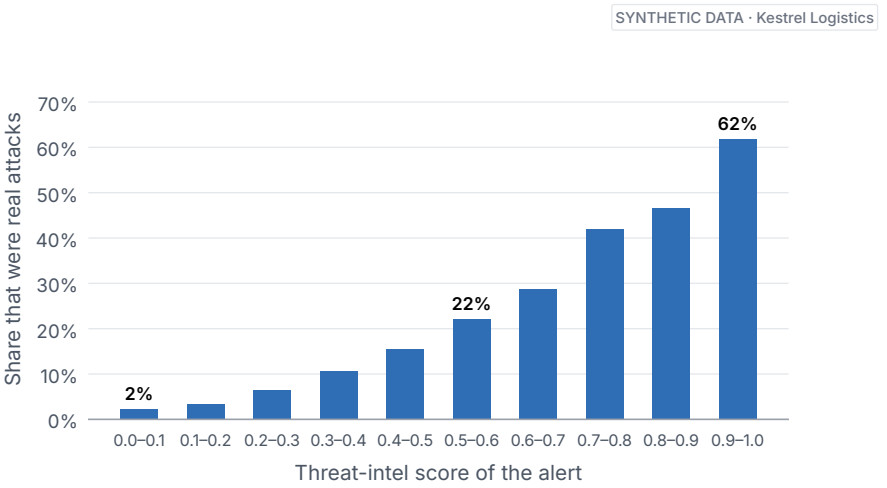


Figure 1.3. Group the alerts by threat score and count how many were real attacks. The share climbs steadily from 2% to 62%. Even at the top, plenty are harmless; even at the bottom, a few are attacks.

There’s no point where alerts switch from harmless to dangerous (Figure 1.3). Even among the lowest scores, 2% were attacks: over a hundred, hidden among 7,270 harmless-looking alerts. Among the highest, only 62% were. What we know about an alert scoring 0.55 is not “attack” or “fine”, but *about one in five like it is an attack.* That number says how worried to be, which alert to check first, and, once we know the costs, where to draw the line.

● KEY IDEA

A good model doesn’t just answer: it says how sure it is, and it’s right about how sure it is.

Being right about how sure you are is called **calibration**, and it’s the thread running through this whole book.

The rest of the story, and where this book fits

A threat score is one neat number. Photos and emails aren’t. So came models that learn their own clues in many layers (**deep learning**), then huge ones trained to guess the next word that learned language itself (**large language models**, LLMs). LLMs know the world but not your company, so we let them look things up (**RAG**), and then act in a loop with tools (**agents**). Chapters 5 and 6 cover all of this.

Watch an agent work and you notice that most of its steps aren't writing. They're small decisions with a few possible answers: *is this real? which team? is this action safe?* Today each goes to an LLM, which writes a paragraph that code then has to dig the answer out of. It's like hiring a novelist to tick a checkbox.

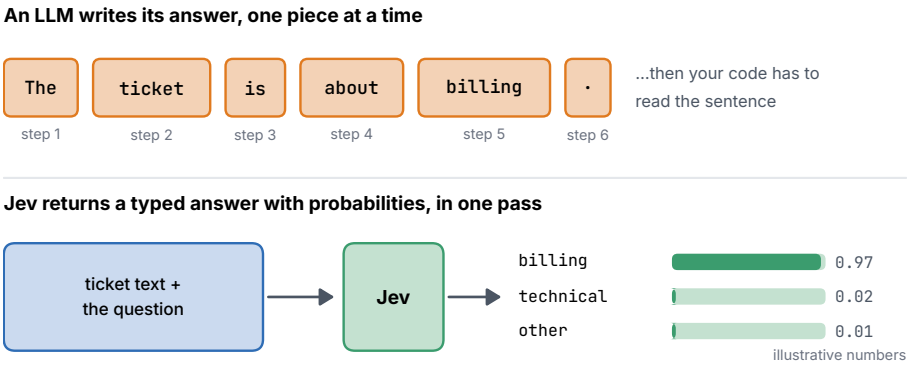


Figure 1.4. The same question, two ways. An LLM writes its answer piece by piece, and your code has to read the sentence. A decision model returns the answer as data, with a probability for every option, in one pass.

In 2026 the start-up TypeSafe AI released **Jev**, a model built only for those checkbox moments. It never writes. You give it a situation and a typed question, and it returns the answer as data, with a probability for every option. TypeSafe calls it a *System One* model, after the psychologist Daniel Kahneman's name for fast, intuitive thinking (Kahneman 2011). I had no access to Jev's API while writing, so every Jev answer in this book comes from a stand-in that copies its public interface, and is marked *synthetic*.

That's the map (Figure 1.5). The next three chapters stay at its foundation on purpose, because everything above rests on one idea: an answer should be a probability, and the probability should mean what it says.

KEEP THESE

- **Machine learning** is getting a computer to work out the rules itself, from examples where the answer is already known.
- A **model** is a function whose settings came from examples rather than from a programmer.
- A model learns exactly what you ask for: ask the wrong question, and it learns the wrong thing.
- A good model doesn't just answer: it says how sure it is, and it's right about how sure it is.

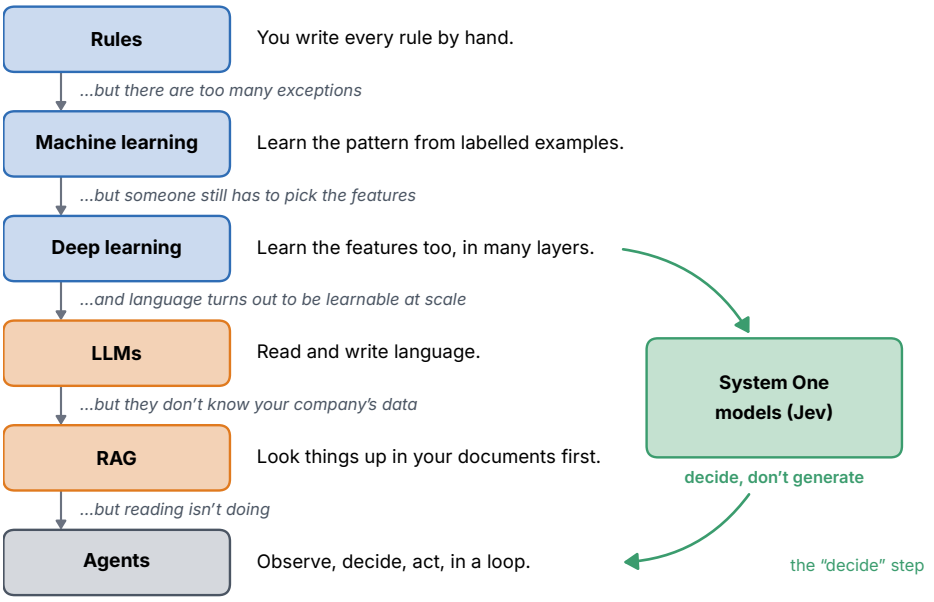


Figure 1.5. The story so far. Each step exists because the one before it hit a problem it couldn't solve. System One models like Jev grow out of deep learning and fit into the “decide” step of an agent.

EXERCISES

1. Name one system you use that follows rules, and one that learned from examples. What gave each away?

Next: “how likely” sounds like a small idea. It hides a trap that fools doctors, judges and security teams.

CHAPTER 2

Probability, and how a machine learns it

But to us, probability is the very guide of life.

— Joseph Butler, The Analogy of Religion, 1736

"Probability? Coins and dice. I don't need it to build AI." That thought is the most expensive mistake in this field. This chapter makes sure you can read a chance without being fooled, and shows how a machine learns one.

BY THE END OF THIS CHAPTER

- Say what "70%" means in a way you could check.
- Spot the base-rate trap, and update a probability by multiplying odds.
- Explain how a machine learns a probability, and how it fools you.

What "70%" means

Your weather app says 70% chance of rain. It stays dry. Was it wrong? One day can't tell you. The forecaster's *record* can: across every day they said "70%", did it rain on about 70 of every 100?

KEY IDEA

A probability is a claim about how often something happens among cases like this one.

This definition matters because you can *check* it. A model that calls an alert "70% likely" makes the weather app's promise, and can be checked the same way.

The trap that fools doctors

Try this before reading on. A detector catches 99% of real attacks and raises a false alarm on 5% of normal activity. Real attacks are rare: one event in a thousand. The detector fires. What's the chance it's a real attack?

Most people say about 95%. The answer is about 2%. In a famous 1978 study, doctors at Harvard Medical School got the same puzzle about a medical test, and the most common answer was 95% (Casscells et al. 1978).

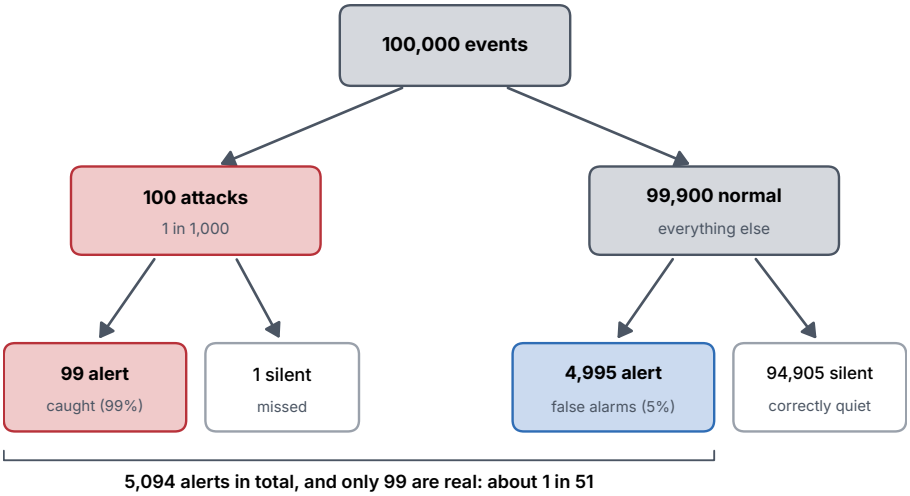


Figure 2.1. The same puzzle, counted out. Of 100,000 events, 100 are attacks and the detector catches 99 of them. But 5% of the 99,900 normal events also set it off: 4,995 false alarms. When the alarm sounds, you're holding one of 5,094 alerts, and only 99 of them are real.

The way out is to count (Figure 2.1). The 5% false-alarm rate is 5% of a *huge* number; the 99% hit rate is 99% of a *tiny* one. The chance before any evidence, here 1 in 1,000, is the **base rate**, and ignoring it is the most common probability mistake there is. It's why a "95% accurate" fraud model can drown a bank in false alarms. Whenever a probability surprises you, turn it into counts (Gigerenzer and Hoffrage 1995).

● KEY IDEA

Rare things stay rare even after the alarm goes off.

A related trap: "how often do attacks look like this?" is not "how often is something that looks like this an attack?" Only 27% of attacks come with a bad threat score, but that doesn't make a bad-score alert 27% likely to be

one; it's 46%. In court, mixing the two up is the *prosecutor's fallacy*, and it has jailed innocent people.

Bayes' rule in one line

How *should* a clue change your mind? Use **odds**: 20% is odds of 20 to 80, or 0.25. Each clue has a strength, its **likelihood ratio**: how much more common it is among attacks than among harmless alerts. A bad threat score is about 10.2 times more common among attacks.

> ● KEY IDEA
>
> **New odds = old odds × likelihood ratio.**

An ordinary alert starts at 8%. A bad threat score takes it to 46%; "outside business hours" to 58%; "sign-in from a new country" to 72%. One warning: this assumes each clue says something *new*. Real clues overlap, so in practice we let a model learn how much each is worth.

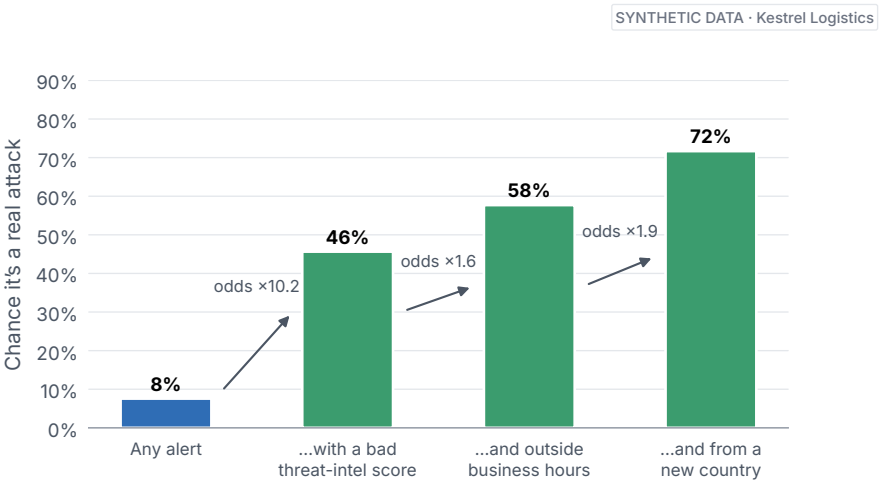


Figure 2.2. Starting from the base rate and multiplying the odds by each clue's likelihood ratio. Three ordinary clues take an alert from "probably nothing" to "probably an attack".

Why accuracy lies

Score the rule *flag any alert with a threat score of 0.5 or more*. **Precision**, how often a flag is real, is 34%. **Recall**, how many attacks it catches, is 45%. **Accuracy**, the share of all alerts judged correctly, is 89%. Sounds good,

until you see that flagging *nothing at all* scores 92%. When one outcome is rare, accuracy mostly measures how common the other one is.

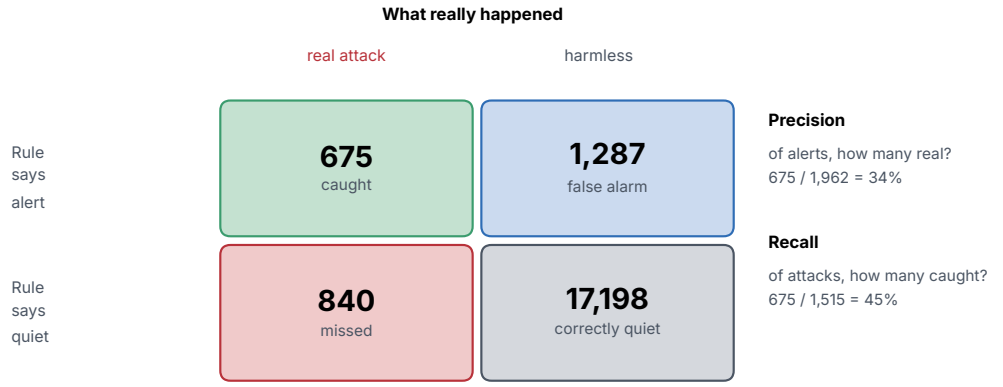


Figure 2.3. Every one of the 20,000 alerts, sorted by what the rule said and what really happened. Precision and recall each ignore one of the two columns, which makes them fair judges of rare events.

How a machine learns a probability

Bayes multiplies odds; adding is easier. Take the logarithm of the odds and multiplication becomes addition. That gives a simple machine: start from a base number, add a **weight** for each clue, and turn the total into a probability with an S-shaped curve. That's **logistic regression**, behind credit and medical risk scores for decades. It learns all the weights *together*, so overlapping clues aren't counted twice.

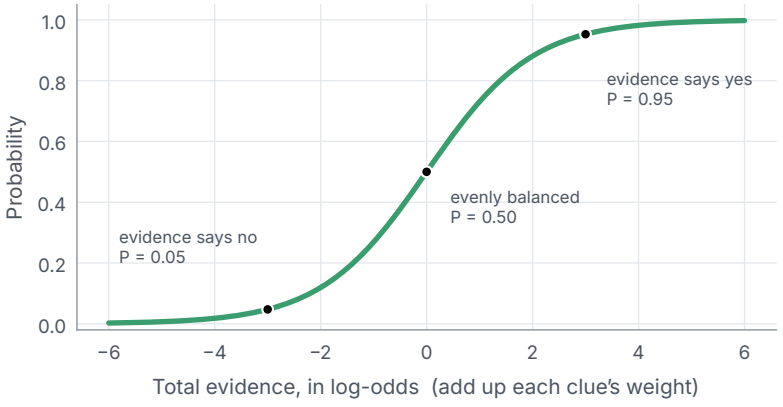


Figure 2.4. The S-curve (the logistic, or sigmoid, function) turns total evidence into a probability. Every step to the right multiplies the odds by the same amount, which is why the curve flattens out: once you're very sure, more evidence changes the probability only a little.

To set the weights, we need to score a probability. Say 90% and be right: lose almost nothing. Say 1% and be wrong: lose a lot, for being confidently wrong. That score is **log loss**, a measure of surprise.

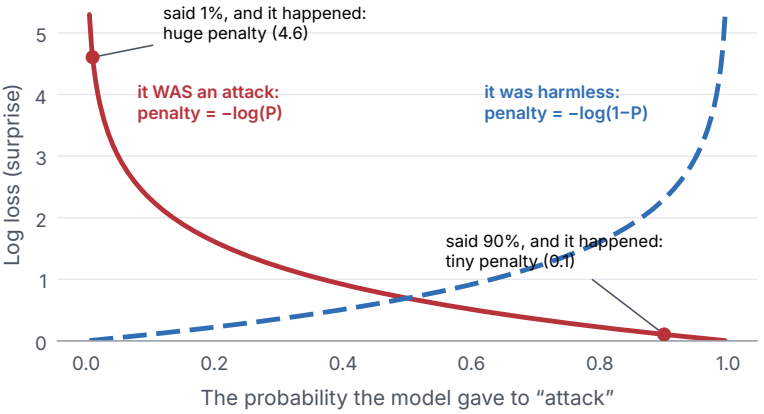


Figure 2.5. Log loss for a single alert. If it really was an attack (red, solid curve), the penalty is small when the model gave "attack" a high probability. It grows very fast as that probability gets close to zero. Harmless alerts (blue, dashed curve) work the same way from the other side.

Log loss has a property the whole book leans on. It's a **proper scoring rule**: the only way to score best on average is to report what you actually believe (Figure 2.5). Exaggerate, and over many cases you lose. So models trained on log loss tend to come out honest, or *calibrated*.

Training is a walk downhill. Picture log loss as hilly ground, where each combination of weights is a place. Stand in thick fog, feel which way the ground slopes, step downhill, repeat. That's **gradient descent**. Run on our alerts and tested on alerts held back from training, it comes within 1% of the best score any model could reach.

> ● KEY IDEA
>
> **All of machine learning is one loop: predict, measure the surprise, step downhill.**

Training an LLM is the same loop with a vastly bigger model.

Two ways a model fools you

Overfitting is memorising instead of learning. A decision tree allowed twenty levels of yes-or-no questions nearly memorises its training alerts

(loss 0.005) and does badly on new ones (0.66). You only see it by testing on data the model never trained on.

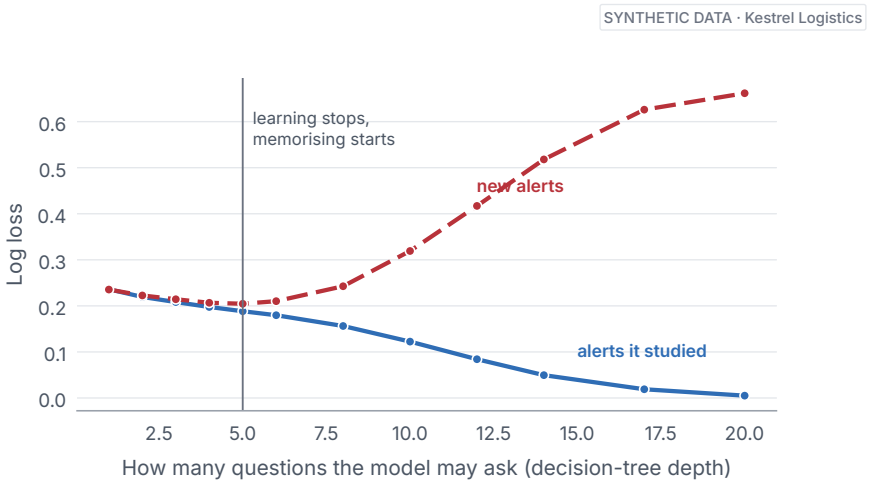


Figure 2.6. A decision tree allowed only a few questions learns general patterns. Allowed dozens, it memorises individual training alerts: its loss on them approaches zero while its loss on new alerts climbs.

Leakage is a clue that secretly contains the answer. Train with analysts' notes and results look amazing, because "confirmed phishing" is written *after* someone decided. Ask of every clue: *would I have this value at the moment of the decision?*

KEEP THESE

- A **probability** is a claim about how often something happens among cases like this one.
- Rare things stay rare even after the alarm goes off.
- New odds = old odds × likelihood ratio.
- All of machine learning is one loop: predict, measure the surprise, step downhill.

EXERCISES

1. A spam filter catches 98% of spam and wrongly flags 1% of real email. If only 1% of incoming email is spam, what fraction of flagged emails are real? Use counts.

Next: our model says 0.8. Does 0.8 really mean 80%?

CHAPTER 3

Calibration: when 0.8 really means 80%

When you can measure what you are speaking about, and express it in numbers, you know something about it.

— Lord Kelvin, 1883

"If a model says 80%, does that mean 80%?" It sounds silly until it costs you money. Nothing in a model's maths forces its 0.8 to match reality. It matches only if the model was built well and *checked*, and even then it can drift. Checking is the most important habit in this book.

BY THE END OF THIS CHAPTER

- Say what *calibrated* means, and why it differs from accurate.
- Check calibration with a reliability diagram and ECE.
- Fix a miscalibrated model, and check it group by group.

The weather forecaster's secret

In the 1970s, researchers checked American weather forecasters against what actually happened. When they said "30% chance of rain", it rained on about 30% of those days; when they said 70%, about 70% (Murphy and Winkler 1977). Most people are nothing like this: ask for a range they're "90% sure" of, and they're right far less than 90% of the time.

Forecasters aren't good because they're always right. They're good because their numbers mean what they say.

KEY IDEA

A model is calibrated when its 80% happens about 80% of the time.

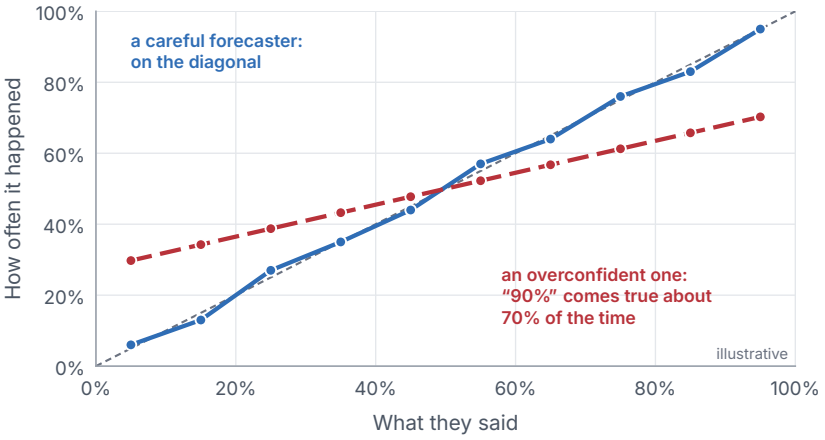


Figure 3.1. Two forecasters. For each thing they said ("30%", "70%" and so on), we count how often it actually happened. The careful forecaster sits on the diagonal: their 70% happens about 70% of the time. The overconfident one says 90% about things that happen only about 70% of the time.

Ranking is not calibration

A model's numbers can be good at two different things. **Ranking**: do attacks get higher numbers than harmless alerts? The **AUC** measures it: the chance a random attack scores above a random harmless alert (1 is perfect, 0.5 a coin toss). **Calibration**: when the model says 0.3, are three in ten of those alerts attacks?

Both models in Figure 3.2 have AUC 0.888. Model B says 40% about alerts that are attacks only a few percent of the time. If you only sort, B is as good as A. The moment you put a threshold on its numbers or multiply them by a cost, it misleads you. Leaderboards report ranking scores; calibration is often not measured at all. For decisions, it's usually what matters most.

● KEY IDEA

Ranking well and being calibrated are different skills; decisions need both.

How to check

Do what the weather researchers did. Take predictions where you know the outcome, group them by what the model said (0–10%, 10–20%...), and in each group compare the average prediction with how often it happened.

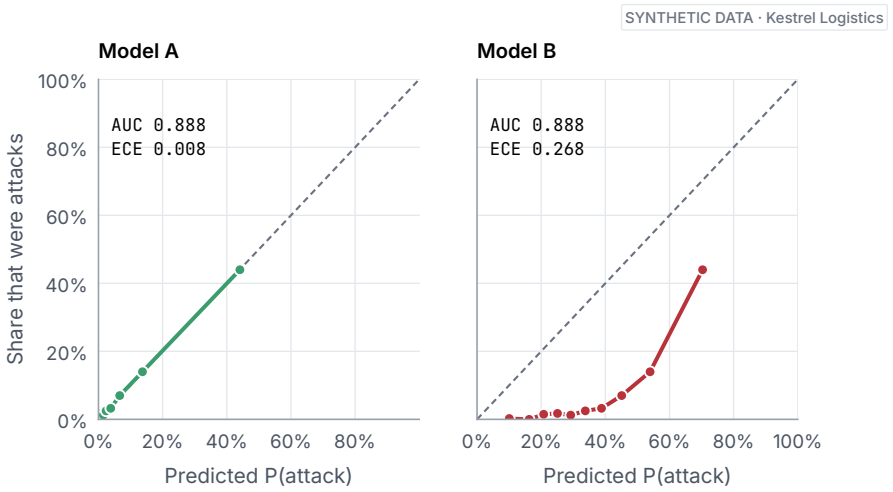


Figure 3.2. Two models, identical at ranking (same AUC) but not at calibration. Model A is the logistic regression from Chapter 2. Model B is the same model with its probabilities distorted: it still sorts the alerts in the same order, but its numbers no longer mean anything.

Plotted, that's a **reliability diagram**: points on the diagonal mean the numbers can be trusted. Always look at how many cases sit in each group; a dot built on twelve alerts is weak evidence.

For one number, use the **expected calibration error (ECE)**: the average gap between "said" and "happened", weighted by how many predictions each group holds (Naeini et al. 2015). Our logistic regression's ECE is 0.008, under one percentage point; Model B's is 0.27. But ECE is an average, so it can hide a badly calibrated corner, and the high end, where the dangerous cases are, is exactly such a corner.

How calibration breaks, and how to fix it

Rebalancing the data. The most common cause, usually well meant. Attacks are rare (8%), so someone trains on a 50/50 mix "so the model sees more attacks". The ranking stays fine, but the model learns a world where attacks are common: its average prediction becomes 29%. Put a threshold on that and the team drowns.

Overfitting pushes probabilities towards 0 and 1. **Very large neural networks** tend to be overconfident (Guo et al. 2017). And **the world changing**, say an attack campaign doubling the base rate, breaks a model that was calibrated last month.

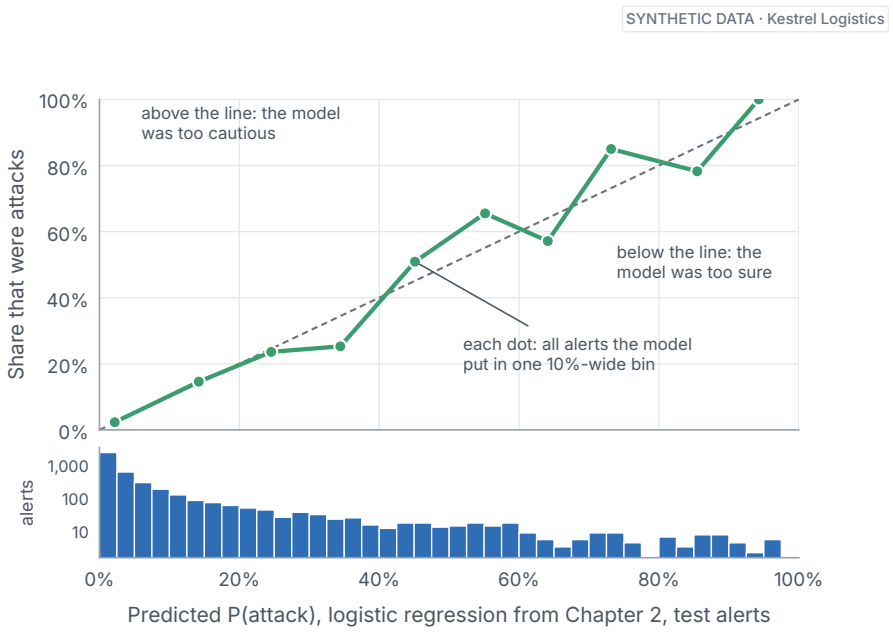


Figure 3.3. The reliability diagram of our Chapter 2 model on alerts it never saw. Points on the dashed diagonal mean the model's numbers can be trusted as they are. Underneath, a histogram (log scale) shows how many alerts fall in each part of the range: most are near zero, and only a few are above 50%.

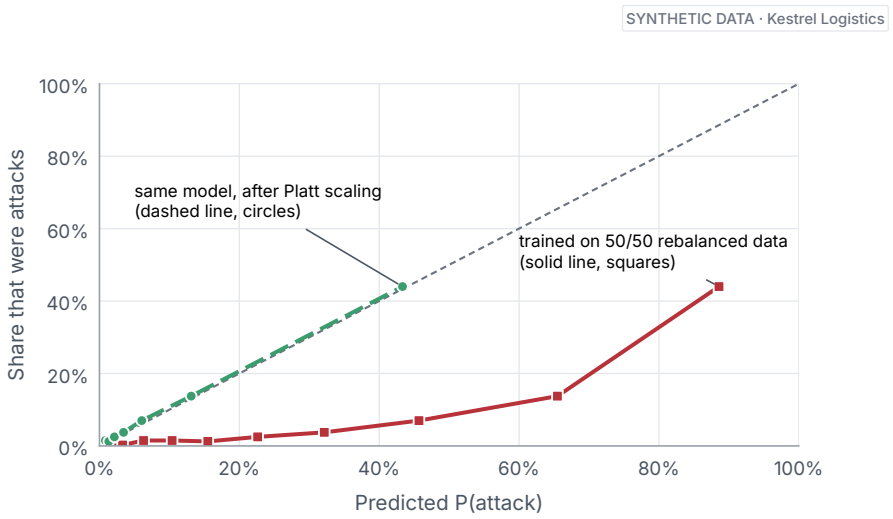


Figure 3.4. A model trained on 50/50 rebalanced data (red, solid line with squares) thinks attacks are about ten times more common than they are, and its probabilities sit far below the diagonal. After Platt scaling fitted on a separate calibration set (green, dashed line with circles), the same model is well calibrated again.

The good news: a model that ranks well but is miscalibrated is usually easy to fix, without retraining. Keep back a **calibration set** it never trained on, and learn a small correction on top:

- **Platt scaling** fits a stretch and a shift on the model's log-odds (Platt 1999). The usual first choice.
- **Temperature scaling** fits one number that softens overconfident answers. The standard fix for neural networks.
- **Isotonic regression** fits any rising staircase (Zadrozny and Elkan 2002). Flexible, but needs more data.

On the rebalanced model, Platt brings ECE from 0.216 to 0.010. Temperature does nothing (0.215), and the reason is worth knowing: rebalancing didn't make the model *overconfident*, it *shifted* every prediction up. A temperature can only stretch around the middle.

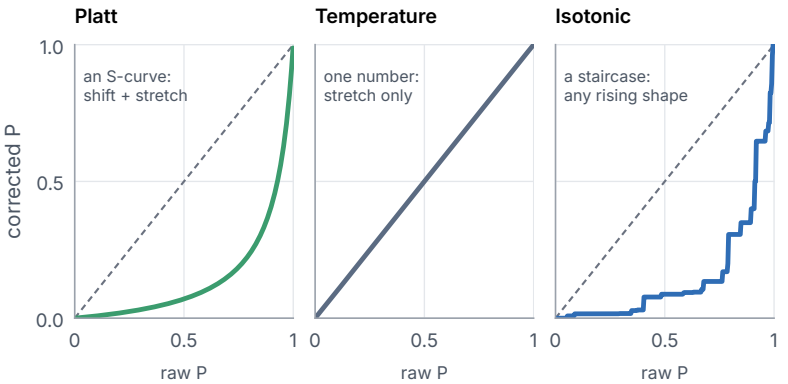


Figure 3.5. The three corrections, each fitted to the rebalanced model on the calibration set. Platt scaling learns an S-curve that can shift and stretch. Temperature scaling learns one number that only stretches around the middle, and here it barely changes anything. Isotonic regression learns any rising staircase.

● KEY IDEA

Temperature fixes overconfidence. It can't fix a wrong base rate.

Calibrated on average, wrong for one group

A model can be well calibrated overall and wrong for one group. Take a model that can't tell which system raised each alert. Overall ECE: an excellent 0.005. But for alerts from laptops and servers it says 6.8% when the truth is 10.4%, and for data-leak alerts 7.7% when the truth is 4.8% (Figure 3.6). The errors cancel in the average, and the policy quietly closes real laptop attacks.

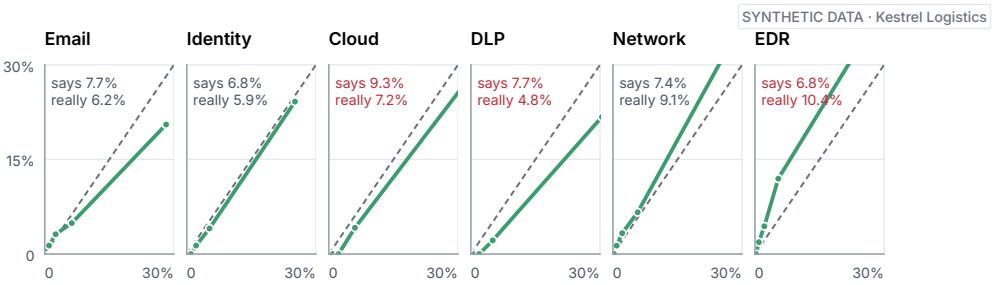


Figure 3.6. Calibration checked separately for each alert source. Across all sources together, this model is well calibrated, with an ECE of about half a percentage point. But it overestimates the risk of DLP and Cloud alerts and underestimates EDR alerts, which really are more dangerous here. Averaging hides both mistakes.

● KEY IDEA

Check calibration inside every group you'll treat differently, not just on average.

That means sources, customer tiers, languages, regions, and, when a model affects people's lives, groups of people.

⚠ WHERE THIS BREAKS

Calibration only holds for data like the data you measured it on, and it fades as tools, attackers and people change. And it says nothing about any single prediction: a perfectly calibrated 70% is still wrong three times in ten. Keep a fresh calibration set and recheck on a schedule.

● KEEP THESE

- A model is calibrated when its 80% happens about 80% of the time.
- Ranking well and being calibrated are different skills; decisions need both.
- Temperature fixes overconfidence. It can't fix a wrong base rate.
- Check calibration inside every group you'll treat differently, not just on average.

EXERCISES

1. Explain to a friend, with no technical words, the difference between a model that ranks well and one that's calibrated.

Next: we have probabilities we can trust. Where do we put the threshold?

CHAPTER 4

From probabilities to actions

Not everything that can be counted counts, and not everything that counts can be counted.

— William Bruce Cameron, 1963

You have probabilities you can trust. Now the question everyone reaches: *where do I put the threshold?* Most people pick 0.5. It's the default in almost every library, and it feels neutral. It isn't. It's a very specific opinion about your business, usually the wrong one.

BY THE END OF THIS CHAPTER

- Turn two costs into one threshold, and see why 0.5 is a special case.
- Treat "ask a person" as a real action.
- Meet the three doors: act, review, escalate.

The umbrella question

The forecast says 30% chance of rain. Do you take an umbrella? Carrying one you didn't need is mildly annoying, call it 1. Getting soaked on the way to a meeting is worse, call it 4. Carrying always costs 1; leaving it costs 4 times the chance of rain. The two are equal at 20%. Above that, carry it. So at 30% you take the umbrella, even though it probably won't rain.

One formula, two costs

Any yes-or-no action can go wrong two ways: a **false alarm** (you acted and didn't need to) and a **miss** (you didn't act and should have).

● KEY IDEA

Act when $P > C_{fa} \div (C_{fa} + C_{miss})$: the false-alarm cost over the sum of both costs.

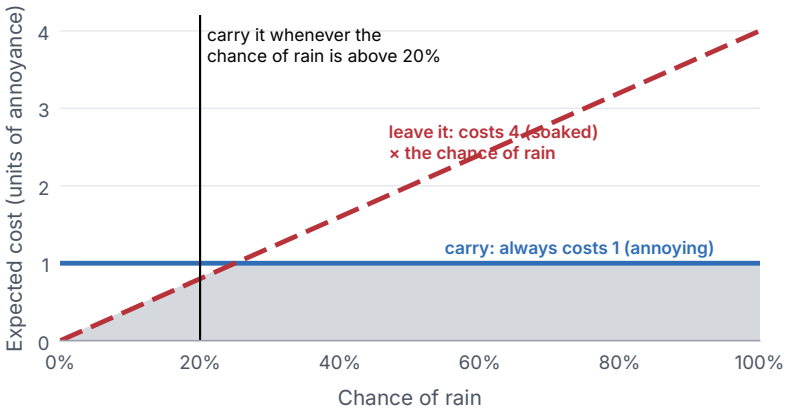


Figure 4.1. The umbrella decision. Carrying always costs 1. Leaving it costs 4 times the chance of rain. The lines cross at 20%: above that, carry it.

The umbrella: $1 \div (1 + 4) = 0.2$. When the two costs are *equal*, the formula gives 0.5. That's where 0.5 comes from: it's right only when every mistake costs the same, which is almost never true of anything worth building a model for.

● KEY IDEA

0.5 is not neutral: it's the threshold for a world where every mistake costs the same.

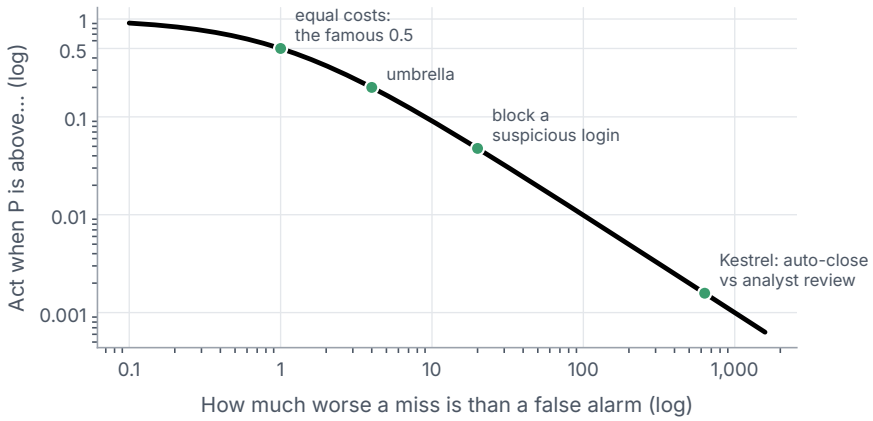


Figure 4.2. The right threshold for any ratio between the cost of a miss and the cost of a false alarm (both axes on a log scale). Equal costs give 0.5. As misses get more expensive, the threshold drops fast.

Different costs, different thresholds (Figure 4.2). Letting a security alert close itself with nobody looking sits far in the corner: a miss costs about 630 times a quick human check, so the threshold drops to about 0.0016.

GOING DEEPER optional

Where the formula comes from. Acting costs $(1 - P) \times C_{fa}$ on average, since you pay only if the case was harmless. Not acting costs $P \times C_{miss}$. Act when the first is smaller, and rearrange: $P > C_{fa} / (C_{fa} + C_{miss})$ (Elkan 2001).

Does it work?

For every alert, the team decides: block or allow? Wrongly blocking something harmless costs about $200 in lost work; letting a threat through costs about $4,000. The formula says block above 0.048. A brute-force search over 300 thresholds lands almost exactly there. Using 0.5 instead costs 2.3 times as much.

Now see why calibration came first. Apply the same formula to the rebalanced model from Chapter 3, whose numbers are too high, and the cost is 62% higher. Its own best threshold is far away, findable only by trial and error, again every time the costs change.

KEY IDEA

With calibrated probabilities, costs set the threshold directly. Without them, you're guessing.

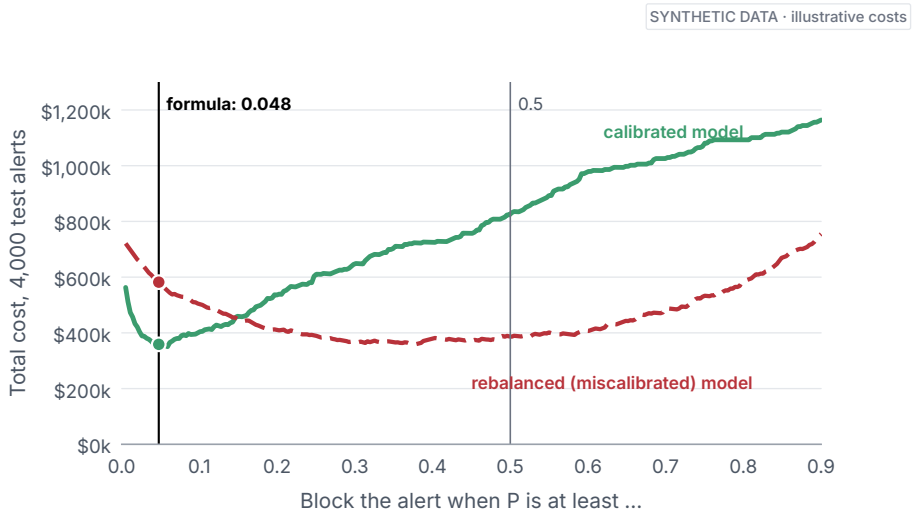


Figure 4.3. Total cost on the test alerts at every threshold. For the calibrated model (green, solid line), the formula's threshold is near the lowest point of the curve. For the same model trained on rebalanced data (red, dashed line), the same threshold lands far from its lowest point.

At the cost-based threshold, only about one block in five is a real threat. Is that poor? Only the costs can say. Each block is worth it if the alert is better than about 1 in 21 to be real, because a miss costs 20 times a false alarm. Precision or recall alone, without costs, is half a story.

"I don't know" is an action too

There's almost always a third option, and it's the most useful: *don't decide; ask a person*. In 1970 the engineer C. K. Chow showed that a machine allowed to refuse its least certain cases makes far fewer errors (Chow 1970).

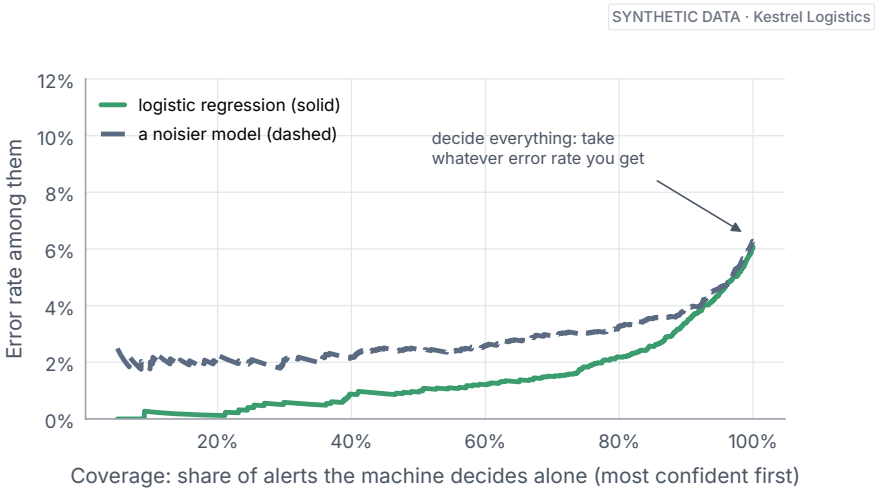


Figure 4.4. Coverage against risk. Reading from right to left: if the machine decides every alert, you take its full error rate. Let it decide only the alerts it's surest about and send the rest to people, and the error rate among its decisions falls steeply. A better model (green, solid line) gives you more coverage at the same risk.

Deciding every alert, the model gets 6.1% wrong. Deciding only its surest 80%, 2.2%. Its surest half, 0.9% (Figure 4.4). This is one of the most useful pictures you can show a manager. It replaces "how good is the model?" with the question they actually have: *how much of this work can we safely hand over, at what error rate?*

● KEY IDEA

Let a model decide only the cases it's surest about, and its error rate falls steeply.

Three doors

Put cost-based thresholds and "ask a person" together and you get the design used for the rest of the book. **Act**: the machine handles it alone. **Review**: a person checks it in normal work. **Escalate**: a person is called in now. The boundaries come from the same cost arithmetic; Chapter 11 sets them, and adds the part skipped here, what to do when there aren't enough people for the middle door.

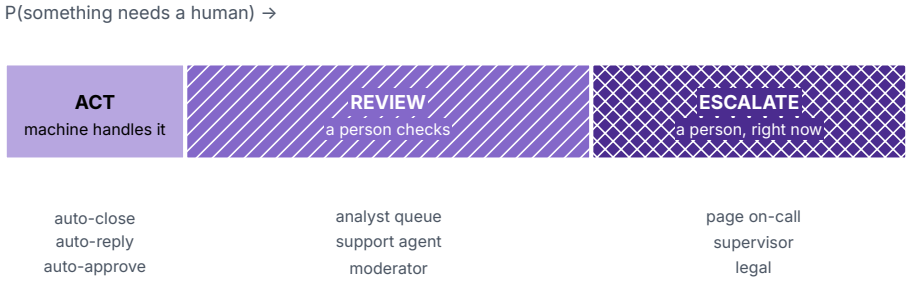


Figure 4.5. The three doors. At one end the model is confident nothing's needed, so the machine handles it. At the other end it's confident something's badly wrong, so a person is called in immediately. In between, where it's unsure, a person looks when they can.

⚠ **WHERE THIS BREAKS**

Costs are guesses, and the formula treats guesses like facts. Try each cost at half and double; if the threshold barely moves, stop arguing about the exact number. Costs also differ by case: a false alarm on the CEO's laptop costs more than on a test server. And some costs aren't money at all: trust, fairness, safety, law. Discuss them openly, or someone's default settings decide them for you.

SET THE THRESHOLD

The sales team says a wrongly blocked customer email costs $800, not $200. Where does the blocking threshold go now?
(800 ÷ 4,800 ≈ 0.17. Fewer alerts get blocked and more threats get through. The model didn't change; the business's priorities did, and the threshold should follow them.)

KEEP THESE

- Act when $P > C_{fa} \div (C_{fa} + C_{miss})$: the false-alarm cost over the sum of both costs.
- 0.5 is not neutral: it's the threshold for a world where every mistake costs the same.
- With calibrated probabilities, costs set the threshold directly. Without them, you're guessing.
- Let a model decide only the cases it's surest about, and its error rate falls steeply.

EXERCISES

1. A hospital test flags a serious but treatable condition. Treating someone who doesn't have it costs $500; missing someone who does costs $50,000. What's the treatment threshold?

Next: how modern AI models work inside, from a single neuron to an LLM, and why that matters for deciding.

CHAPTER 5

How modern AI models work

Any sufficiently advanced technology is indistinguishable from magic.

— Arthur C. Clarke, 1973

"Deep learning" and "LLM" sound like magic. They're four plain ideas stacked together, and each one matters for deciding. This chapter covers them, then shows why "just ask the LLM" is an awkward way to make a decision.

BY THE END OF THIS CHAPTER

- Explain layers, embeddings and attention in plain words.
- See why big models become overconfident.
- Explain why asking an LLM for a decision is awkward.

Idea one: layers bend the line

A single artificial neuron is the logistic regression from Chapter 2: weigh the clues, add them up, squash into a probability. It can only cut the world in two with a straight edge. But some behaviour is suspicious when it's unusual in *any* direction (far too much activity or far too little), and no straight line separates the middle from the edges.

So send the clues through a layer of neurons first. Each draws its own line, and the next combines them; three lines make a triangle, twelve something like a circle. Each hidden neuron *bends* its answer first (negative becomes zero), and enough bends can draw almost any shape. A network with several hidden layers is **deep**. Training is the same downhill walk; the error is passed back through the layers like shared-out blame (**backpropagation**) (Rumelhart et al. 1986).

Depth matters because a deep network takes raw input, like pixels or characters, and *learns its own clues* (LeCun et al. 2015). On neat hand-made features there's nothing left to find: a small network matches logistic regression (0.180 against 0.175), and a big one does worse.

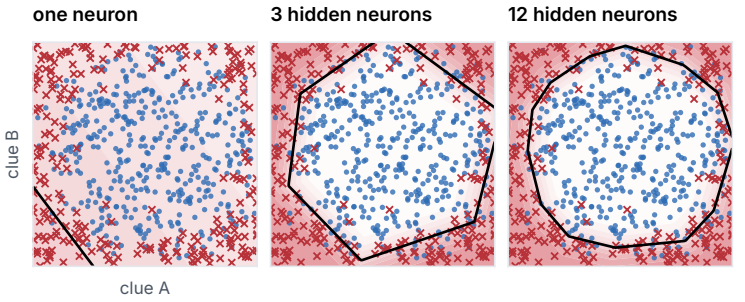


Figure 5.1. A toy problem. Blue dots are normal activity; red crosses surround them on every side. One neuron (left) can only draw a straight line through it. With a hidden layer of three neurons (middle) the boundary becomes a triangle; with twelve (right), close to a circle.

● KEY IDEA

Deep networks are worth their cost on raw, messy input. On neat features, start with something simple.

Idea two: meaning as geometry

A network does arithmetic, so words must become numbers. The trick came from linguistics: *you know a word by the company it keeps.* Count which words appear near each word, squash the counts into a short list of numbers, and words used alike get similar lists. That list is an **embedding**. Similar documents land near each other too, which is the engine inside search and RAG.

But embeddings capture what things are *about*, not the exact numbers inside them. Predicting from each alert's 50 most similar past alerts gives honest probabilities (ECE 0.015) but poor ranking (AUC 0.68, against 0.89): to an embedding, a threat score of 0.05 and 0.95 look alike.

Idea three: words that look at each other

"invoice, not phishing" clears an alert; "phishing, not invoice" raises one. Same words. **Attention** fixes this: *every word asks which other words should change its meaning, and listens to them in proportion.* Trained on tiny notes where "not" flips the meaning, attention learns on its own to look at "not", reaching 100% accuracy where word-counting tops out at 84%.

A **transformer** is attention stacked dozens of times (Vaswani et al. 2017). One distinction matters for this book. A **reader** lets every word see the

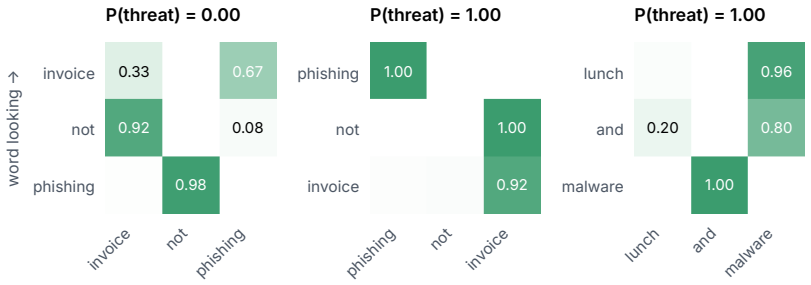


Figure 5.2. What each word attends to after training. In "invoice not phishing", the word "phishing" puts nearly all its attention on "not". Nobody told the model about negation; it found that looking at the word before was the cheapest way to get the answers right.

whole input, which suits understanding and deciding. A **writer** lets each word see only earlier words, which writing requires. LLMs are writers.

Idea four: scale, and a side effect

Train a huge writer to guess the next word of everything ever written, and it picks up grammar, facts and some reasoning (Kaplan et al. 2020). Then tune it on good answers and on which answers people prefer (Ouyang et al. 2022). People prefer answers that sound sure. OpenAI's own report found GPT-4's confidence well calibrated before that tuning, and clearly less so after (OpenAI 2023).

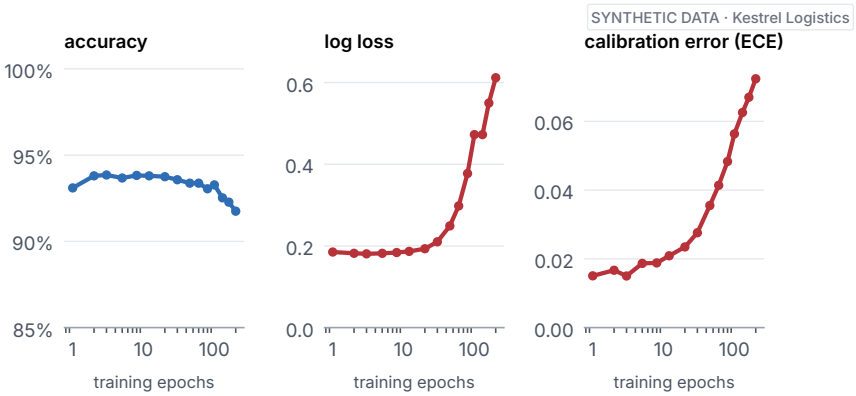


Figure 5.3. The same training run, on test alerts. Accuracy (left) barely moves. Log loss (middle) and calibration error (right) are flat for a while, then climb steeply. Watch only accuracy and you'd never notice.

There's a deeper version you can watch happen. Train a network far bigger than the job, for too long: accuracy barely moves, but calibration error climbs from 0.019 to 0.072 (Figure 5.3). Once most training cases are right, the only way to lower log loss is to push answers towards certainty. The

model becomes certain of everything it saw, noise included. One temperature (Chapter 3) brings the error back to 0.046.

> ● KEY IDEA
>
> **Flexible models trained to make their training loss as small as possible learn to be too certain.**

How an LLM writes, and why that's awkward for deciding

An LLM writes one **token** (a word or piece of a word) at a time: read everything so far, give every possible next token a probability, pick one, append it, repeat. Three consequences matter for decisions.

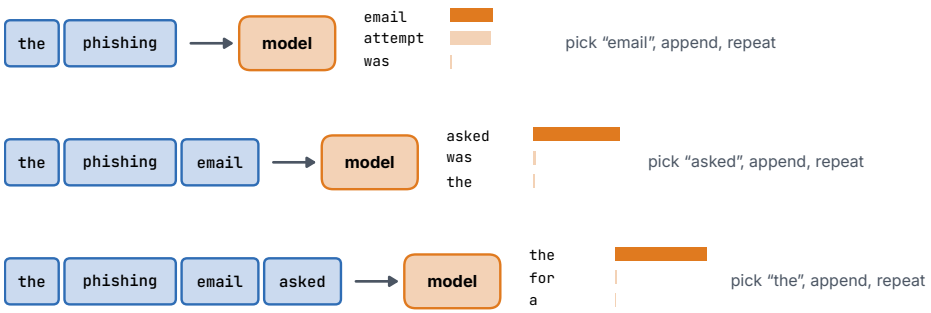


Figure 5.4. The generation loop. The model reads everything so far, produces a probability for every possible next token, one gets picked, it's appended, and the loop runs again.

Length is time and money. Each token waits for the one before. A one-word answer takes about 0.5 seconds in our illustrative figures; 1,500 tokens of step-by-step reasoning, about 25.

The same question gets different answers. Picking usually involves a weighted dice roll. Asked about forty borderline alerts twenty times each, a typical LLM flipped its verdict on 39 of them. Setting temperature to zero hides the uncertainty; it doesn't remove it.

The answer is buried in text. Ask for JSON and 4.7% of answers come back broken, chatty or with invented fields. "JSON mode" guarantees the *shape*, not the *truth*: a perfectly formatted "benign" for a real attack.

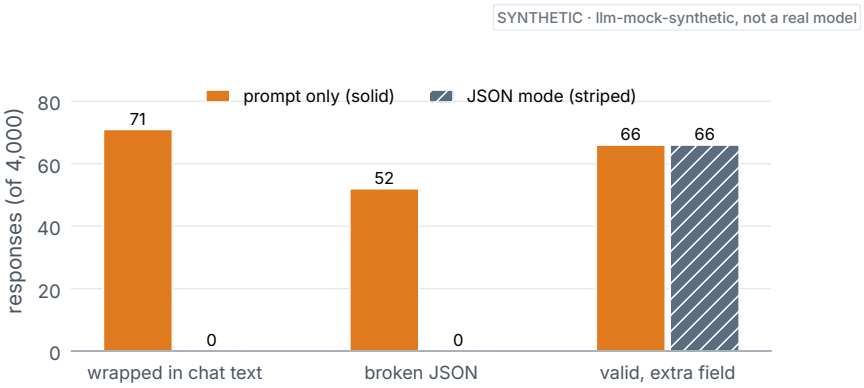


Figure 5.5. What came back when we asked for JSON, over 4,000 alerts. With only the prompt, some responses were wrapped in friendly chat, some were cut off mid-object, and some were valid JSON with a field nobody asked for. With JSON mode on, the first two problems vanish. The third doesn’t.

KEY IDEA

Constrained decoding makes the answer parse. It doesn’t make it right.

Three kinds of “confidence”

Say the JSON is perfect: `"verdict": "malicious", "confidence": 0.95`. Can you put a threshold on that 0.95?

The **written confidence** is just text: the stand-in LLM used only 6 different values, mostly 0.9 and above, and it barely ranks alerts (AUC 0.66). The **token probability** ranks better (0.84) but is overconfident, and it’s the probability of a *word*, not of an answer: “malicious”, “Malicious” and ” malicious” are different tokens. A **decision model’s probability** is designed to be the answer itself (Figure 5.6). That’s the gap System One models aim at.

KEY IDEA

The probability of a token isn’t the probability of an answer.

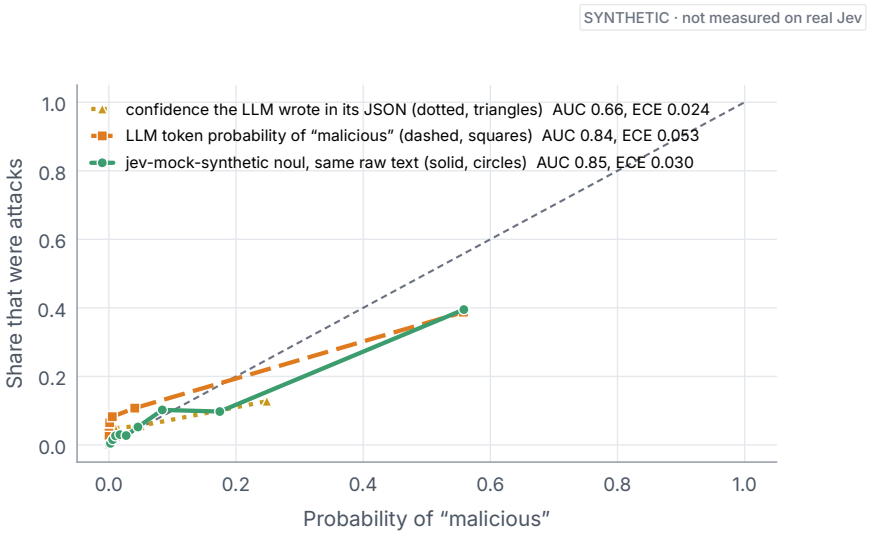


Figure 5.6. Reliability of three "confidences" on the same test alerts, all reading the same raw text. The stated confidence barely separates attacks from harmless alerts. The token probability ranks well but is overconfident. The decision model ranks best and also sits close to the diagonal. Synthetic: the stand-ins reflect design choices, not measurements of real models.

KEEP THESE

- Deep networks are worth their cost on raw, messy input. On neat features, start with something simple.
- Flexible models trained to make their training loss as small as possible learn to be too certain.
- Constrained decoding makes the answer parse. It doesn't make it right.
- The probability of a token isn't the probability of an answer.

EXERCISES

1. A vendor's classifier is "97% accurate" and returns a confidence with every answer. You plan to act automatically above 0.9. What would you ask for first?

Next: retrieval and agents, and why nearly every weak point in them is a small decision.

CHAPTER 6

RAG, agents, and where they break

Simplicity is prerequisite for reliability.

— Edsger W. Dijkstra, 1975

Ask an LLM what your company's backup account is allowed to do, and you'll get a confident, well-written, invented answer. It has never seen your documents. So we hand it the right pages, then give it tools and a loop so it can act. That's where most of today's AI excitement is, and most of the failures nobody notices. Watch for one pattern: nearly every weak point is a small decision.

BY THE END OF THIS CHAPTER

- Explain RAG, and why retrieval is the weak link.
- See that most of an agent's work is deciding.
- Stop a planted sentence from steering a decision.

Give it the right page

Retrieval-augmented generation (RAG): before the model answers, search your documents for the most relevant passages, paste them into the prompt, and ask it to answer from them (Lewis et al. 2020). On a test collection of company policies mixed with look-alike noise, the search put the right passage first 81% of the time.

The part that decides whether RAG works isn't the LLM. If the right passage isn't found, the LLM builds the likeliest-sounding answer from what it *can* see: fluent, with a source, and wrong.

● KEY IDEA

A RAG system can only be as good as its retrieval. Measure retrieval first.

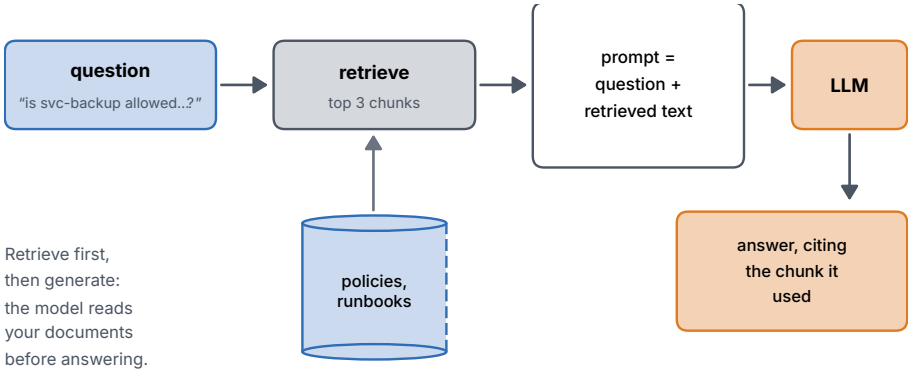


Figure 6.1. Retrieval-augmented generation. The question goes to a search step first, which pulls the most relevant chunks from your documents. The LLM sees the question plus those chunks, and answers from them, ideally citing which one it used.

Some questions have no answer in the documents. A good system says "I don't know", and the signal is simple: how well the *best* passage matches. With a threshold on that score, the system answers 72% of answerable questions and only 8% of unanswerable ones. A system built for *writing* turns on a small yes-or-no decision with a threshold.

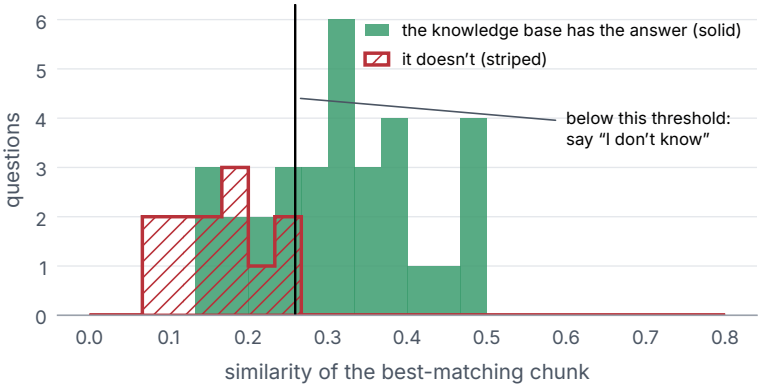


Figure 6.2. The similarity of the best-matching chunk, for questions the knowledge base can answer (green, solid bars) and questions it can't (red, striped outline). They overlap, but not much. Below a threshold, the system should decline to answer.

A model in a loop

An **agent** is a model in a loop, with tools: look, decide, act, look again. A "tool call" sounds like the model pressing buttons, but it only *writes down* which tool it wants and with what arguments (Yao et al. 2023); your code

checks it and runs it. That request is a choice from a fixed list: a typed decision wearing text.

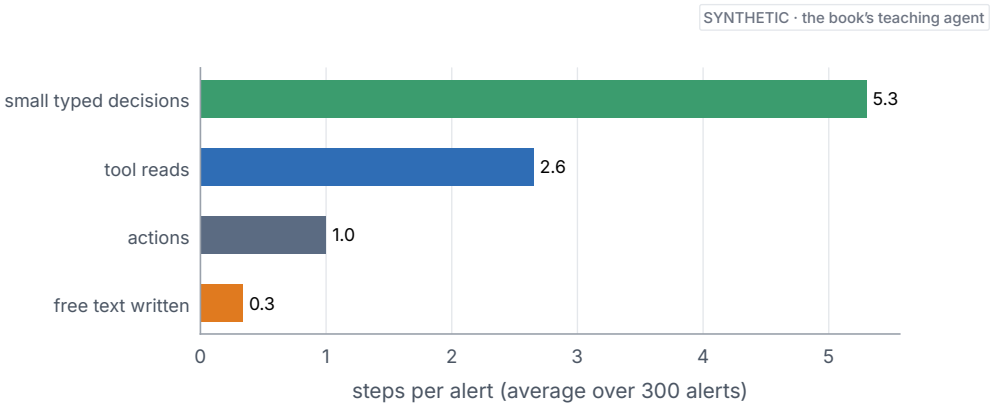


Figure 6.3. What the agent spent its steps on, averaged over 300 alerts. Small typed decisions dominate. Free text, the thing LLMs are famous for, is a small minority of the work.

The book's security agent decides which evidence to gather, whether it has enough, and the verdict, then acts. Counting its steps, decisions were 57% of all steps; writing, 4% (Figure 6.3). Count any agent you build or buy and you'll likely see the same: deciding is the everyday work.

● KEY IDEA

Watch an agent work, and most of what it does is decide.

Errors multiply

If each step is right 95% of the time, a ten-step task succeeds 0.95 multiplied by itself ten times: 60%. Twenty steps: 36%. That's why agents dazzle in demos (a few steps on an easy case) and stumble in real use (twenty steps on an odd one). Decisions are most of the steps, so most of the risk. A decision that comes with a calibrated probability can be caught and sent for review; one that comes back as a confident sentence can't.

Inputs can lie

Agents read text other people wrote. An attacker who gets text in front of the model can plant instructions or false claims: **prompt injection** (Greshake et al. 2023). It isn't only an LLM problem; any model that reads attacker-written text can be steered.

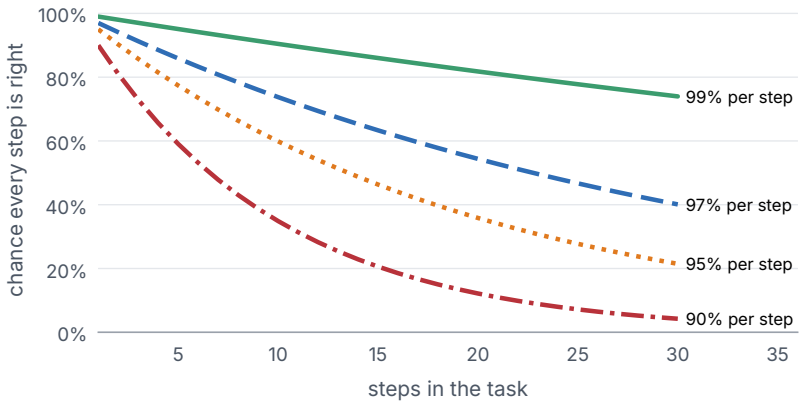


Figure 6.4. The chance that every step of a task goes right, for different per-step reliabilities. Small per-step error rates become large task failure rates as the number of steps grows.

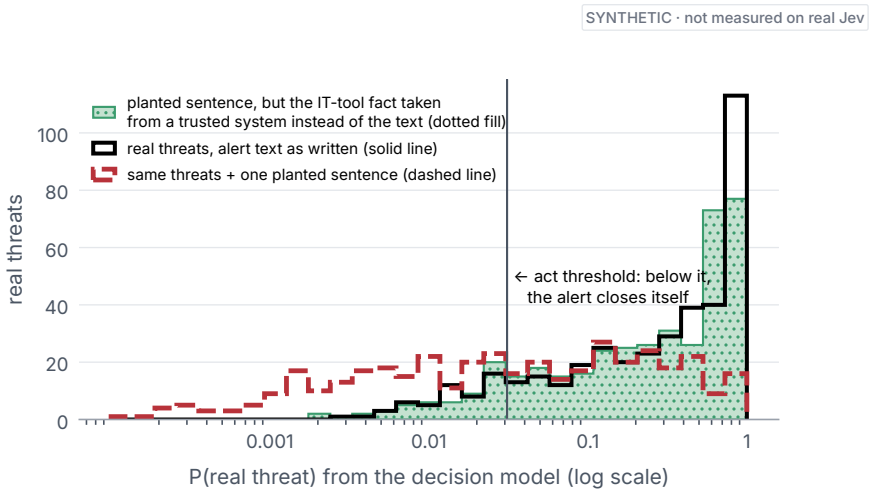


Figure 6.5. How likely the decision model thinks each real threat is, reading the alert text as written (black, solid line) and with one planted sentence (red, dashed line). The sentence pulls many real threats below the auto-close threshold. When the same fact comes from a trusted system instead (green, dotted fill), the planted sentence does nothing.

Some alerts say “Matches approved IT tooling (change ticket on file)”, a strong sign of harmlessness. Plant that sentence in a file name, and the share of real threats that would close unseen jumps from 13% to 50% (Figure 6.5). The fix is design, not a smarter model: ask the change-ticket system itself, and pass its answer as a separate field the attacker can’t write. Then only 14% close.

> **KEY IDEA**
>
> **Take the facts that lower suspicion from systems attackers can’t write to.**

The decision layer

Nearly every protection in this chapter is the same thing: a check, turned into a decision, with a threshold chosen from what mistakes cost. Put them together and you have a **decision layer** between what an agent proposes and what actually happens. It checks proposals against what’s allowed, asks typed questions, applies cost-based thresholds, takes risky facts from trusted systems, and sends each case through one of three doors: act, review or escalate. Whenever anything fails, from a timeout to a parse error, the case goes to a person.

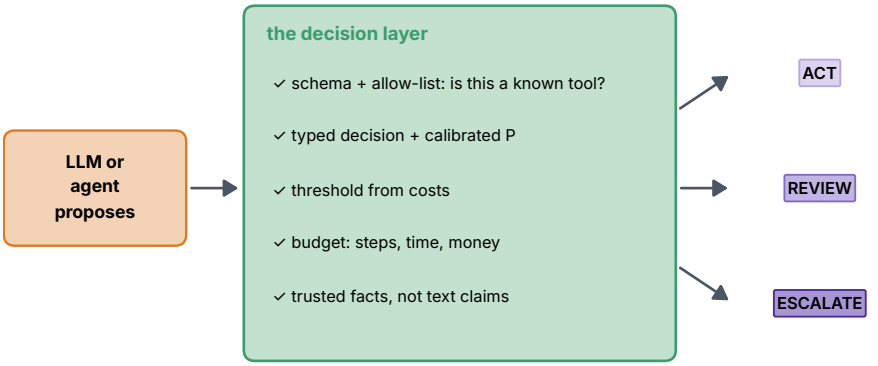


Figure 6.6. A decision layer between what an agent proposes and what actually happens: schema and allow-list checks, a typed decision with a calibrated probability, a threshold from costs, a budget and trusted facts, then one of three doors.

None of that needs a smarter model. It needs decisions fast enough to use everywhere, cheap enough to ask often, and calibrated enough to put a threshold on.

SET THE THRESHOLD

An agent can close an alert, add a note, block a sender's domain, or cut a laptop off the network. Which should it do automatically when confident, and which always need a person?
(Closing and noting are cheap and reversible: fine with a calibrated threshold. Blocking a domain is reversible but can cost business: stricter threshold, plus a list of partners never to block. Cutting off a laptop stops someone's work and may destroy evidence: a person decides.)

KEEP THESE

- A RAG system can only be as good as its retrieval. Measure retrieval first.
- Watch an agent work, and most of what it does is decide.
- Take the facts that lower suspicion from systems attackers can't write to.

EXERCISES

1. A vendor says their agent gets 90% of steps right, and a typical task is fifteen steps. What task success rate would you expect?

Next: the idea behind System One models, and what we actually know about Jev.

CHAPTER 7

Fast and slow: System One models and Jev

A wise man, therefore, proportions his belief to the evidence.

— David Hume, 1748

TypeSafe didn't call Jev a better LLM. It called it the first of a new class: a *System One* model (TypeSafe AI 2026). Is that a real idea, or a clever name? Both. This chapter explains the idea, then separates what we actually know about Jev from what's only claimed.

BY THE END OF THIS CHAPTER

- Describe System 1 and System 2, in people and in software.
- Route cases by doubt between a fast and a slow system.
- Sort claims about a new model into three piles.

Two ways of thinking

What's 2 + 2? The answer was just there. Now 17 × 24. That felt different: you had to stop and work in steps. The psychologist Daniel Kahneman called these **System 1**, fast and automatic, and **System 2**, slow and effortful (Kahneman 2011). System 1 runs almost everything you do and is usually right, but it can be confidently wrong. (A bat and a ball cost $1.10; the bat costs $1.00 more than the ball. The ball isn't 10 cents (Frederick 2005).)

Software maps onto this surprisingly well. Rules and classifiers are System 1: fast, good at familiar patterns. LLMs, especially "reasoning" ones, are the nearest thing to System 2: slow, general, and paid for by the token. For years the industry has used System 2 machinery for everything, including the thousands of quick judgements a day that System 1 handles well.

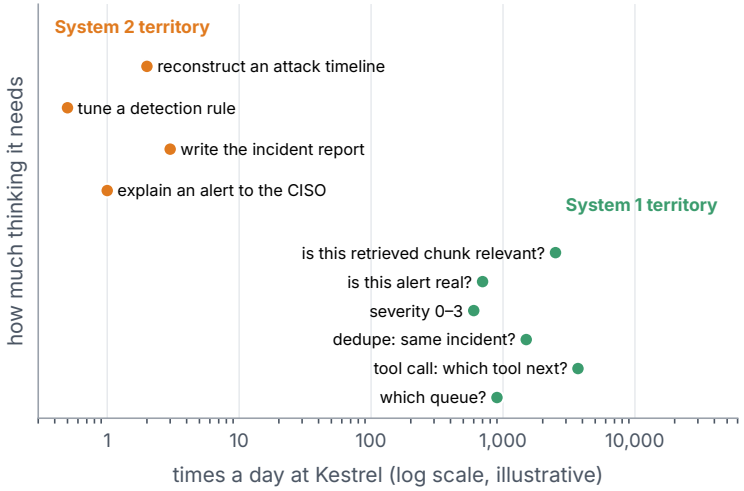


Figure 7.1. Tasks in and around a SOC, placed by how often they happen and how much thinking each needs (illustrative). The frequent ones are grouped at the bottom right: quick judgements made hundreds or thousands of times a day. The few that need real reasoning happen rarely.

● KEY IDEA

Most of the work is System 1 work, but most of the machinery we use is System 2.

Route by doubt

With a fast system and a slow one, how do you split the work? A good expert sorts most cases in seconds and *notices* the ones that don't feel right. So: let the fast system see everything, and send only the cases it's unsure about to the slow one.

On our alerts, sending the 20% that System 1 is least sure about gets 79% of everything a perfect investigation could add. A random 20% gets 3% (Figure 7.2). And you pay the slow price only on that 20%.

● KEY IDEA

A fast system is valuable in two ways: its speed, and knowing which cases it shouldn't decide.

This only works if the fast system is *calibrated*: its 0.5 must really mean "could go either way". An overconfident System 1 keeps the dangerous cases for itself. Kahneman and the psychologist Gary Klein, after years

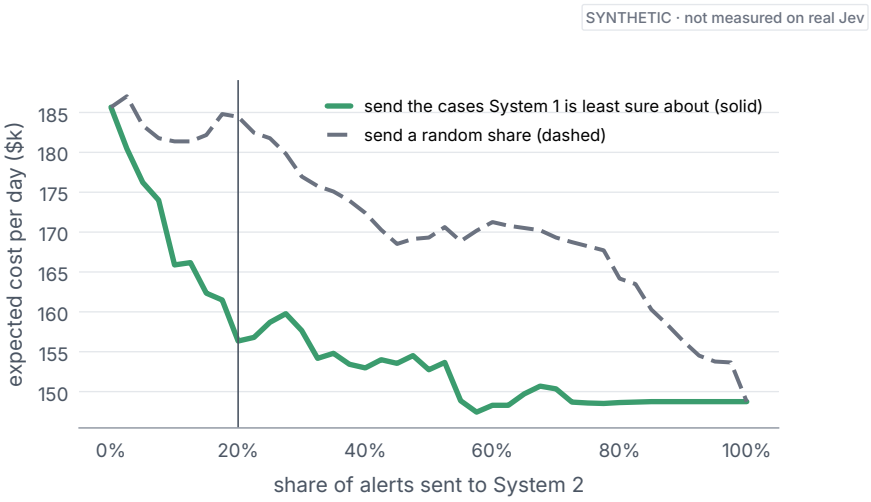


Figure 7.2. Expected cost per day as more alerts go to System 2. Sending the alerts System 1 is least sure about (green, solid line) gets most of the benefit early. Sending a random share (grey, dashed line) gets almost nothing until you send nearly everything.

on opposite sides, agreed on when fast judgement can be trusted: a regular world, lots of practice, and quick, clear feedback (Kahneman and Klein 2009). For machines I'd add a fourth condition: honest doubt.

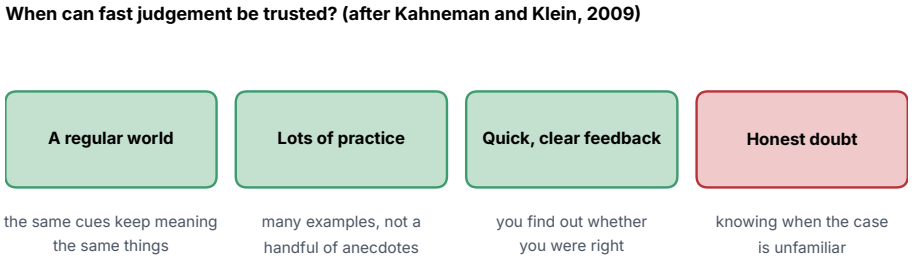


Figure 7.3. Conditions under which fast, intuitive judgement tends to be reliable, adapted from Kahneman and Klein. SOC triage mostly meets the first three. The fourth is what separates good experts, and good models, from confident ones.

What we know about Jev: three piles

I've never called Jev; it was in early access when I wrote this. What I had was its official Python library (anyone can read it), its documentation, the launch coverage and early papers. How much can anyone say from the outside? More than you'd think, less than most articles suggest. The trick is to sort every claim into three piles.

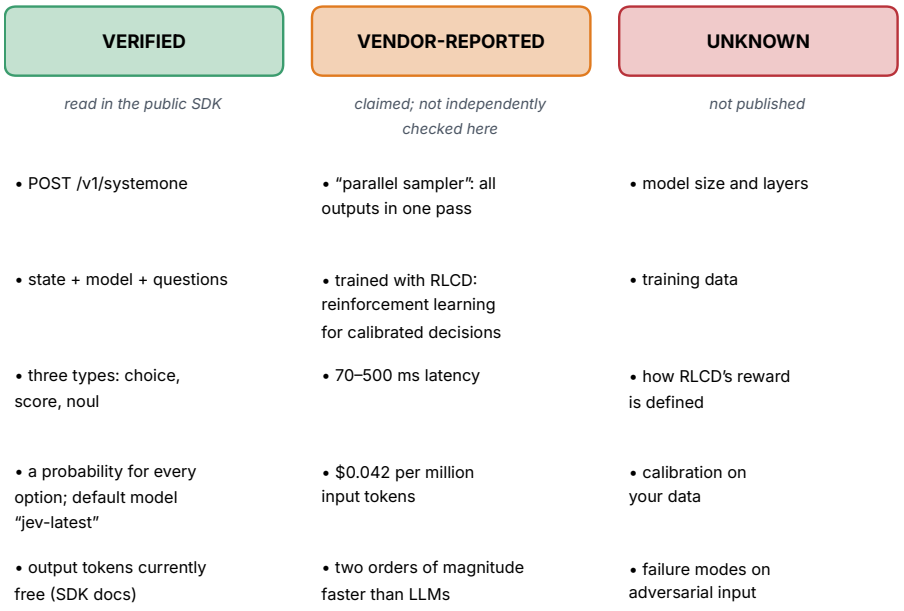


Figure 7.4. Every claim about Jev in this book, sorted into three piles. The first pile you can check yourself today. The second is what TypeSafe says. The third is what nobody outside the company can know yet.

● KEY IDEA

Sort every claim about a new model into three piles: verified, vendor-reported and unknown.

Verified: the interface. You send one request with three things: the **state** (the situation, as text or structured data), the **model** name, and a set of named, typed **questions**. You get back, for each question, a typed answer with a probability for every option. There are three question types: yes/no, pick-one and rate-on-a-scale (Chapter 8).

Vendor-reported: two ideas. A "parallel sampler" that produces all its outputs in one pass instead of word by word, which TypeSafe says is where the speed comes from. And a training method, **RLCD**, said to reward *calibrated* probabilities. We don't know the details. But Chapter 2 tells you how such a reward could work: a proper scoring rule makes honesty the best strategy. That's my reasoning, not something TypeSafe has published.

Unknown: model size, design and training data; calibration on *your* data; behaviour on attacker-written input; how behaviour changes between versions. Exactly what a decision layer depends on most.

request (what the SDK sends)

```
POST /v1/systemone

{

  "state": "I was charged twice…",

  "model": "jev-latest",

  "questions": {

    "topic":  {"type": "choice", "criteria": {…}},

    "urgent": {"type": "noul", "instructions": …},

    "tone":   {"type": "score", "criteria": […]}

  }

}
```

response

```
{

  "model": "jev-mock-synthetic",

  "answers": {

    "topic":  {"choice": "billing",

               "probabilities": {…}},

    "urgent": {"noul": 0.61},

    "tone":   {"score": 0.22,

               "probabilities": {…}}

  },

  "usage": {"input_tokens": 89, …}

}
```

Figure 7.5. A real request as the official SDK builds it, and the response. The answers here come from the book's stand-in, but their shape is just what the SDK's own data models expect.

Check the numbers against each other

TypeSafe reports a response time of about 70 to 500 milliseconds and a price of $0.042 per million input tokens, with output free. At 500 tokens a decision, that's about $21 per million decisions, against roughly $660 at this book's illustrative mid-range LLM price. Real LLM prices vary a hundredfold, so treat the ratio as a size, not a fact. And always ask of speed claims: *faster than what, doing what?* A non-reasoning LLM asked for a one-word label is far closer than a reasoning one (regolo.ai 2026).

> ● KEY IDEA
>
> **When a vendor gives you several numbers, check whether they agree with each other.**

In TypeSafe's launch demo, Jev played the video game *Doom*, about ten decisions a second at a reported $7 an hour. That's 36,000 decisions an hour, about $0.0002 each, which at the listed price means about 4,600 tokens of game description per decision: plausible. The numbers agree. That doesn't prove them; if they had implied three tokens, or three million, you'd have learned something.

● KEY IDEA

Unknowns about a new model aren't reasons to avoid it; they're reasons to test it on your own data.

⚠ WHERE THIS BREAKS

The two-systems idea is a metaphor, not a fact about the brain, and a System One model isn't "intuitive"; it's a trained function. Routing by doubt fails when the fast system is confidently wrong about a whole kind of case, such as a new attack it has never seen; random audits and drift checks are there to catch what doubt misses. And vendor numbers come from the vendor's tasks, setup and comparisons; early-access models change fast, so pin a version and recheck.

● KEEP THESE

- Most of the work is System 1 work, but most of the machinery we use is System 2.
- A fast system is valuable in two ways: its speed, and knowing which cases it shouldn't decide.
- Sort every claim about a new model into three piles: verified, vendor-reported and unknown.
- When a vendor gives you several numbers, check whether they agree with each other.
- Unknowns about a new model aren't reasons to avoid it; they're reasons to test it on your own data.

EXERCISES

1. List five decisions from your own work that are System 1 work and two that need System 2. What makes the difference?

Next: the three question types, and the design choices that make each work.

CHAPTER 8

Asking the right question: choice, score, noul

Perfection is achieved, not when there is nothing more to add,
but when there is nothing left to take away.

— Antoine de Saint-Exupéry, 1939

A System One model asks you to put every question in one of three shapes: yes or no, pick one, or rate on a scale. "Only three? Real decisions are messier." They are, but look at your own working day: most messy decisions break into a few questions of exactly these shapes. *Is this real? Which team? How urgent?* The mess is usually in how the question is asked, not in the shape of the answer.

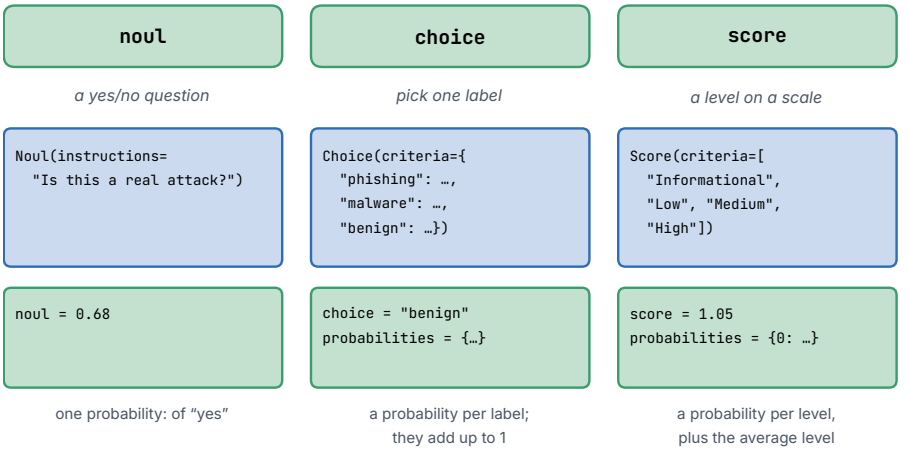


Figure 8.1. The three question types, what you send, and what comes back. A noul returns one probability. A choice returns a probability for every label. A score returns a probability for every level, plus the average level.

BY THE END OF THIS CHAPTER

- Choose between yes/no, pick-one and scale questions.
- Avoid the forced-choice trap and the misleading average.
- Decide using every probability, not just the top answer.

Yes or no: one fact at a time

TypeSafe calls a yes-or-no question a **noul**. It returns one number: the probability of *yes*. It's the most useful type, and the easiest to ask badly. Three habits help.

One fact per question. "Is this a real attack on a critical server?" joins two facts, and a *no* can't tell you which was false. Ask two questions and combine them in your own code, where the logic is visible.

Say exactly what yes and no mean. Is an employee breaking a company rule "malicious"? Is a test attack the company paid for? Spell it out: *yes* means "a real attacker or real malware is involved"; *no* means "normal activity, a paid test, or a false positive".

No double negatives. "Is this not unlikely to be benign?" confuses people and models alike.

Pick one: give every case somewhere to go

A **choice** returns a probability for every label, adding up to 1. That "adds up to 1" has a sting: the model *must* spread all its belief across the labels you gave it. If the right answer isn't there, the probability doesn't vanish; it lands on wrong labels.

I asked the stand-in to label 400 harmless alerts twice. With "benign" among the options, 96% were labelled benign. Without it, every single one got a threat label, most with high confidence. The model wasn't exactly wrong: you asked *which threat*, and it answered. But anyone reading "malware, 0.95" would act on a falsehood.

● KEY IDEA

A choice's probabilities add up to 1 over your options, so give every real case somewhere to go.

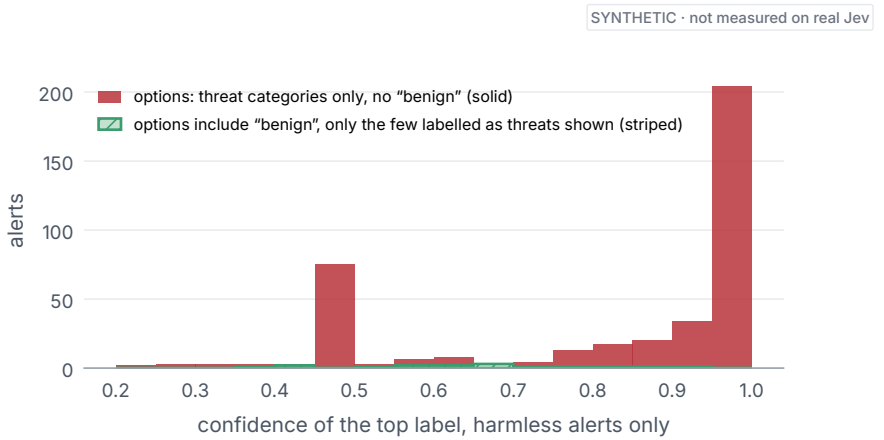


Figure 8.2. How confident the top label was for harmless alerts. With "benign" among the options (green, striped), almost all were labelled benign; only the few that weren't are shown. Without it (red, solid), every harmless alert got a threat label, most of them with high confidence.

So always include "none", "other" or "not enough information" when those can happen. And make sure only one option can be true at a time; when labels can overlap, ask several yes/no questions instead.

Rate on a scale: don't trust the average

A **score** returns a probability for each level of an ordered scale, plus the *average* level. That average can badly mislead you.

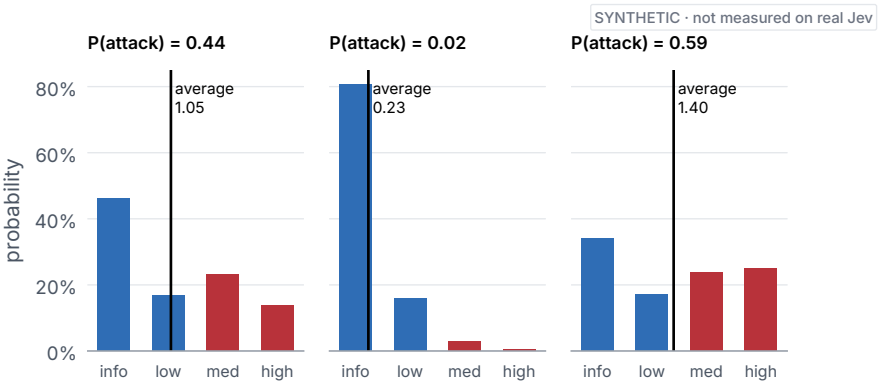


Figure 8.3. Severity probabilities for three alerts, with the average level marked. The first alert's average sits at "low". But its probability is split between "informational" and "medium or high". Almost nothing is actually at "low".

The first alert in Figure 8.3 has an average severity of 1.05, which reads as "low". But there's almost no probability *at* low. The model is really saying

this is either nothing, or serious, with a 37% chance of medium or high. That two-peaked shape appears whenever the model is unsure *whether* something is real. The average of two very different possibilities describes neither.

> ● KEY IDEA
>
> **Averages hide the extremes. Put thresholds on the probability of the levels you care about.**

"Call someone if P(medium or high) is above 0.3" is a rule you can reason about and set with costs. "Call someone if the average is above 1.5" isn't.

Decide with all the probabilities

The most likely label isn't always the one to act on. Suppose a choice says: benign 0.40, credential theft 0.35, phishing 0.15, malware 0.10. "Benign" is top, but 60% of the probability says *some kind of threat*, and closing the alert risks all of it. Weigh each action by what it would cost under each possible truth, and take the cheapest; here, that's sending the case to the identity team. That's Chapter 4's formula, generalised. And it's why you want a probability for every option: a text answer that just says "benign" throws away exactly the information you need.

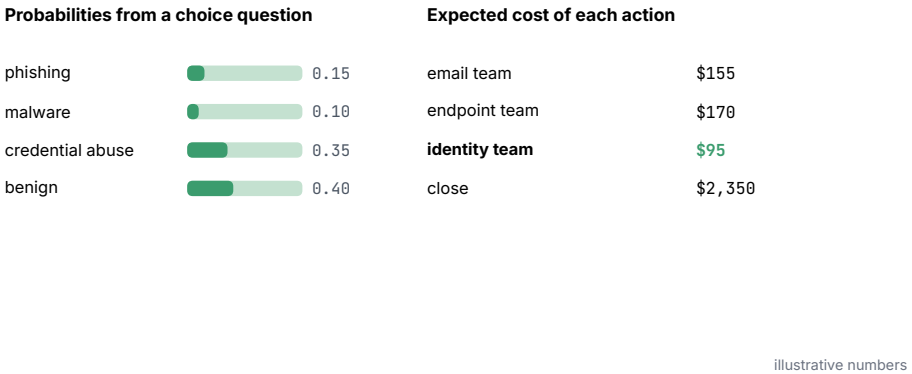


Figure 8.4. An illustrative choice between four labels, and the expected cost of four actions. "Benign" is the single most likely label, but closing the alert is the most expensive action, because the other 60% of the probability is spread across threats that are costly to miss.

Real decisions need several answers about the same situation. Ask them together, in one call; then combine them in ordinary code, where every rule is visible and testable.

⚠ WHERE THIS BREAKS

Three shapes can't express everything: a quantity ("how many megabytes?"), free text ("which username?") or several fields from a document still need something else, often an LLM (Chapter 12's "extract, then decide"). Separate answers about the same case may not agree with each other; your combining code should expect that. And wording changes the probabilities, so test a reworded question like any other change.

● KEEP THESE

- A choice's probabilities add up to 1 over your options, so give every real case somewhere to go.
- Averages hide the extremes. Put thresholds on the probability of the levels you care about.

EXERCISES

1. Design the typed questions for a decision you know (loan applications, support tickets, insurance claims). Which are yes/no, which pick-one, which scales? What's the "none of these" option each choice needs?

Next: TypeSafe says Jev's probabilities are calibrated; a critic says they can't be. Here's how to find out yourself.

CHAPTER 9

Testing a model's calibration yourself

Trust, but verify.

— Russian proverb, made famous by Ronald Reagan, 1987

A week after Jev launched, two claims were going round. TypeSafe's: Jev returns *calibrated* probabilities (TypeSafe AI 2026). And a reply from the data scientist Alex Molas, titled "Jev can't be calibrated": treat its outputs as scores, because calibration depends on *your* data, and Jev never sees your data (Molas 2026). Which is right? Both, in a precise sense, and you can settle it for your own use in an afternoon.

BY THE END OF THIS CHAPTER

- Explain why calibration depends on your data.
- Fix a base-rate mismatch with a few hundred labels.
- Know how many labels a calibration check needs.

Why the critic has a point

A probability is a claim about *how often, among cases like this one* (Chapter 2). Two companies send the same alert text; the model reads the same input and returns the same number to both. But if one company is attacked often and the other rarely, the true share of attacks among alerts like this differs between them. One number can't be right for both.

This is the base-rate trap again, now catching the model. A model tuned on one mix of outcomes can be perfectly calibrated there and wrong in the same direction everywhere the mix differs.

KEY IDEA

Calibration is a property of a model *and the data it's used on*.

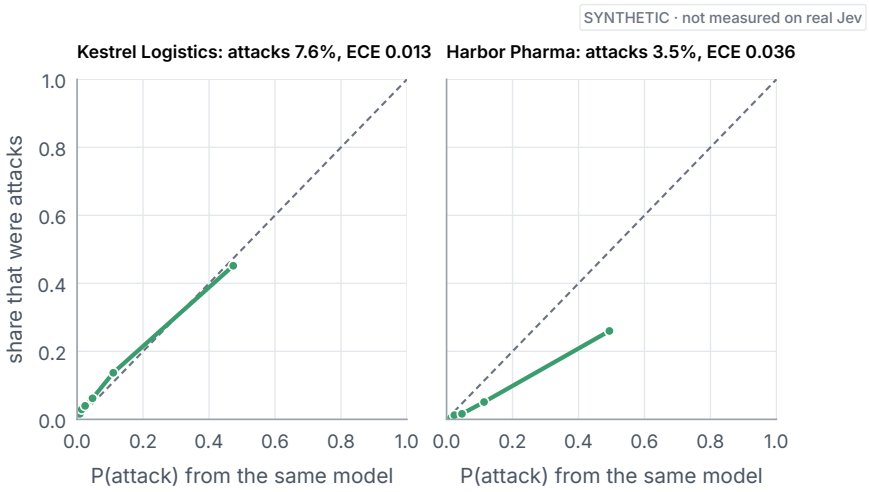


Figure 9.1. The same model's reliability at two companies. At Kestrel, whose mix of alerts matches what the stand-in was built around, it sits close to the diagonal. At Harbor, where attacks are rarer, every probability is too high.

To show it, I made a second synthetic company, Harbor Pharma: same detectors, same-looking alerts, but rarer attacks (3.5% against 7.6%). The same model's calibration error is 0.013 at Kestrel and 0.036 at Harbor, always in the same direction: Harbor's average predicted risk is 7.1% against a real rate of 3.5% (Figure 9.1). Nothing about the model changed. Only the world did.

Fixing it

This kind of error is often easy to fix. If the probabilities were right for one base rate and you know yours, adjust the odds by the ratio of the two: Bayes' rule again, with only the starting point moved (Saerens et al. 2002). With the base rate known, Harbor's ECE falls to 0.010. **Platt scaling** fitted on just 300 of Harbor's labelled alerts does even better: 0.007. It can also fix probabilities that are too spread out or too bunched, not only too high.

How many labels?

Small samples badly mislead calibration measurements. The stand-in's true ECE across 20,000 alerts is 0.013. Measured on a random 100 labelled alerts, the typical result is 0.043, over three times too high, and one sample in twenty says 0.070 (Figure 9.3). With few cases per group, each group's rate jumps around by chance, and ECE adds up the *size* of those jumps whatever their direction, so noise always pushes it up.

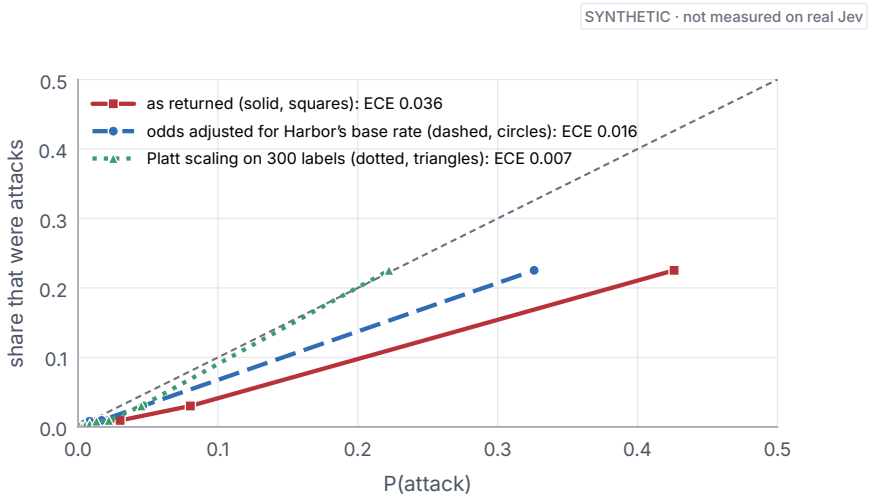


Figure 9.2. Harbor's alerts, on the cases not used for fitting: as returned (red, solid line), after adjusting the odds for Harbor's base rate (blue, dashed line), and after Platt scaling on 300 labelled alerts (green, dotted line). Both fixes bring the curve most of the way back to the diagonal.

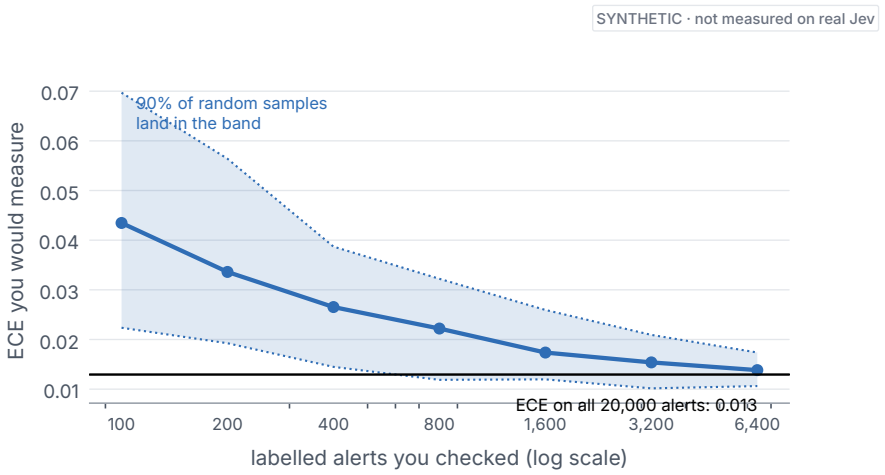


Figure 9.3. The ECE you'd measure from a random sample of labelled alerts, for different sample sizes, on a model whose ECE over all 20,000 alerts is shown by the labelled black horizontal line. With a hundred labels the measurement is both noisy and too high on average: a calibrated model looks miscalibrated. The error shrinks as samples grow.

KEY IDEA

With few labels, ECE is biased upwards. Report a range, and don't call a model miscalibrated on a hundred cases.

Rule of thumb: a few hundred labels are enough to *fix* a big error that always goes the same way; a few thousand to *confirm* calibration within a point or two.

For pick-one questions, start by checking the top label: when its confidence is around 0.8, is it right about 80% of the time? The stand-in's top category averages 89% confidence and is right 90% of the time. Then check the labels you actually act on.

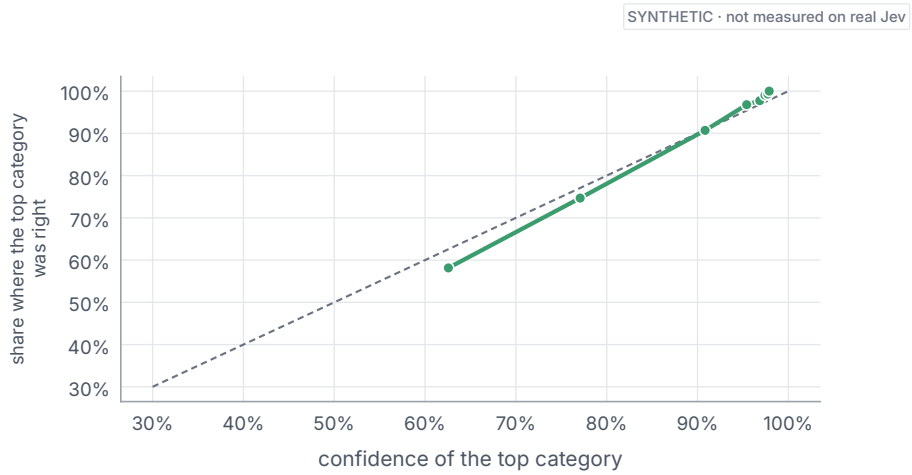


Figure 9.4. Top-label calibration for the category question on 2,500 of Kestrel's alerts: the confidence of the model's top category against how often that category was right.

A routine you can repeat

1. **Collect labels** from recent cases with settled outcomes, including a random sample of cases the machine closed itself, or you'll only ever label what people looked at.
2. **Plot, don't just score.** A reliability diagram with the counts showing, and ECE with a range. Look hardest near your thresholds.
3. **Slice** by every group you treat differently.
4. **Fix**: a base-rate adjustment if that's the problem, otherwise Platt or temperature scaling on your labels.
5. **Repeat** on a schedule, and whenever the model version or your data changes.

1	**Collect labels**	a few hundred to a few thousand recent cases with known outcomes
2	**Plot, don't just score**	reliability diagram + ECE with a range; look near your thresholds
3	**Slice**	every group you act on differently: source, customer, language
4	**Fix**	base-rate adjustment if you know it; Platt or temperature on labels
5	**Re-test**	on a schedule, and after any model version or data change

Figure 9.5. A testing routine for any model that returns probabilities, Jev included.

So: the vendor may well be right that Jev is calibrated on the data they tested. The critic is right that this doesn't make it calibrated on yours. The routine turns the argument into a measurement.

● KEY IDEA

Treat a new model's outputs as scores until you've tested them on your own data.

⚠ WHERE THIS BREAKS

Labels aren't perfect. People disagree, some attacks are never found, and outcomes arrive late: an alert closed today may belong to a break-in found next month. Measure on "mature" labels whose outcomes have settled. And beware labels that exist only because of the model's own decisions: if it auto-closed a case, nobody labelled it.

● KEEP THESE

- Calibration is a property of a model *and the data it's used on*.
- With few labels, ECE is biased upwards. Report a range, and don't call a model miscalibrated on a hundred cases.
- Treat a new model's outputs as scores until you've tested them on your own data.

EXERCISES

1. You have 150 labelled cases and measure ECE 0.05. A colleague says the model is badly calibrated. What would you say, and do?

Next: six ways to make the same decision, from a hand-written rule to an LLM to Jev, in one fair contest. Jev doesn't win every round.

CHAPTER 10

The bake-off: six ways to make a decision

There are two ways of constructing a software design: One way is to make it so simple that there are obviously no deficiencies, and the other way is to make it so complicated that there are no obvious deficiencies.

— C. A. R. Hoare, 1980

Every team that makes decisions has the same endless argument. The senior expert says the hand-written rules are fine: you can read every one. The data scientist says a trained model would beat them, given enough labels. The new engineer says an LLM reads cases better than both. And now someone has read about Jev. Each is right about something; they're judging different things. So let's put every number on one page: six methods, one decision, one fair contest.

BY THE END OF THIS CHAPTER

- Compare six ways of making the same decision.
- Score methods on more than accuracy.
- Pick a method from a scorecard.

The contestants

The decision: *is this alert a real threat?* Every method gets three weeks of labelled history (14,973 alerts) and is scored on a fourth, unseen week (5,027 alerts, 412 real). The six: hand-written **rules**; **logistic regression** on the alert's structured fields; a simple **text classifier** trained on the raw text; an **LLM asked for JSON**; an **LLM-as-judge** rating each alert 1 to 10; and **Jev**, reading the structured fields.

One crucial warning. Both LLM contestants and Jev are the book's stand-ins, and the LLM stand-in was built to read alerts *less accurately*. So where

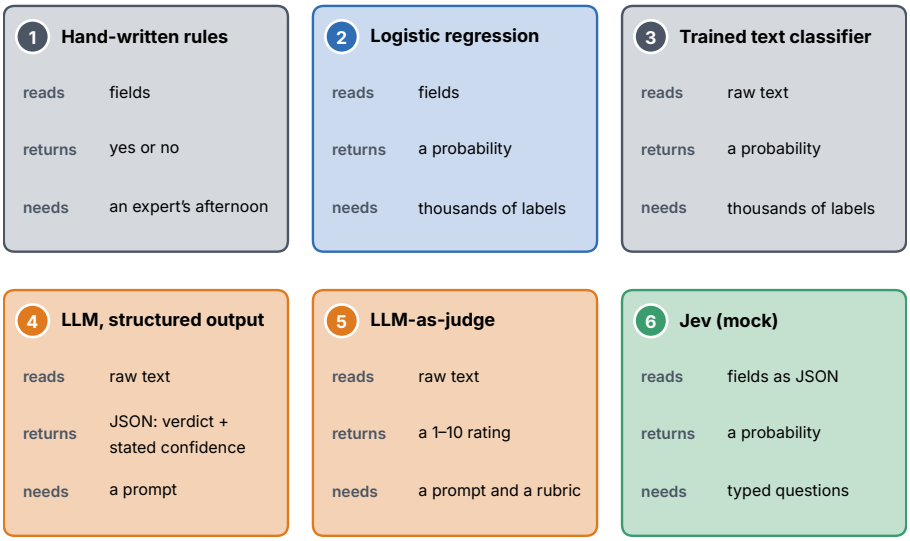


Figure 10.1. The six contestants: what each one reads, what it returns, and what it needs before it can start.

the LLM ranks below Jev, that's construction, not evidence about real models. What you *can* rely on is everything that comes from how each *kind* of method works: how written confidences behave, how often text output breaks, what asking twice does, and what labels cost.

The results

Ranking. Logistic regression wins, narrowly: AUC 0.896 against Jev's 0.892 (Figure 10.2). No luck involved: it trained on fifteen thousand of this company's own labelled alerts, on clean fields, for a decision that depends on them almost in a straight line. That's classic machine learning's home ground. Rules come last (0.66): a rule only says yes or no, so it can't say which of its hundreds of yeses to check first.

Calibration. Most methods sit near the diagonal. Not the LLMs. The LLM's *written* confidence used only 6 different values across five thousand alerts, mostly 0.9, 0.95 and 0.99; a number a model writes is text, and text likes round numbers. The LLM-as-judge's 1-to-10 rating, read as a probability, is the worst calibrated of all (0.102).

Labels needed. Logistic regression needs about 3,000 labelled cases to catch up with Jev, which needs none to start. A mature team has months of history; a new team, or a brand-new decision nobody has labelled, doesn't.

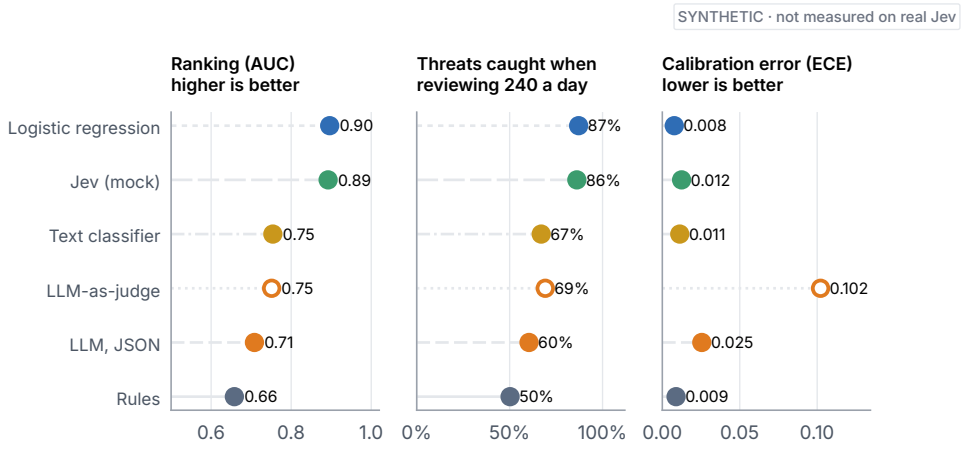


Figure 10.2. Three views of how well each method separates threats from harmless alerts, on the live week. Left: ranking quality (AUC). Middle: the share of all real threats found if the team reviews the top 240 alerts each day. Right: calibration error, where lower is better.

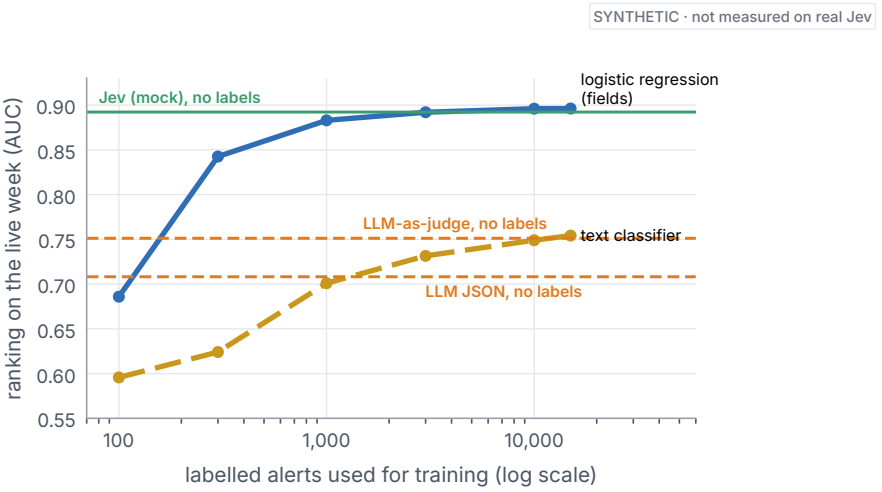


Figure 10.3. Ranking on the live week as the trained methods see more labelled alerts, averaged over random samples. Jev and the LLMs need no labels to start, so they're flat lines. Logistic regression catches up with Jev at about 3,000 labels.

● KEY IDEA

Labels are the hidden cost of trained models: a method that needs none can start today.

Speed and price. Nothing beats a rule, and classic models are close behind: millionths of a second, on computers you already own. Jev's vendor-reported figures sit in between, about $21 per million decisions; the illustrative LLM, about $740 and 1.4 seconds.

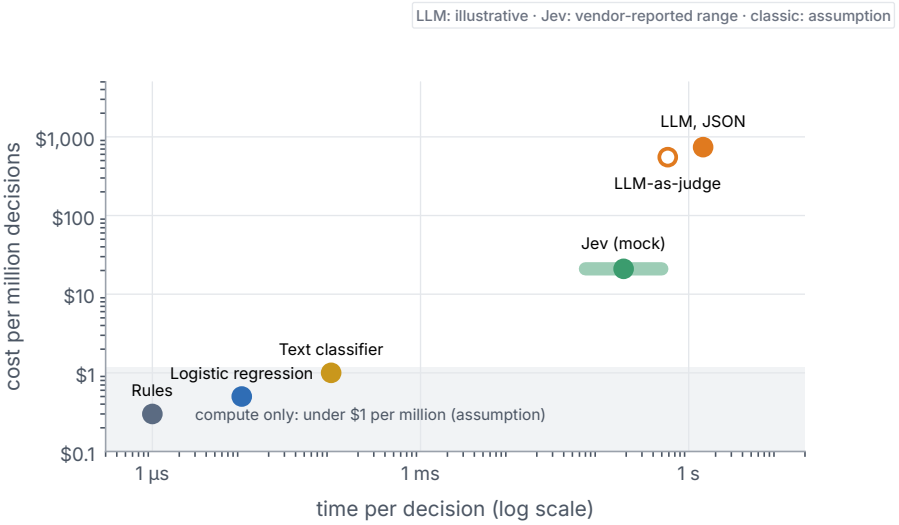


Figure 10.4. Time and cost per decision for each method, on log scales. Classic models run in millionths of a second, for fractions of a cent per million decisions (an assumption: computing cost only). The LLM figures are this book's illustrative assumptions; Jev's are vendor-reported.

● KEY IDEA

If a rule or a logistic regression can make a decision well, use one of them.

Jev's price advantage is against *LLMs*, not against everything.

Stability. Ask the LLM the same question five times with sampling on, and 5.2% of verdicts change at least once; 3.2% of its JSON answers couldn't be parsed at all. Typed answers can't fail to parse.

SYNTHETIC · not measured on real Jev

	ranking (AUC)	caught at 240/day	calibration (ECE)	cost per million	time per decision	labels needed	same answer twice?	explains itself?
Rules	0.66	50%	0.009	< $1	1 µs	none	yes	fully
Logistic regression	**0.90**	**87%**	**0.008**	< $1	10 µs	~3,000	yes	weights
Text classifier	0.75	67%	0.011	$1	100 µs	15,000+	yes	word weights
LLM, JSON	0.71	60%	0.025	$740	1.4 s	none	not always	in words*
LLM-as-judge	0.75	69%	0.102	$552	583 ms	none	not always	in words*
Jev (mock)	0.89	86%	0.012	$21†	70–500 ms†	none‡	yes (mock)	no

Bold: best in column. † vendor-reported. ‡ none to start; about 300 to check and calibrate. Classic costs: compute only (assumption).
* reasons written after the fact, which may not be how the verdict was reached (Chapter 6). LLM figures: illustrative.

Figure 10.5. The whole contest on one page. Bold marks the best in each numeric column. Every accuracy number is synthetic; every cost and time is an assumption, an illustration, or vendor-reported, as marked.

The scorecard

Read Figure 10.5 by column, and ask which columns *your* decision cares about. Thousands of labels, clean fields, a stable decision: logistic regression is hard to beat. Every decision explained to an auditor: rules win that column outright. No labels, raw text, or many different decisions: the System One model's row is strongest, starting with zero labels and, after a calibration check, giving probabilities you can put thresholds on. And the LLM? Weakest at deciding here, but it does what no other row tries: it reads anything, and it *writes*. Chapter 12 gives it exactly that job.

● KEY IDEA

There is no best method, only a best method for a particular decision.

⚠ WHERE THIS BREAKS

One decision, one synthetic company, stand-ins for the LLM and Jev, and only about four hundred real threats in the test week, so second-decimal differences in AUC could be chance. The value of a bake-off is the *method*: run the same contest on your own data, with the real models.

SET THE THRESHOLD

A new team takes over cloud-storage alerts, a decision nobody has labelled. They'll get about 300 labels a week. Which method first, and when to reconsider?
(Start with one that needs no labels. Use the first 300 labels to check and fix its calibration. After a few weeks, train a logistic regression beside it; if they agree, keep the cheaper one, and if they disagree, study the cases where they do.)

KEEP THESE

- Labels are the hidden cost of trained models: a method that needs none can start today.
- If a rule or a logistic regression can make a decision well, use one of them.
- There is no best method, only a best method for a particular decision.

EXERCISES

1. Build a scorecard for a decision in your own work. Which three columns matter most?

Next: a probability isn't an action. Three doors (act, review, escalate) and thresholds where the costs say.

CHAPTER 11

Act, review, or escalate

The buck stops here.

— Sign on President Harry S. Truman's desk

An alert comes in at 2 a.m. The model says 0.37. You know it's a probability and how to check it. Now someone asks the only question that matters: *"So what do we do with it?"* This step, from a number to an action, is the least exciting part of any AI system, and where most of the money is saved or lost.

BY THE END OF THIS CHAPTER

- Set act and escalate thresholds from costs.
- Fit a policy to the number of people you have.
- Run the policy safely: audits, logs and drift checks.

Why 0.5 fails

Every tutorial says: above 0.5, it's a threat; below, it's fine. Try it on a realistic week, with three weeks of history and a fourth treated as live. About 689 alerts a day close themselves, and every day about 37 of them were real threats (real phishing, real malware, a real stranger with someone's password) closed with nobody looking.

0.5 quietly assumes two false things: that a missed threat and a false alarm cost the same, and that there are only two things you can do. There are three.

Three doors

Every case leaves through one of three doors. **Act**: the system handles it alone, and nobody reads it. Most cases should go this way. **Review**: a person checks it as part of normal work, about twelve minutes of skilled time. **Escalate**: a phone rings, at 2 a.m. if need be, because this one can't wait. In

customer support the doors are auto-reply, agent and supervisor; in content moderation, auto-approve, reviewer and legal team.

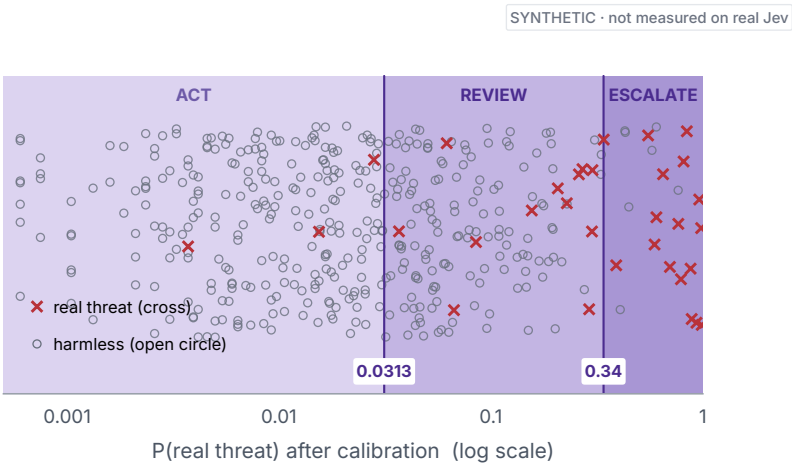


Figure 11.1. 420 alerts from the live week, placed by how likely the (calibrated) model thinks they are to be real threats. Note the log scale: most alerts are crowded at the far left, where the model is confident they're harmless. The red crosses, real threats, are mostly to the right, but not all of them.

● KEY IDEA

Act, review, escalate: machines handle the clear cases at each end, and people handle the middle.

Let the costs set the thresholds

For each door, write down what it costs *on average*, given the chance P the alert is real. These are my estimates for a mid-sized company; the method matters more than the values.

- **Act.** If it was real, a threat got through: about $10,000 expected loss. Cost: $P \times \$10{,}000$.
- **Review.** Twelve minutes at $75 an hour is $15, and reviewers miss about one threat in twenty. Cost: $\$15 + P \times 5\% \times \$10{,}000$.
- **Escalate.** Waking someone for nothing costs about $400. Cost: $(1 - P) \times \$400$.

At each P, pick the cheapest line. The two crossings in Figure 11.2 are your thresholds, and a pencil finds them: act below about 0.0016, escalate above about 0.43. Look at that first number. The machine may close

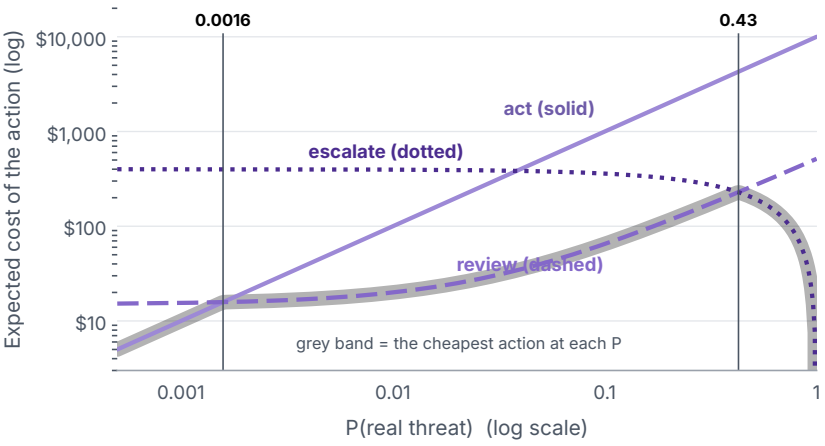


Figure 11.2. Expected cost of each action as the probability of a real threat rises (both axes on a log scale). Acting is cheapest when P is tiny, escalating when P is large, and review wins everywhere in between. The two crossing points are the thresholds.

an alert alone only if it's *99.8% sure* it's harmless. That isn't paranoia; it's arithmetic. An unseen real threat costs about 630 times a review.

Check the probabilities first. Those lines assume *P* means what it says. Among live alerts the raw model called "under 2% likely", it said 0.6% on average; the real rate was 1.4%, 2.5 times riskier than claimed. Most models are overconfident at the extremes. Platt scaling fitted on history brings them back close to the truth.

● KEY IDEA

Never put a threshold on a probability you haven't checked against recent outcomes.

Then the real limit appears: people

Switch on the cost-based policy and it sends about 638 alerts a day to review: a full day's work for about 16 people. The team has 6, so the queue can take 240 a day. Anything beyond that simply isn't looked at, and nobody chooses which.

● KEY IDEA

A policy that ignores capacity isn't a policy. It's a wish.

So ask a better question: *among policies that fit the team we actually have, which is cheapest?* Fitted on history and tested on the live week, the ma-

chine now closes 432 alerts a day, people review 243, and 44 are escalated. Real threats closed unseen fall from 37 a day to about 6.1.

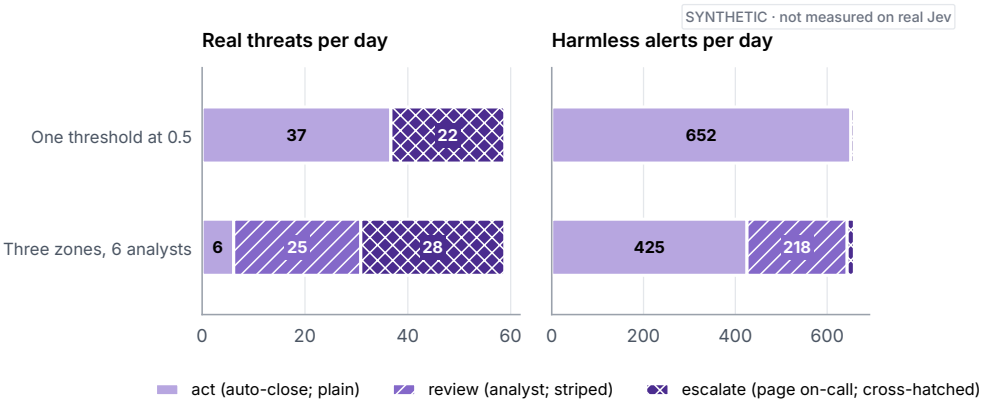


Figure 11.3. Where the live week's real threats (left) and harmless alerts (right) end up each day, under one threshold at 0.5 and under the three-zone policy that respects capacity. The light, plain segment on the left (act) is the number that matters: threats closed with no person involved.

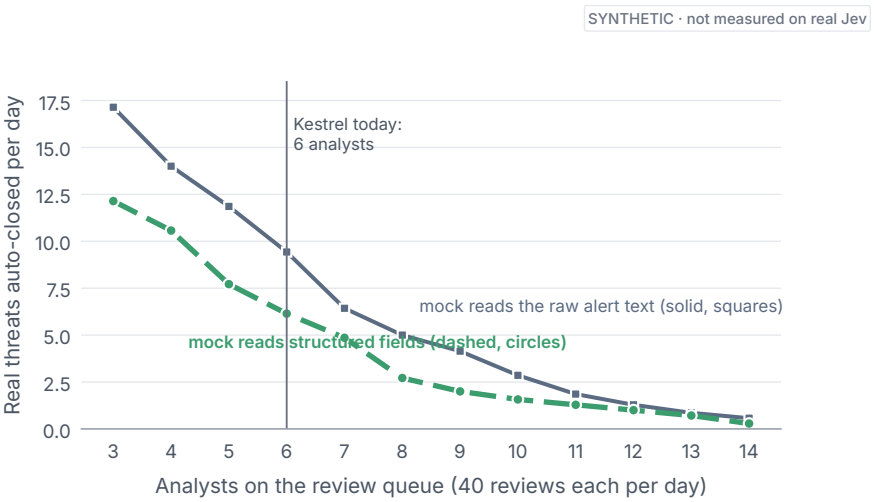


Figure 11.4. Threats auto-closed per day, for every team size from 3 to 14 analysts, with thresholds fitted again for each. The gap between the two curves is what better input is worth: the same model reading structured alert fields instead of raw alert text.

About six a day still isn't zero. Now think like the person who approves the budget. Figure 11.4 shows the best policy for every team size. Going from six people to eight cuts unseen threats from 6.1 to 2.7 a day, avoiding about $34,000 of expected loss daily. The model doesn't make that call. But it's now a decision with a clear price, not a guess. The grey curve, the same model reading raw text instead of clean fields, is worse at every team size: *better input buys the same as more people.*

Two more rules

Stakes override confidence. A model can be 99% sure an alert on the payroll server is harmless, but the other 1% is still on the payroll server. Nothing on a critical system closes itself, however confident the model. And anything that doesn't clearly belong behind another door goes to a person: *when in doubt, review.*

Audit the act zone at random. Cases sent to people come back with an answer. Cases the machine closed come back with *nothing*, so you never see the machine's own mistakes, in exactly the zone where they hurt. Send a small random share, say 3%, of auto-closed cases to a person anyway.

● KEY IDEA

Cases the machine closes never come back with an answer, unless you audit a random sample.

Running it for real

Write the policy down with a version number ("v3: act below 0.031, escalate at 0.34, fitted on 1–21 September, six people"), so it can be reviewed and rolled back. Log every decision: what the model saw, what it said, which policy turned that into an action. When the model fails or times out, the case goes to review, never to act.

Then watch three numbers every day: the share of cases in each zone, the queue against capacity, and predicted threats against confirmed ones. When a phishing campaign hits in week five, nothing about any single alert looks odd. But the model expected 7% of alerts to be threats and the real figure is 12%, and the queue overflows. Both are obvious on a daily chart. Then a person chooses: recalibrate on the latest week, lower the act threshold for the affected kind of alert for a while, or add review capacity.

⚠ WHERE THIS BREAKS

Thresholds are fitted to the past, and attackers act in the future. An attacker who learns what makes your model relax will shape their activity to land in the act zone. Random audits, keeping scores private, and rules that keep critical systems out of the act zone make that harder, not impossible. And the cost figures are guesses too: revisit them when the business changes, not just when the model does.

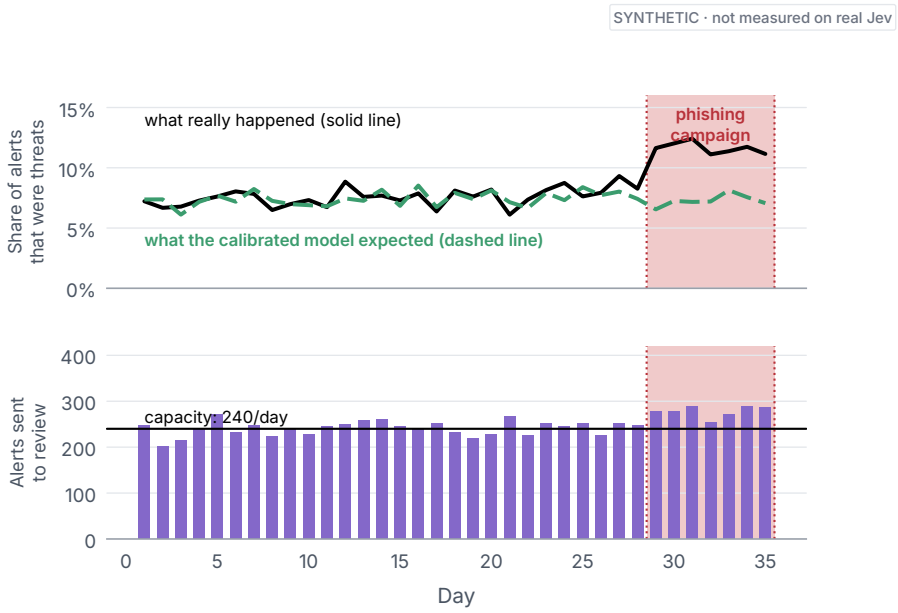


Figure 11.5. Top: the share of alerts that were real threats each day (black, solid line) against what the calibrated model expected (green, dashed line). Bottom: alerts sent to review each day against the team's capacity. In week five (shaded, between dotted lines), reality moves away from the model, and the queue overflows.

KEEP THESE

- Act, review, escalate: machines handle the clear cases at each end, and people handle the middle.
- Never put a threshold on a probability you haven't checked against recent outcomes.
- A policy that ignores capacity isn't a policy. It's a wish.
- Cases the machine closes never come back with an answer, unless you audit a random sample.

EXERCISES

1. Design an act/review/escalate policy for something outside security: refunds, insurance claims or content moderation. What does each door mean, which mistake is expensive, and what limits the review door?

Next: the common designs decision layers use inside real systems, and a hybrid agent built from them.

CHAPTER 12

Decision patterns, and a hybrid agent

Civilization advances by extending the number of important operations which we can perform without thinking about them.

— Alfred North Whitehead, 1911

Cooks stop needing recipes once they see the few steps every recipe shares. Software has steps like that too, called **patterns**: common solutions with names. This chapter collects the patterns decision layers use, then builds an agent from them and races it against one where the LLM does everything.

Every pattern shares one rule.

● KEY IDEA

A decision model never writes and never acts on its own: it answers a typed question, and code acts.

BY THE END OF THIS CHAPTER

- Recognise six patterns for decision models in real systems.
- Find a decision step that's wrong without any error.
- Split work between a decision model and an LLM.

Six patterns

1. Extract, then decide. Input is messy (an email, a chat, a PDF) but the decision is small. LLMs read messy input superbly; they're weaker at the decision itself. So let the LLM turn the text into fields, and let a decision model decide from the fields. On our alerts, extracting first raised ranking (AUC) from 0.708, with the LLM deciding alone, to 0.865. But the security

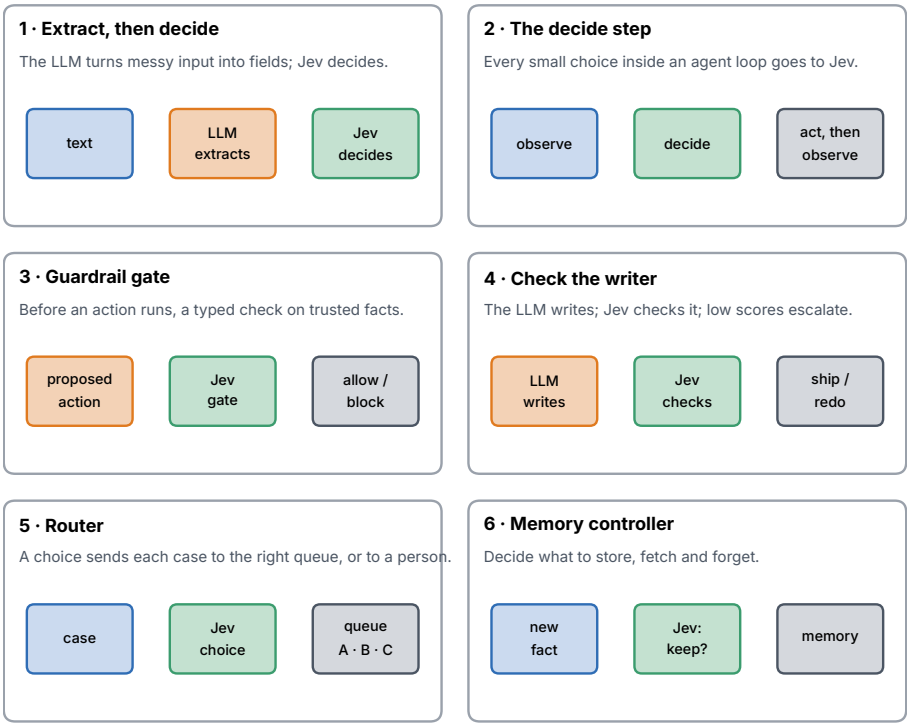


Figure 12.1. The six patterns in this chapter, as small diagrams. In every one, something else writes or acts, and a typed question decides.

system's own structured fields scored higher still (0.892): the alert text never mentions how critical the machine is or the user's job. No extractor can find a fact that isn't written down. So when structured data exists, use it.

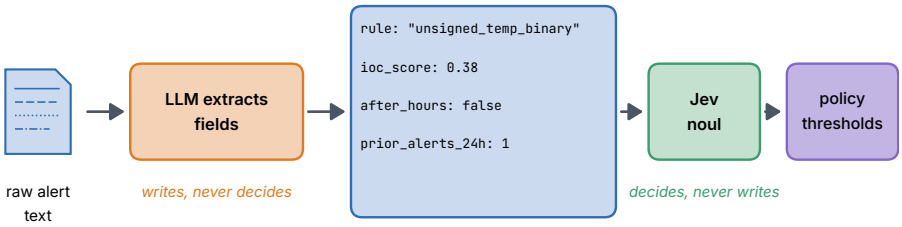


Figure 12.2. Extract, then decide. The LLM reads the raw text and writes structured fields; a decision model answers a typed question about them; policy thresholds turn the answer into an action. The LLM writes but never decides; the decision model decides but never writes.

2. The decide step. An agent's loop is mostly small decisions. Give them to a decision model as typed questions, and call the LLM only when there's something to write. Because each decision comes with probabilities, the loop can notice when it's unsure and stop, instead of continuing confidently.

3. The guardrail gate. Before any risky action, ask a typed question about it using only *trusted* facts, never text an attacker could write. High probability allows the action, low blocks it, the middle asks a person: the three doors again, for actions instead of alerts.

4. Check the writer. Some outputs must be written: an incident note, a customer reply. After the LLM drafts, ask typed questions *about* the draft: is every claim supported by the evidence? Is the tone right? Then send it, return it once for a rewrite, or give it to a person. Limit retries, or "write, reject, rewrite" never ends.

5. The router. Send each case to the right team with a pick-one question, but only automatically when the top option is clearly ahead. Routing every alert by its top label sent 88% to the right team; routing only when the top probability was at least 0.8 covered 80% of alerts and got 94% of those right. The rest went to a general queue. The same pattern can route between a cheap model and a slow one: Chapter 7's routing by doubt.

6. The memory controller. Agents keep notes and look things up, and each read and write is a small decision: is this worth remembering? Where should I look? A published system, Jev-Mem, uses a System One model for

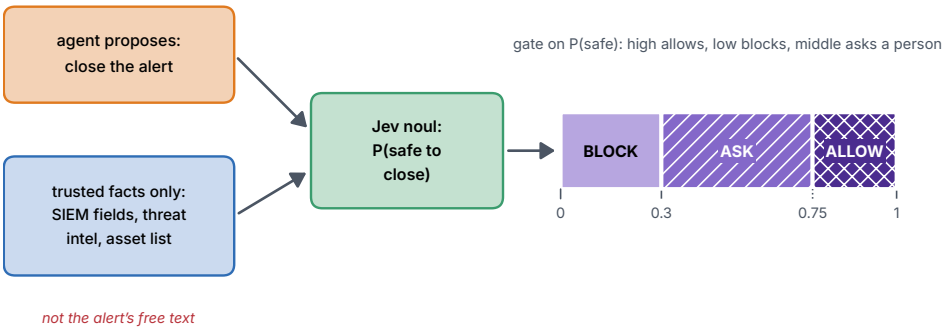


Figure 12.3. A guardrail gate. The agent proposes an action. A decision model estimates whether it's safe, from trusted facts only, never from text an attacker could write. High probability allows the action, low probability blocks it, and the middle asks a person.

exactly these and calls a slow reasoning model only for the final answer; its authors report better answers and faster responses than the systems they compared (Jiang et al. 2026). Memory mistakes fail silently (a discarded fact just isn't there when needed), so log what gets thrown away.

The hybrid agent

Now build Chapter 6's security agent as a **hybrid**: the decision model makes every decision, and the LLM writes only what a person will read. Every run leaves a **trace**, a step-by-step record of what was observed and what each decision said with what probability. It's a debugging tool, an audit log and an explanation in one.

The bug that didn't crash

When I first ran it over a live week, everything looked fine: no errors, every answer parsed, actions spread across the three doors. Then I ran a check worth running on every decision step of every agent: *count its answers*. Across 700 alerts, the "which evidence next?" step never once chose the threat-intelligence lookup, the most useful fact about an alert. Verdicts were reached without it, ranking dropped from 0.887 to 0.723, and about 22.6 real threats a day were auto-closed.

Typed answers always have the right format. The right format isn't the right answer, and nothing downstream will complain.

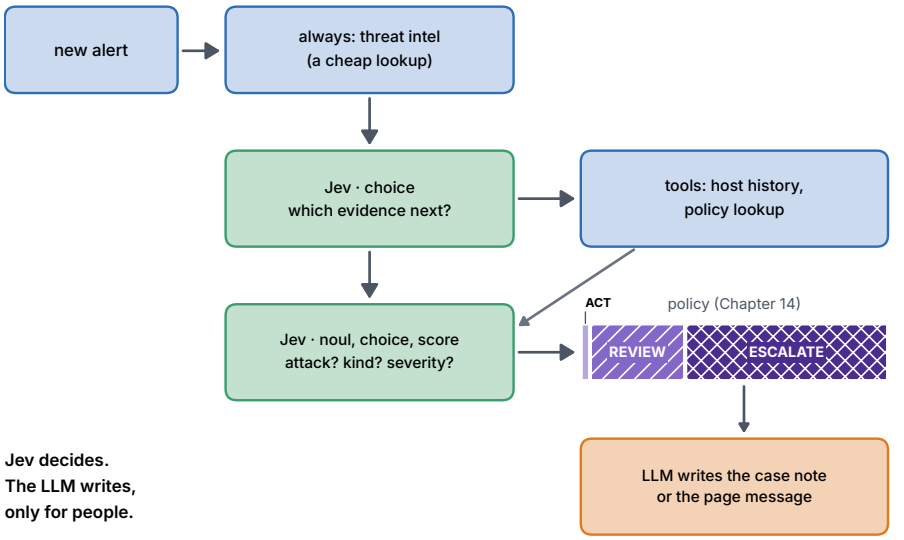


Figure 12.4. The hybrid agent. A cheap lookup always runs first. Jev decides which other evidence to collect, then answers the three verdict questions together. Chapter 14's policy turns the verdict into an action, and the LLM writes a case note or urgent message only when a person will read it.

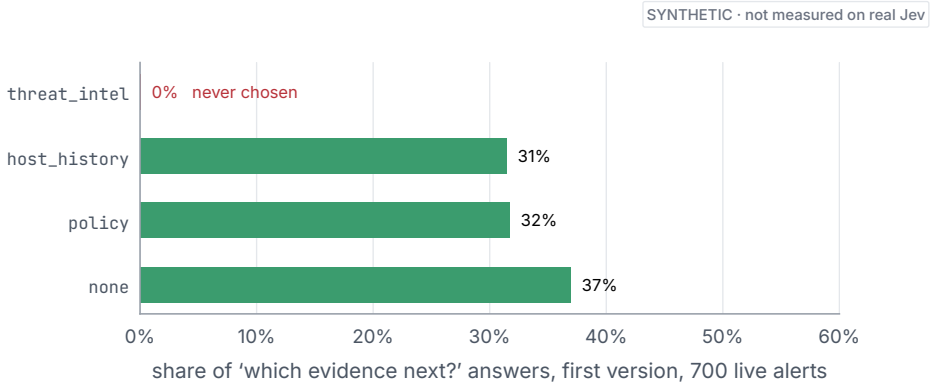


Figure 12.5. How often the first version of the agent chose each option at its "which evidence next?" step, across 700 live alerts. It never once chose threat intel.

KEY IDEA

Count every decision step's answers: an option never or always chosen is a warning, even without a crash.

The fix wasn't a better model. A threat-intel lookup takes a fraction of a second and costs nothing; deciding *whether* to look it up costs about as much as just looking.

KEY IDEA

When looking is cheaper than deciding whether to look, don't decide. Just look.

Hybrid against all-LLM

The all-LLM agent runs the same loop, tools and thresholds; only who decides changes.

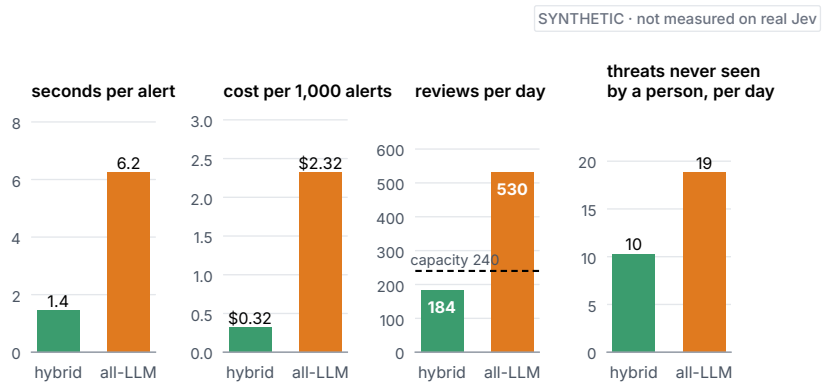


Figure 12.6. Four measures for the two agents on the same 700 live alerts, scaled to a day. The LLM's stated confidences sent far more alerts to review than the queue can take, so more real threats were never seen by a person.

The hybrid took 1.4 seconds per alert; the all-LLM agent, 6.2. The bigger difference is the queue (Figure 12.6). The LLM's written confidences cluster at a few round values, so the thresholds have almost nothing to separate, and it sent about 530 alerts a day to a review queue that can take 240. The hybrid sent 184. A queue at twice capacity isn't worked twice as hard; the extra alerts just aren't seen. Counting threats closed unseen *and* never reached, the hybrid missed about 10 a day; the all-LLM agent, about 19.

And where does the time go? The all-LLM agent spends most of it deciding. The hybrid spends its largest share *writing*: notes a person will read. That's where an LLM's time belongs.

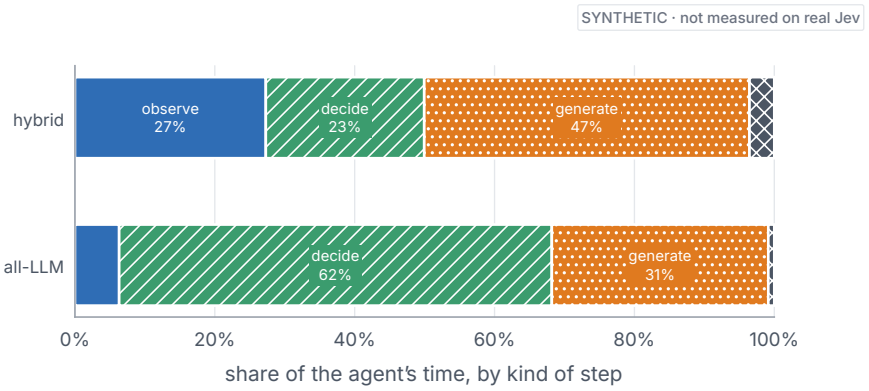


Figure 12.7. Share of each agent's total time spent on each kind of step. In the hybrid, the largest share is the LLM writing for people. In the all-LLM agent, most of the time goes on deciding.

● KEY IDEA

Use the LLM for the work only an LLM can do.

⚠ WHERE THIS BREAKS

Everything here is synthetic, timings included, and part of the quality gap between the agents is built into the stand-ins. Treat the speed, the queue and the design lessons as the findings. And combining patterns adds failure points: an extractor feeding a gate feeding a router has three places to be wrong, and the errors multiply.

● KEEP THESE

- A decision model never writes and never acts on its own: it answers a typed question, and code acts.
- Count every decision step's answers: an option never or always chosen is a warning, even without a crash.
- When looking is cheaper than deciding whether to look, don't decide. Just look.
- Use the LLM for the work only an LLM can do.

EXERCISES

1. Design a guardrail gate for an email agent that can message customers. What's the typed question, which facts are trusted, and where do the thresholds go?

Next: one team's triage, start to finish, and the same design far from security.

CHAPTER 13

One team, start to finish, and beyond

Everybody has a plan until they get punched in the mouth.

— Mike Tyson, attributed

Each chapter so far took one piece of the problem. This one runs the whole thing the way a real team would: the team as it was, the new system run quietly before it may act, then switched on, then hit by an attack campaign. Then we leave security behind and see the same design at work elsewhere.

BY THE END OF THIS CHAPTER

- Run a new decision system in shadow mode.
- See why the order of a queue matters.
- Recognise decision layers far from security.

Before: the queue

The security team gets about 718 alerts a day and can review 240. Before this project, alerts flagged by the senior expert's rules went to the front of one queue; everything else waited in arrival order; anything not reached within a day "aged out", closed unseen. A very ordinary setup, and it hides its failures: nobody sees what ages out, so nobody knows how much was real. In our synthetic week we do know. About 27.1 real threats a day were never seen by anyone.

The system, and a week in shadow mode

The new system uses nothing you haven't met: typed questions (Chapter 8), calibration fitted on history (Chapters 3 and 9), the three doors sized

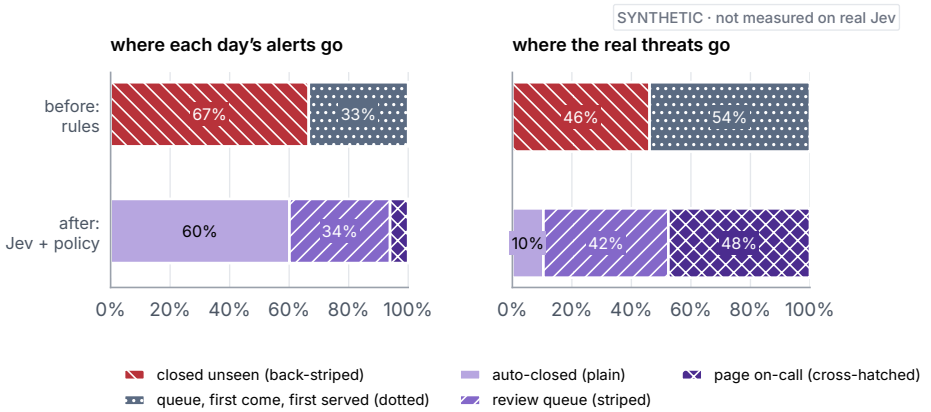


Figure 13.1. Where each day's alerts go, and where the real threats go, before and after. Before, most alerts aged out of the queue unseen, and so did a large share of real threats. After, most alerts are closed by the policy on purpose, and most real threats are queued or paged.

to the team (Chapter 11), and an LLM that writes only for people (Chapter 12).

You don't switch a system like this on and hope. You run it in **shadow mode**: it sees every alert and records what it *would* have done, while people work as before.

> ● KEY IDEA
>
> **Run a new decision system in shadow mode before you let it act.**

The meeting afterwards is mostly about one number: real threats the system *would* have auto-closed, about 6.1 a day. Bad, until you compare it with the 27.1 the old queue let age out. It was also more than forecast from history, and the right thing to say in that meeting is exactly that: the forecast was in range but too hopeful, and the daily monitor will tell chance from trend. The team switched it on with a random audit of auto-closed alerts and a fail-safe to review.

Order matters as much as size

The new queue is sorted by probability, likeliest first. In arrival order, with the same alerts in the same queue, the typical real threat would wait about 158 minutes. Sorted, about 4. Same people, same alerts. Only the order changed. Overall, the share of real threats reaching a person within an hour went from 37% to 83%.



Figure 13.2. How long it takes for a real threat to reach a person. Before, rule-flagged threats were seen quickly and the rest waited up to a day or were never seen. After, urgent calls reach someone in about fifteen minutes and the queue takes the likeliest threats first.

● KEY IDEA

A probability decides *what* a person looks at first, not only *whether* they look.

The campaign

Then a phishing campaign hit: link alerts fired three times as often, and far more were real. By day one the queue was over capacity, and the probabilities for link alerts were now too *low*, calibrated on weeks when those alerts were mostly harmless. The daily monitor caught both on day two. On day three the team re-estimated the base rate for link alerts, corrected those probabilities (Chapter 9), and approved overtime.

It helped: real threats never seen fell from about 144 to 121 over the week (Figure 13.3). But with honest probabilities, the queue grew *past* even the overtime capacity: far more alerts deserved a look than the bigger team could manage.

● KEY IDEA

Calibration tells you truthfully how much work there is. It can't make the work smaller.

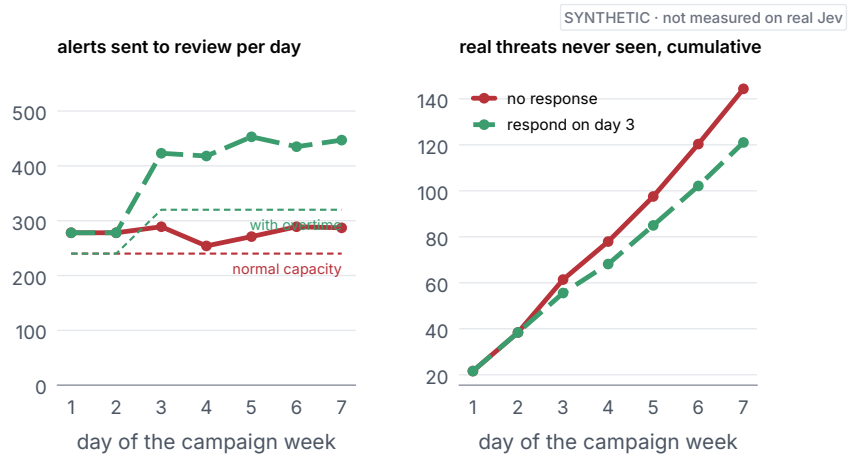


Figure 13.3. The campaign week, day by day, with and without a response on day 3. Left: alerts sent to review, against the team's capacity. Right: real threats never seen by a person, added up over the week. Recalibrating link alerts and adding overtime helped, but it didn't make the problem go away.

The team was never the problem. The old queue decided what they saw by accident; the new one decides on purpose.

Beyond security

Security was only the teaching example. Once you can see a decision layer, you see them everywhere.

Figure 13.4 describes six of them the same way: what the model reads, the typed questions, the escape option every choice needs, and who decides. What changes from card to card is *who makes the final call*. Where decisions are many and cheap (content moderation, payment fraud), no team could look at them all: the model decides and people audit a sample. Where they're few and costly (a hospital's clinical intake), the model should never close a case; its job is to put the right patient at the top of the list, and a person decides.

> ● KEY IDEA
>
> **Where decisions are many and cheap, the model decides; where they're few and costly, a person does.**

The threshold formula never changes; the costs do. A missed urgent support ticket costs a little goodwill, so the threshold lands near 5%. A missed emergency at a hospital's front desk costs far more than seeing someone

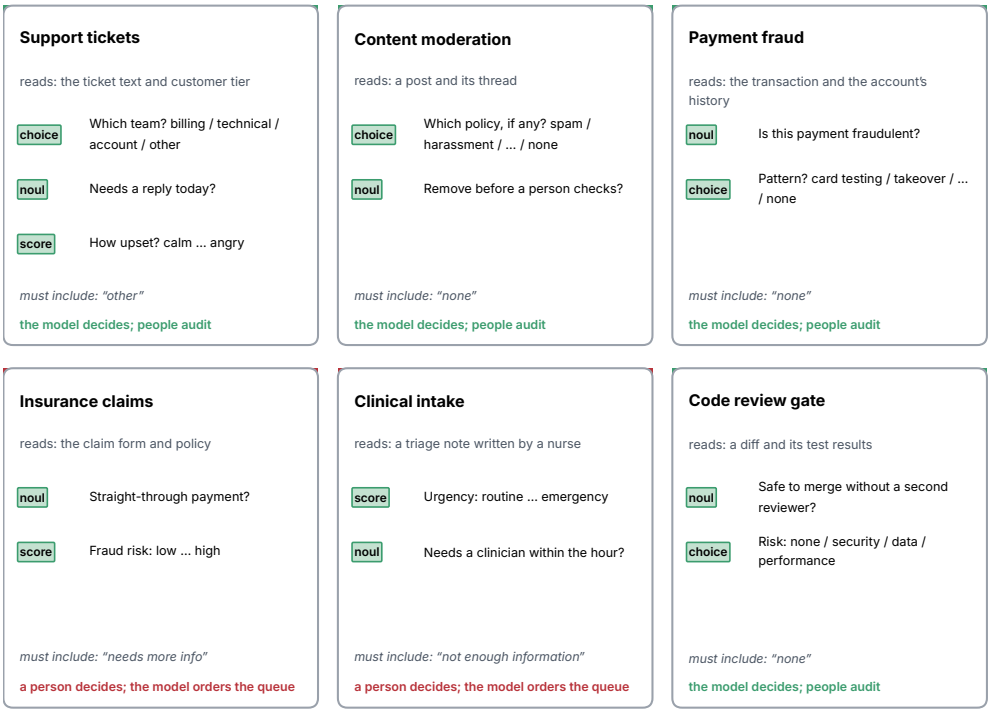


Figure 13.4. Six decision layers, each described the same way: what the model reads, the typed questions it answers, the option every choice must include, and who makes the final call. Green cards, which end "the model decides; people audit", let the model decide. Red cards, which end "a person decides", keep a person as the decider.

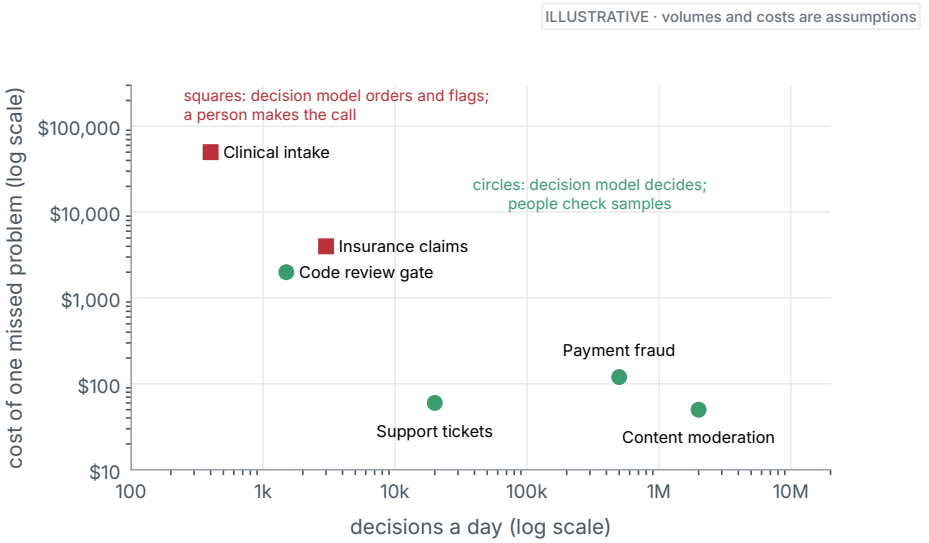


Figure 13.5. The six domains placed by how many decisions a day they involve and how much one missed problem costs (illustrative). Where volume is high and each mistake is cheap, the model decides and people check samples. Where each mistake is expensive, the model orders the queue and a person makes the call.

early, so it drops to about 0.3%, which demands probabilities calibrated even below one percent.

And always measure before you trust. On 400 synthetic support tickets, the stand-in that did well on security alerts sent only 74% to the right team, and account problems only 54% of the time, because "I can't log in" shares words with "the app isn't working". Routing only when the top label reached 0.6 covered 55% of tickets and got 85% right. The test took twenty lines of code.

Not everything is a decision layer. It fits when there are many similar cases, a fixed set of answers, outcomes you can learn from, and a cost you can put on a mistake. It doesn't fit one-off judgements, answers that must be written, or problems that need a plan rather than a verdict.

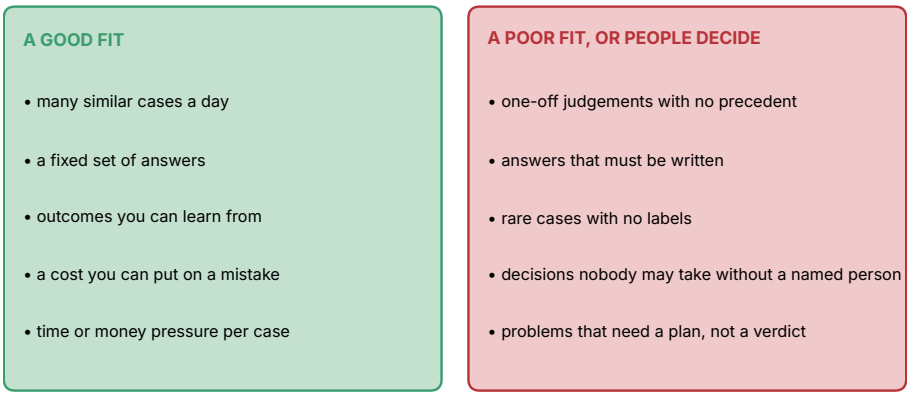


Figure 13.6. Signs that a decision suits a System One model, and signs that it doesn't, or that a person must decide.

⚠ **WHERE THIS BREAKS**

Every number here is synthetic or an assumption; real teams differ in ways a simulation leaves out. Shadow mode is only as good as the labels you compare against. And "a person decides" only protects anyone if that person has time to think: a clinician approving four hundred suggestions an hour isn't deciding, just clicking. Watch the workload of the people in the loop as carefully as the model.

KEEP THESE

- Run a new decision system in shadow mode before you let it act.
- A probability decides *what* a person looks at first, not only *whether* they look.
- Calibration tells you truthfully how much work there is. It can't make the work smaller.
- Where decisions are many and cheap, the model decides; where they're few and costly, a person does.

EXERCISES

1. Pick a decision from your own work and write its card: what the model reads, the typed questions, the escape option, and who decides.

Next: what it takes to run a decision model as a real service, and how to build a small one yourself.

CHAPTER 14

From model to service

Hope is not a strategy.

— A saying among site reliability engineers

Every project reaches the moment when the prototype works and someone asks: "Can we run it on Monday?" A model that decides well on a laptop is not a service. A service answers at 3 a.m. when the API is slow, explains itself to an auditor a year later, tells someone when the world has changed, and changes safely when you improve it. This chapter covers what that takes, and ends by building a small System One model of your own.

BY THE END OF THIS CHAPTER

- Call a model you don't own safely: versions, retries, failures.
- Make every decision explainable with a record.
- See what's inside a small System One model.

Calling a model you don't own

Build against a stand-in, then flip one switch. The official library builds requests, retries failures and parses answers; only its last layer, the *transport*, talks to the network, and you can swap it. The book's stand-in plugs in there, so your code runs exactly as it would for real, for free. Then go live by changing one setting.

Pin the model version. A name like "latest" can point to a new model at any time.

● KEY IDEA

Fix the model version, so a change in behaviour is your decision, not a surprise.

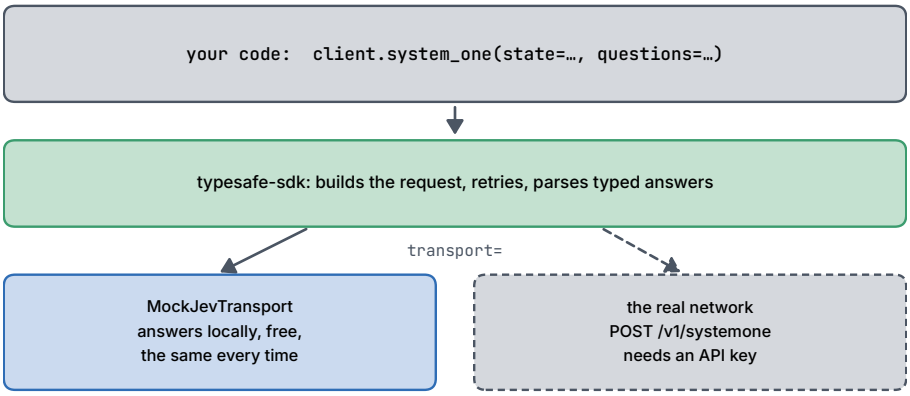


Figure 14.1. The official library does all the work of building requests, retrying and parsing answers. The transport decides where requests go. The book's stand-in answers locally; the default sends them over the network to TypeSafe.

Retry the right errors. Your own mistakes (no key, a model name that doesn't exist) fail the same way every time: fix them. Problems outside your code (rate limits, server errors, dropped connections) are worth retrying. If one request in five fails, only 79% of calls succeed with no retries and 99.2% with two. But retries cost time, and a login check may have only 300 milliseconds.

Decide what failure means. So treat a failed call as just another case the policy must handle. For a login, "allow, but flag for review" may be right; for closing an alert, "don't close, send to a person". Put that rule in the policy where people can see it.

> ● KEY IDEA
>
> **When time matters, decide in advance what a failed call means.**

Ask together, and test for free. Four questions in one call used about 179 input tokens; in four calls, 343. And record real answers once, then replay them in every test run, so tests cost nothing and a changed question fails loudly.

The service

You'll recognise every box in Figure 14.2. What turns a chain of steps into a service is four habits.

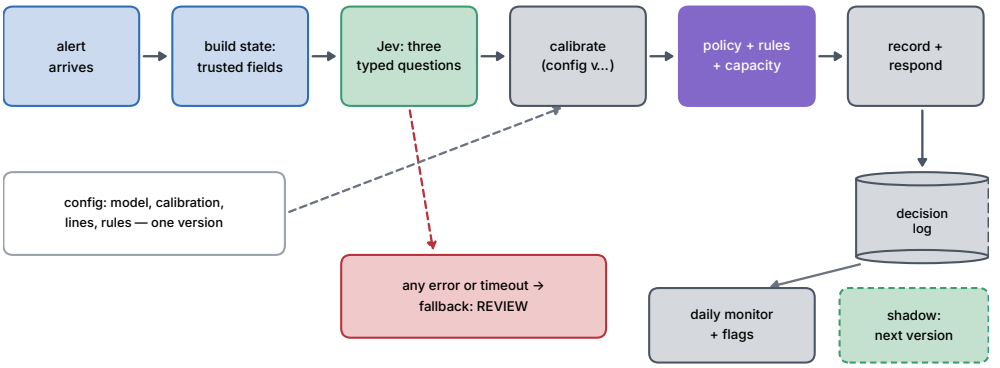


Figure 14.2. The decision service. Each alert's trusted fields go to Jev as three typed questions; the answer is calibrated, and the policy, business rules and capacity turn it into a zone. Every decision is recorded before it is returned. Any failure falls back to review. A daily monitor reads the log, and a candidate version can run in shadow beside the current one.

One configuration, one fingerprint. The model version, calibration numbers, thresholds and every business rule live in one place, and every decision is stamped with its version.

● KEY IDEA

Everything that can change a decision lives in one place, with a version number.

A record for every decision. A decision model gives probabilities, not reasons. But record the raw and calibrated probabilities, the thresholds they met, the rule that fired and the version that decided, and every *action* becomes explainable, even though the model explains nothing.

Fitted together. A dry run over a live week flagged day one: the queue was over capacity. The thresholds had been fitted for 240 reviews a day, and *afterwards* someone added a sensible rule, never auto-close an alert on a critical system. Every alert it moved to review was work the thresholds hadn't allowed for. Fitted *with* the rule, the queue fit again, from about 275 a day to 245.

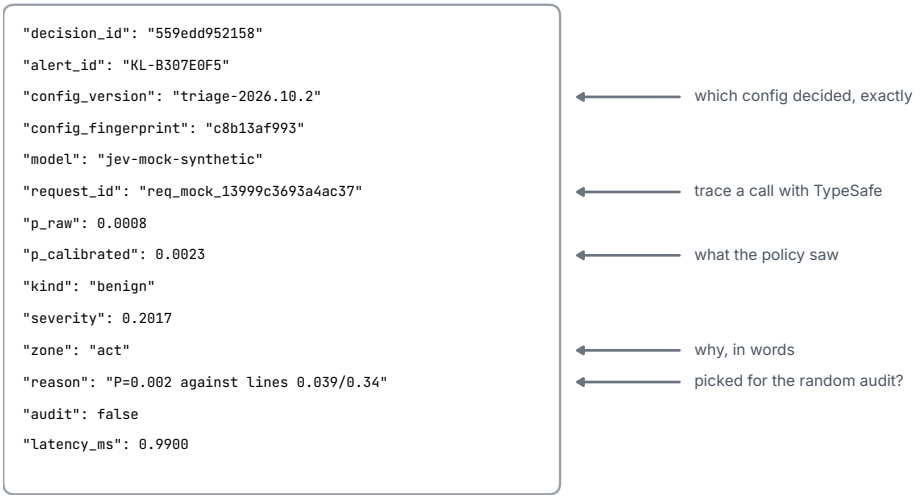


Figure 14.3. One decision record, with notes on why each field is there. Together they say what the service saw, what the model said, what the policy did, which version decided, and why.

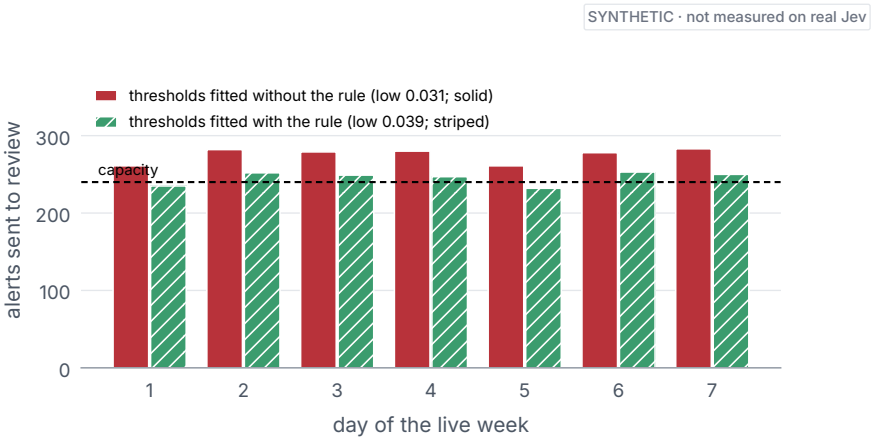


Figure 14.4. Alerts sent to review each day of the live week, before and after the fix. Fitted without the critical-asset rule, the thresholds let the queue run over capacity every day. Fitted with it, the act threshold rises slightly and the queue fits.

KEY IDEA

Thresholds and business rules are one decision, and they have to be fitted together.

A monitor, an owner, and shadow mode. Once a day, check the share of cases in each door, the queue against capacity, the fallback rate, and predicted against confirmed threats. On a campaign's first day, two flags go up, and the second, *predicted real far below confirmed real*, says in plain numbers that the probabilities are no longer calibrated. A named owner can switch the service to "review everything" while they find out why. And every improvement runs in shadow beside the current version first: here, the candidate agreed on 96% of alerts, and the shadow run measured, before anyone paid it, the threats its changes would let through.

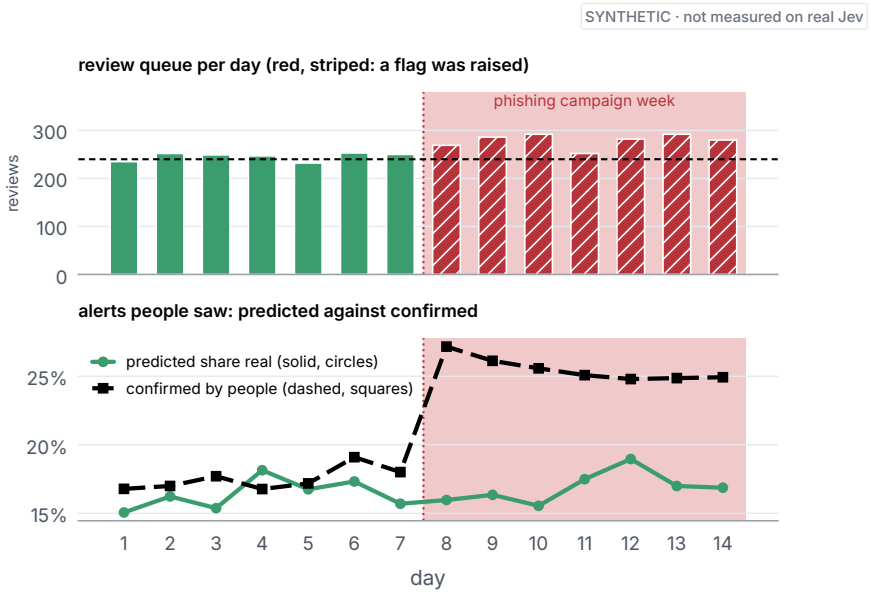


Figure 14.5. Two weeks of the daily monitor: the live week, then the phishing campaign. Top: the review queue, red and striped on days a flag was raised. Bottom: for alerts people saw, the share the model predicted to be real against the share they confirmed. The campaign opens a gap on its first day.

Build your own System One model

You don't need to build one to use one. But building a small one shows what its probabilities are made of.

The design follows the interface. A shared **encoder** reads an alert once and turns it into 32 numbers. Three small **heads** read those numbers, one

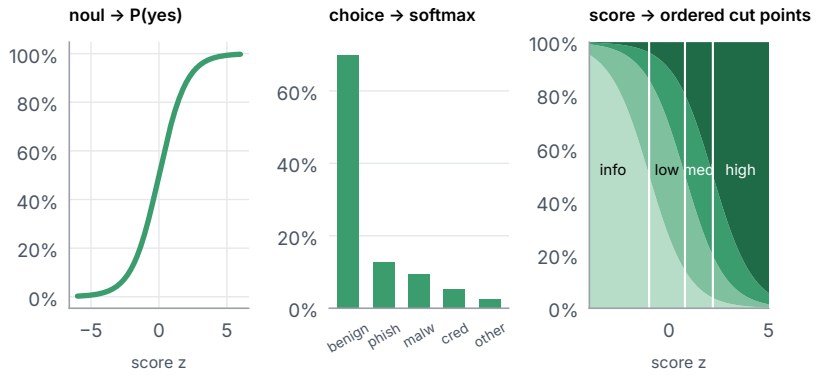


Figure 14.6. The three heads. A noul turns one number into P(yes) with the S-curve. A choice turns one number per label into probabilities that add up to 1. A score turns one number into probabilities for levels in order, by cutting it at points that must stay in order.

per question type (Figure 14.6). Yes/no uses Chapter 2's S-curve. Pick-one uses a softmax, so the probabilities add to 1. Rate-on-a-scale uses one "dial" cut at ordered points, so that "high" stays above "medium". All three are trained together with log loss, which rewards honest probabilities (Chapter 2), then a temperature per head is fitted on held-back data (Chapter 3). The whole model has 2,091 numbers and trains in seconds.

Calibration error on a fresh week fell from 0.017 to 0.009 after the temperature, with ranking unchanged. A deliberately oversized, overtrained version was the warning: its calibration error was 0.069, and after a temperature it was no better, 0.077. A correction can soften answers; it can't restore what memorising destroyed.

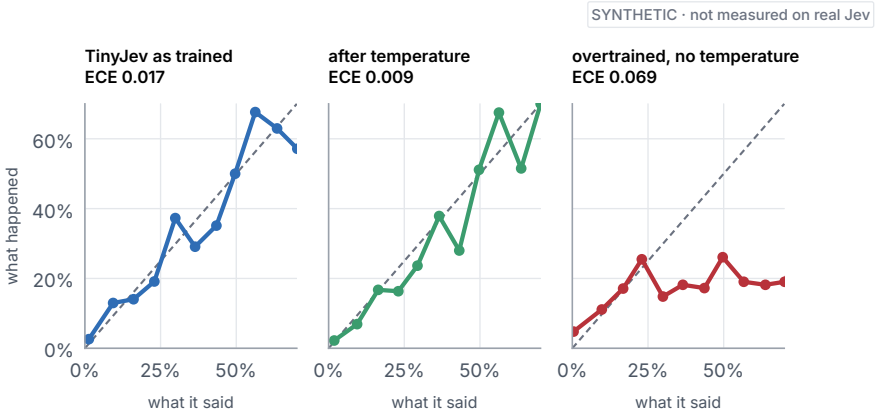


Figure 14.7. Reliability of P(attack) on the live week. Left: TinyJev as trained, slightly overconfident. Middle: after dividing by the fitted temperature. Right: the overtrained network before any temperature; its problems are ones no single temperature can fix.

● KEY IDEA

Train with a proper scoring rule, stop before the model memorises, then calibrate on data it has never seen.

⚠ WHERE THIS BREAKS

This is a teaching design. A real service also needs authentication, rate limits, a proper log store, access control on the configuration, and protection for personal data in what it records. Shadow comparisons only show where versions *disagree*; if both are wrong the same way, keep the random audit going. And the small model reads structured fields from one company; it will drift as soon as the world changes.

● KEEP THESE

- Fix the model version, so a change in behaviour is your decision, not a surprise.
- When time matters, decide in advance what a failed call means.
- Everything that can change a decision lives in one place, with a version number.
- Thresholds and business rules are one decision, and they have to be fitted together.
- Train with a proper scoring rule, stop before the model memorises, then calibrate on data it has never seen.

EXERCISES

1. For a service you rely on, list three errors you'd retry and three you'd fix. What should a failed call *mean* for each decision it makes?

Next: what changes, for the industry and for you, when deciding becomes cheap, fast and calibrated.

CHAPTER 15

What changes now

The future is already here. It's just not very evenly distributed.

— William Gibson, attributed

Most predictions about AI soon look foolish, so this last chapter keeps to three things: what the book showed, what it suggests, and what nobody knows yet.

BY THE END OF THIS CHAPTER

- See the decision layer as its own layer.
- Understand when cheaper decisions mean more work for people.
- Leave with a plan for your first decision.

A layer of its own

For the last few years, the default AI system has been one big model doing everything (reading, deciding, writing, calling tools), all as text.

The right side of Figure 15.1 is the design this book kept arriving at. The LLM does what only an LLM can: read anything, write for people. The decision layer answers small typed questions in one pass, with probabilities you can test. The policy sets thresholds from costs and capacity. Underneath, tools act and people review. Each layer is smaller than the system, testable on its own with Chapters 2 to 4's tools, and replaceable without rebuilding the rest. That, more than any benchmark, is why I think this split will last.

● KEY IDEA

Split the system: the LLM reads and writes, the decision layer decides, the policy acts, people own the costs.

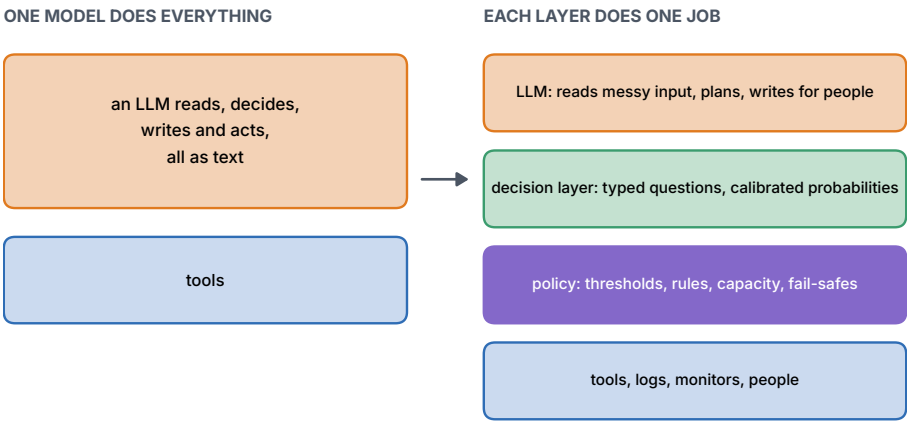


Figure 15.1. Two ways to build an AI system. Left: one model does everything as text. Right: each layer does one job. An LLM reads and writes, a decision layer answers typed questions with probabilities, a policy turns those into actions, and tools, logs and people sit underneath.

The Jevons paradox of decisions

In 1865 the economist William Stanley Jevons noticed something odd about coal (Jevons 1865). James Watt's steam engine got far more work from each ton, yet Britain burned *more* coal, not less: cheaper coal-powered work got used for more things. Photos and light followed the same pattern. When each cost almost nothing, we didn't keep the savings; we took thousands of photos and lit streets all night (Fouquet and Pearson 2006). Jev's name is short for Jevons. The coal this time is decisions.

Every decision has a *value*, the loss it avoids, and is worth making only when that beats its price and arrives before its deadline. A company makes a few thousand careful decisions a day, but *could* make millions: every email, every login, every log line, each worth only a fraction of a cent to check. At this book's illustrative LLM price of $0.00066 a decision, almost none are worth it. At Jev's vendor-reported $0.000021, many are.

The surprise is where the change shows up. Inside jobs a company already does, cheaper decisions multiply the number made (in our illustrative model, 8 times as many) while the bill *falls*. The bill rises only when cheap decisions make *new* jobs worth doing, like checking every file shared outside the company.

KEY IDEA

Cheaper decisions rarely raise the bill for jobs you already do; they raise it by making new jobs worth doing.

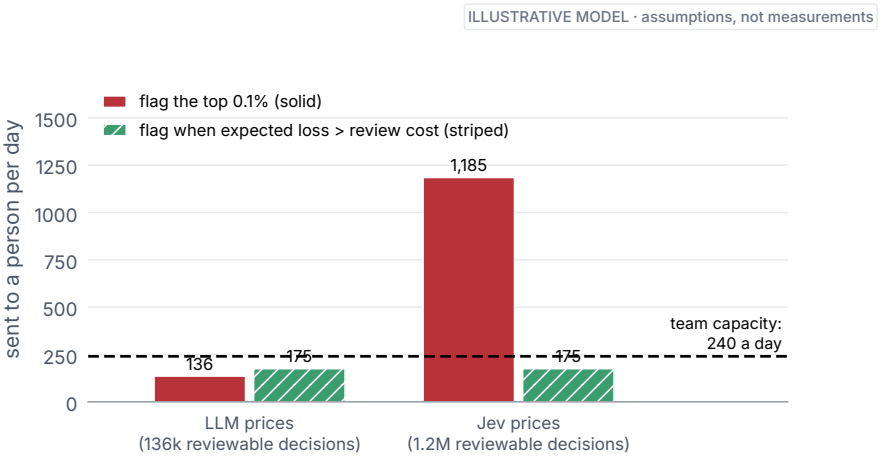


Figure 15.2. Alerts sent to a person per day, under two rules (illustrative). A rule that flags a fixed share of what the model looks at grows with the number of decisions, and at Jev volumes it sends far more than the team can handle. A cost-based threshold, which flags only when the expected loss is larger than the cost of a review, barely moves.

There's a hidden cost: more decisions means more cases that end with a person. A rule that seemed sensible at low volume, "send the top 0.1% to a person", sends 136 a day under the team's capacity of 240, then 1,185 at the new volume, though nobody changed the policy (Figure 15.2). A cost-based rule, which flags a case only when its expected loss beats the cost of a review, sends 175 either way.

KEY IDEA

When decisions get cheap, set thresholds by cost, not by share.

What gets valuable, and what nobody knows

If deciding gets cheap, what becomes scarce? Judgement about the decision itself. Someone still has to decide what a mistake costs, where the thresholds go, how much the review queue can take, and whether the probabilities are still calibrated this week. No model does that for you. That's why Chapters 2 to 4 were worth the long way round: they're the difference between a system that makes a million cheap decisions well and one that makes a million cheap mistakes.

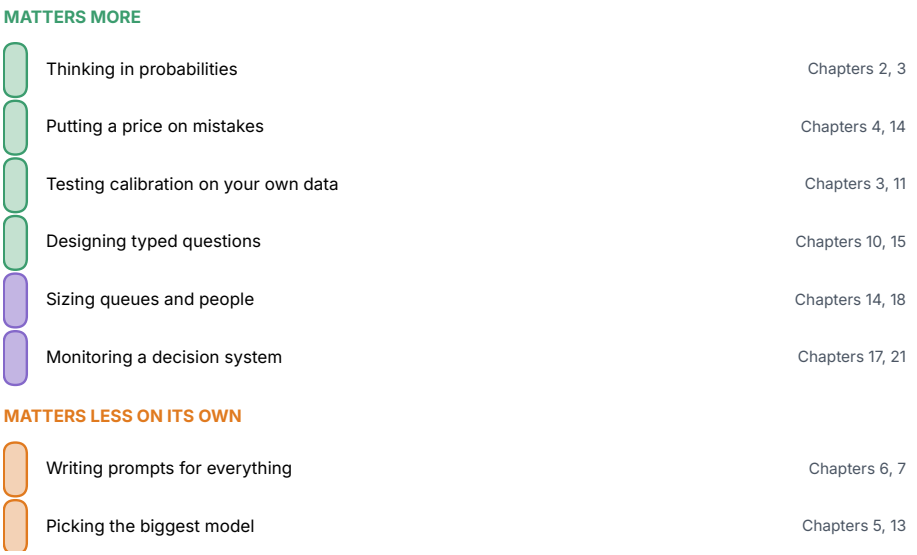


Figure 15.3. Skills that matter more when decisions get cheap, and skills that matter less on their own, with the chapters that teach each.

● KEY IDEA

When deciding gets cheap, judgement about the decision itself becomes scarce.

Much is still unknown: how well any System One model is calibrated on your data, how its behaviour changes between versions, how it handles attackers, what prices will settle at. None is a reason to wait. Calibration is a test you can run this week; version changes are handled by a pinned version and a monitor; prices are a spreadsheet. And the pattern will outlast any one vendor. A typed question in, a probability out, and a policy in your own code can move to a different model in an afternoon.

● KEY IDEA

You need decisions you can measure, thresholds you can move and a model you can replace.

What to do on Monday

If you do one thing after closing this book, list every decision your system makes, and count them. Most people are surprised how many there are, and how many are small, typed and frequent.

? **Does Jev's calibration hold on data like yours?**

Chapter 11 tells you how to find out.

? **How fast do System One models improve, and do they drift between versions?**

Pin versions; re-test.

? **Will prices stay this low?**

Early-access prices change. Chapter 12's model shows what depends on it.

? **Who else builds decision models?**

The interface pattern outlives any one vendor.

? **How will rules on automated decisions evolve?**

Keep people deciding where stakes are high.

Figure 15.4. Open questions, and what you can do about each today.

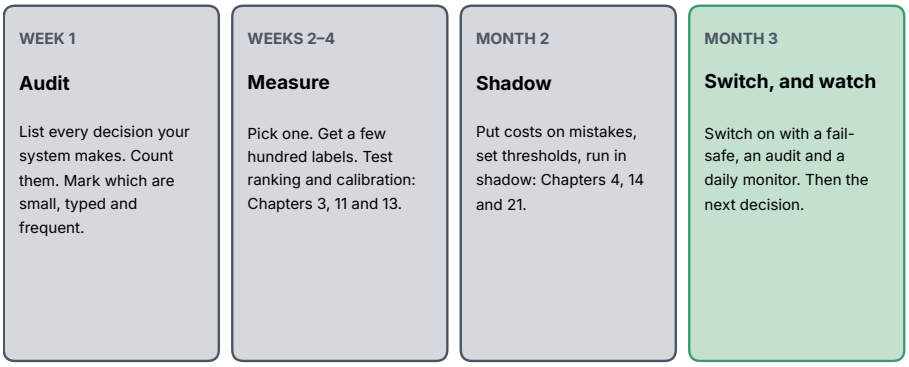


Figure 15.5. A first three months, one decision at a time.

Then pick *one*. Get a few hundred labels and test its calibration. Put prices on its mistakes and set its thresholds. Run it in shadow mode. Switch it on with a fail-safe, a random audit and a daily monitor. Then do the next one. It isn't exciting. It's how every system in this book got better.

> ⚠ **WHERE THIS BREAKS**
>
> This chapter is the least certain in the book. The cost and speed figures are illustrative, and Jev's are vendor-reported from early access. The claim that a decision layer will become standard is my reading of the evidence, not a finding; it could be wrong if general models become fast and well calibrated enough that splitting stops paying. If that happens, everything about costs, thresholds and calibration still applies; only the box it sits in changes.

> **SET THE THRESHOLD**
>
> Your own system: take its most frequent decision. What does a wrong "yes" cost? A wrong "no"? Where does the threshold go? And how would you know, a month from now, whether the probabilities behind it are still calibrated? (There's no answer at the back of the book for this one. If you can fill in all four, you've learned what this book set out to teach.)

Alan Turing ended his most famous paper with a sentence that fits this moment: "We can only see a short distance ahead, but we can see plenty there that needs to be done" (Turing 1950).

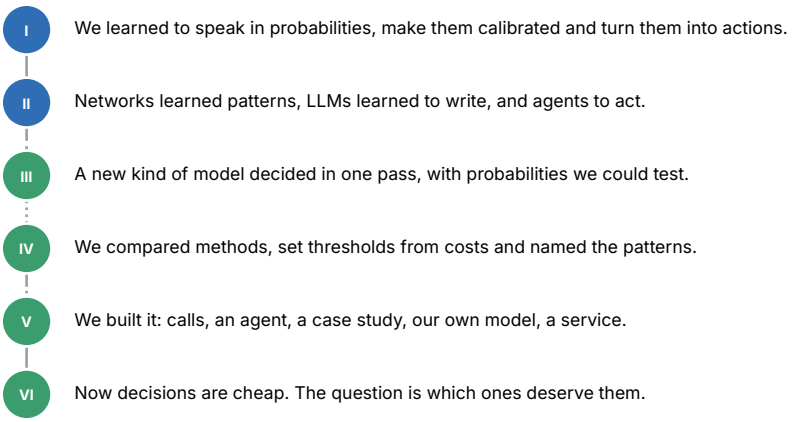


Figure 15.6. The whole book as one story.

KEEP THESE

- Split the system: the LLM reads and writes, the decision layer decides, the policy acts, people own the costs.
- Cheaper decisions rarely raise the bill for jobs you already do; they raise it by making new jobs worth doing.
- When decisions get cheap, set thresholds by cost, not by share.
- When deciding gets cheap, judgement about the decision itself becomes scarce.
- You need decisions you can measure, thresholds you can move and a model you can replace.

EXERCISES

1. Write the decision audit for one system you know: every decision, how often it happens, what a mistake costs, and who owns it.

Thank you for reading. The code behind every figure is in the companion repository, ready for your own data.

Glossary

One plain sentence for each term, and the chapter where it's explained fully.

Act, review, escalate The three zones of a decision policy: handle it automatically, send it to a person, or raise it urgently (Chapter 11).

Agent A program that uses a model to observe, decide and act in a loop, calling tools along the way (Chapter 6).

Attention A way for each word in a sequence to look at the others and decide which ones matter for its meaning (Chapter 5).

Audit (random) Sending a random share of automatically handled cases to a person, so that mistakes the system is sure about still get seen (Chapter 11).

AUC The chance that a model gives a randomly chosen real case a higher score than a randomly chosen harmless one. It measures ranking, not calibration (Chapter 3).

Base rate How common something is before you look at any evidence, like the share of alerts that are real threats (Chapter 2).

Brier score The average squared gap between a predicted probability and what happened. Lower is better (Chapters 2 and 3).

Calibration A model is calibrated when the things it calls 70% likely happen about 70% of the time: its probabilities are honest (Chapter 3).

Capacity How many cases the people in the loop can handle in a day. A policy that ignores it isn't a policy (Chapter 11).

Cassette A file of recorded requests and responses that a test can replay without calling the real API (Chapter 14).

CISO Chief information security officer: the person in charge of an organisation's security (Chapter 13).

Choice A typed question with a set of named options. The answer is a probability for each option, adding up to 1 (Chapter 8).

Confidence In a Jev `choice` or `score` answer, a number from 0 to 1 computed from the shape of the probabilities. It's high when most of the probability is on one option, and low when it's spread out. It isn't simply the top probability (TypeSafe's quick-start shows 0.78 beside 0.85). The book's mock uses the top probability as a stand-in (Chapter 7).

Context window The amount of text an LLM can read at once (Chapter 6).

Cost-based threshold The probability above which acting is cheaper than not acting, set by what each kind of mistake costs (Chapter 4).

Decision layer The part of a system that turns probabilities into actions: typed questions, calibration, thresholds, rules and fail-safes (Part IV).

Decision record The logged account of one decision: what was seen, what the model said, what the policy did, which version decided and why (Chapter 14).

Drift A change in the world, or in the data, that makes a model's probabilities less accurate than when they were checked (Chapter 11).

DLP (data loss prevention) Security tools that warn when company data may be leaving the organisation, such as a large file sent to a personal email address (Chapter 3).

ECE (expected calibration error) The average gap between what a model said and what happened, across groups of similar predictions (Chapter 3).

Elasticity How much more of something people use when its price falls (Chapter 15).

Embedding A list of numbers that places a word, sentence or item in a space where similar things sit close together (Chapter 5).

Epoch One full pass of training over all the training examples (Chapter 5).

Escape option The "other", "none" or "not enough information" option that gives cases fitting nowhere somewhere to go (Chapter 8).

Extract, then decide A pattern in which an LLM turns messy input into fields and a decision model decides on those fields (Chapter 12).

F1 One score that balances how many flagged cases are real and how many real cases get flagged. It measures ranking and labelling, not calibration (Chapter 3).

Fail-safe The decided meaning of a failure, such as "if the call fails, send the case to review" (Chapters 11 and 14).

Fine-tuning Further training of an existing model on examples from one task (Chapter 5).

Gradient descent Improving a model step by step by nudging its numbers in the direction that reduces the loss (Chapter 2).

Guardrail gate A typed check on trusted facts that runs before an agent's action is allowed to happen (Chapter 12).

Hallucination An LLM stating something fluent and false (Chapter 5).

Hybrid agent An agent in which a decision model answers the small decisions and an LLM only writes (Chapter 12).

Isotonic regression A calibration method that learns any increasing mapping from scores to probabilities. It's flexible, but needs more data than Platt scaling (Chapter 3).

Jevons paradox When making something more efficient to use makes people use more of it in total, not less (Chapter 15).

JSON mode An LLM setting that makes its output valid JSON, though not necessarily correct JSON (Chapter 5).

Label The known right answer for an example, used to train or test a model (Chapter 1).

LLM (large language model) A model trained to predict the next piece of text, which lets it read and write almost anything (Chapter 5).

LLM-as-judge Using an LLM to rate or check an output, often on a numbered scale (Chapter 10).

Log loss A score that punishes confident wrong answers heavily. It's a proper scoring rule (Chapter 2).

Logistic regression A model that adds up weighted evidence and passes the total through an S-curve to get a probability (Chapter 2).

Loss A number that measures how wrong a model is on some examples. Training tries to make it smaller (Chapter 2).

Mock A stand-in for a real service that answers in the same shape. This book's is `jev-mock-synthetic` (Chapter 14).

Noul TypeSafe's name for a yes-or-no question. The answer is the probability of yes (Chapter 8).

Observe, Decide, Act The loop an agent runs: read something, choose what to do, do it, repeat (Chapter 6).

Ordinal Having a meaningful order, like severity levels. An ordinal model keeps that order in its probabilities (Chapter 14).

Overconfidence Probabilities that are more extreme than the evidence justifies. It's common in large models and in overtrained small ones (Chapter 5).

Overfitting Learning the training examples so closely that performance on new examples gets worse (Chapter 2).

Platt scaling A calibration method that fits an S-curve mapping a model's scores to calibrated probabilities (Chapter 3).

Policy The rules that turn a probability into an action, usually as thresholds between zones (Chapter 11).

Prior shift A change in the base rate between where a model was calibrated and where it's used. You can correct it if you know the new base rate (Chapter 9).

Prompt injection Text planted in an input that tries to make a model follow the attacker's instructions (Chapter 6).

Proper scoring rule A way of scoring predictions under which reporting your honest probability gives the best average score (Chapter 2).

RAG (retrieval-augmented generation) Giving an LLM relevant documents, found by search, before it answers (Chapter 6).

Reliability diagram A chart of what a model said against what happened. A calibrated model sits on the diagonal (Chapter 3).

RLCD TypeSafe's name for reinforcement learning for calibrated decisions, which it says trains Jev. The details aren't published (Chapter 7).

RLHF Reinforcement learning from human feedback, which trains an LLM towards answers people prefer (Chapter 5).

Router A pattern that sends each case to a destination with a choice question, falling back to a person when unsure (Chapter 12).

Score A typed question with ordered levels. The answer is a probability for each level and an expected level (Chapter 8).

Shadow mode Running a new system or version beside the current one, recording what it would do without letting it act (Chapters 13 and 14).

SIEM (security information and event management) The tool that collects a company's logs and raises security alerts. Kestrel's alerts come from one (Chapter 10).

Softmax The function that turns a list of scores into probabilities that add up to 1 (Chapters 5 and 8).

State What a decision model reads: the situation, as text or structured fields (Chapter 7).

Structured output An LLM answer constrained to a schema, such as JSON with named fields (Chapter 5).

System 1, System 2 Kahneman's names for fast, automatic thinking and slow, deliberate thinking (Chapter 7).

System One model TypeSafe's term for a model that answers typed questions with probabilities in one fast pass, without writing text (Chapter 7).

Temperature (calibration) One number that softens or sharpens every probability a model gives, fitted on held-out data (Chapter 3).

Temperature (sampling) A setting that makes an LLM's choice of next token more or less random (Chapter 5).

Three-zone policy A policy with two thresholds, splitting cases into act, review and escalate (Chapter 11).

Threshold A number that splits cases into two groups, like a line on the probability scale: above it you do one thing, below it another (Chapters 1 and 4).

Token A piece of text, often part of a word, that an LLM reads and writes one at a time (Chapter 5).

Trace The step-by-step record of what an agent observed, decided and did (Chapter 12).

Transformer The network design built from attention layers that underlies modern LLMs (Chapter 5).

Transport The lowest layer of an HTTP client, which sends a finished request and returns a response. The book's stand-in is one (Chapter 14).

Triage Sorting cases by how likely they are to be real and how urgent they are, so the most important ones are handled first (Preface, Chapter 13).

Typed question A question whose answer has a fixed shape: a noul, a choice or a score (Chapter 8).

Vendor-reported A claim made by the company selling the product, not independently checked (Chapter 7).

Cheat sheets

Everything on these pages is explained fully in the chapter given in brackets. They're here for the day you need the formula and not the explanation.

Setting the threshold (Chapter 4)

For a yes-or-no decision, act when the probability of a problem is above

$$p^* = \frac{C_{\text{false alarm}}}{C_{\text{false alarm}} + C_{\text{miss}}}$$

A false alarm costing $400 and a miss costing $10,000 give $p^* = 400/10{,}400 \approx 0.04$.

Three zones (Chapter 11)

Say a review costs C_r and misses a share m of real problems. A miss costs C_{miss} and a false urgent call costs C_{page}. Then the cheapest thresholds for calibrated probabilities are

$$\text{low} = \frac{C_r}{C_{\text{miss}}\,(1-m)} \qquad \text{high} = \frac{C_{\text{page}} - C_r}{C_{\text{page}} + m\,C_{\text{miss}}}$$

Then check the queue. If the review zone holds more cases than people can clear, raise the low (act) threshold until it fits, and write down what that costs.

Measuring probabilities (Chapters 2 and 3)

For predictions p_i and outcomes $y_i \in \{0, 1\}$:

$$\text{Brier} = \frac{1}{n}\sum_i (p_i - y_i)^2 \qquad \text{log loss} = -\frac{1}{n}\sum_i \left[y_i \log p_i + (1-y_i)\log(1-p_i)\right]$$

$$\text{ECE} = \sum_b \frac{n_b}{n}\left|\,\bar{p}_b - \bar{y}_b\,\right|$$

where each bin b holds n_b predictions with average $\bar{p}_b$ and observed rate $\bar{y}_b$. AUC measures ranking only: a model can have a perfect AUC and useless probabilities.

Fixing probabilities (Chapters 3 and 9)

Method	What it fits	Use when
Temperature	$p' = \sigma(z/T)$, one number	the model is too confident (or not confident enough) everywhere
Platt scaling	$p' = \sigma(az + b)$, two numbers	a few hundred labels; the usual first fix
Isotonic	any increasing curve	thousands of labels
Prior shift	odds × $\dfrac{b'/(1-b')}{b/(1-b)}$	only the base rate changed, from b to b'

Here $z = \log \frac{p}{1-p}$ and σ is the S-curve.

Testing a model's calibration on your data (Chapter 9)

1. Collect labels for a random sample of real cases, not just the interesting ones.
2. Measure ranking (AUC) and calibration (ECE, with a bootstrap range) separately.
3. Compare the average predicted probability with the observed base rate. A big gap usually means the base rate has changed (prior shift).
4. Fix the base rate first; then use Platt scaling on a few hundred labels if needed.
5. Test again on labels you didn't fit on. Repeat on a schedule and whenever the model version changes.

Designing typed questions (Chapter 8)

- **Noul**: one fact per question; say what counts as yes and as no; no double negatives.
- **Choice**: options that don't overlap; always an escape option ("other", "none", "not enough information").
- **Score**: put thresholds on the probability of the levels you care about, not on the average level.
- Ask related questions in one call: the state is sent once.
- Decide with all the probabilities and the costs of each action, not just the top label.

Calling a decision model (Chapters 7, 8 and 14)

- Ask several typed questions about the same situation in **one call**: yes/no (noul), pick one (choice), rate on a scale (score).
- Every choice needs an escape option: "none", "other", "not enough information".

- **Pin the model version.** "latest" can change under you.
- Retry rate limits and server errors; never retry your own mistakes, fix them.
- Decide in advance what a **failed call means**; the safe default is "send to a person".
- Log the probabilities, the policy version and the request ID with every decision.

Choosing a method (Chapter 10)

If you have…	Start with
thousands of labels, clean fields, one stable decision	logistic regression
a rule an expert can write and an auditor must read	rules
no labels, raw text or many different decisions	a decision model, checked on a few hundred labels
messy input and a clear decision	an LLM to extract, a decision model to decide
output a person must read	an LLM, checked by typed questions

Before a decision service runs (Chapter 14)

A fixed model version · a config with a version number and a fingerprint · a meaning for every failure · a record for every decision · random audits of automatic actions · a daily monitor · shadow runs for new versions · an owner who can switch everything to review.

Bibliography

Casscells, Ward, Arno Schoenberger, and Thomas B. Graboys. 1978. 'Interpretation by Physicians of Clinical Laboratory Results'. *New England Journal of Medicine* 299 (18): 999–1001.

Chow, C. K. 1970. 'On Optimum Recognition Error and Reject Tradeoff'. *IEEE Transactions on Information Theory* 16 (1): 41–46.

Elkan, Charles. 2001. 'The Foundations of Cost-Sensitive Learning'. *Proceedings of the Seventeenth International Joint Conference on Artificial Intelligence (IJCAI)*, 973–78.

Fouquet, Roger, and Peter J. G. Pearson. 2006. 'Seven Centuries of Energy Services: The Price and Use of Light in the United Kingdom (1300–2000)'. *The Energy Journal* 27 (1): 139–77.

Frederick, Shane. 2005. 'Cognitive Reflection and Decision Making'. *Journal of Economic Perspectives* 19 (4): 25–42.

Gigerenzer, Gerd, and Ulrich Hoffrage. 1995. 'How to Improve Bayesian Reasoning Without Instruction: Frequency Formats'. *Psychological Review* 102 (4): 684–704.

Greshake, Kai, Sahar Abdelnabi, Shailesh Mishra, Christoph Endres, Thorsten Holz, and Mario Fritz. 2023. 'Not What You've Signed up for: Compromising Real-World LLM-Integrated Applications with Indirect Prompt Injection'. *Proceedings of the 16th ACM Workshop on Artificial Intelligence and Security*.

Guo, Chuan, Geoff Pleiss, Yu Sun, and Kilian Q. Weinberger. 2017. 'On Calibration of Modern Neural Networks'. *Proceedings of the 34th International Conference on Machine Learning (ICML)*.

Jevons, William Stanley. 1865. *The Coal Question: An Inquiry Concerning the Progress of the Nation, and the Probable Exhaustion of Our Coal-Mines*. Macmillan.

Jiang, Dongming, Yi Li, and Bingzhe Li. 2026. *Jev-Mem: System-One-Controlled Agentic Memory for Efficient AI Agents*. arXiv:2609.23986. Accessed 29 September 2026. https://arxiv.org/abs/2609.23986.

Kahneman, Daniel. 2011. *Thinking, Fast and Slow*. Farrar, Straus and Giroux.

Kahneman, Daniel, and Gary Klein. 2009. 'Conditions for Intuitive Expertise: A Failure to Disagree'. *American Psychologist* 64 (6): 515–26.

Kaplan, Jared, Sam McCandlish, Tom Henighan, et al. 2020. *Scaling Laws for Neural Language Models*. https://arxiv.org/abs/2001.08361.

LeCun, Yann, Yoshua Bengio, and Geoffrey Hinton. 2015. 'Deep Learning'. *Nature* 521: 436–44.

Lewis, Patrick, Ethan Perez, Aleksandra Piktus, et al. 2020. 'Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks'. *Advances in Neural Information Processing Systems 33 (NeurIPS)*.

Molas, Alex. 2026. *Jev Can't Be Calibrated*. Blog post, 23 September 2026. Accessed 29 September 2026. https://www.alexmolas.com/2026/09/23/jev-cant-be-calibrated.html.

Murphy, Allan H., and Robert L. Winkler. 1977. 'Reliability of Subjective Probability Forecasts of Precipitation and Temperature'. *Journal of the Royal Statistical Society, Series C (Applied Statistics)* 26 (1): 41–47.

Naeini, Mahdi Pakdaman, Gregory F. Cooper, and Milos Hauskrecht. 2015. 'Obtaining Well Calibrated Probabilities Using Bayesian Binning'. *Proceedings of the AAAI Conference on Artificial Intelligence*.

OpenAI. 2023. *GPT-4 Technical Report*. https://arxiv.org/abs/2303.08774.

Ouyang, Long, Jeff Wu, Xu Jiang, et al. 2022. 'Training Language Models to Follow Instructions with Human Feedback'. *Advances in Neural Information Processing Systems 35 (NeurIPS)*.

Platt, John C. 1999. 'Probabilistic Outputs for Support Vector Machines and Comparisons to Regularized Likelihood Methods'. In *Advances in Large Margin Classifiers*. MIT Press.

regolo.ai. 2026. *Jev and System One Models: Benchmarks, Open-Source Alternatives, and When to Use Them*. Blog post. Accessed 29 September 2026. https://regolo.ai/jev-and-system-one-models-benchmarks-open-source-alternatives-and-when-to-use-them/.

Rumelhart, David E., Geoffrey E. Hinton, and Ronald J. Williams. 1986. 'Learning Representations by Back-Propagating Errors'. *Nature* 323: 533–36.

Saerens, Marco, Patrice Latinne, and Christine Decaestecker. 2002. 'Adjusting the Outputs of a Classifier to New a Priori Probabilities: A Simple Procedure'. *Neural Computation* 14 (1): 21–41.

Turing, Alan M. 1950. 'Computing Machinery and Intelligence'. *Mind* 59 (236): 433–60.

TypeSafe AI. 2026. *Introducing System One Models & Jev*. TypeSafe AI blog. Accessed 29 September 2026. https://typesafe.ai/blog/introducing-system-one-models-and-jev.

Vaswani, Ashish, Noam Shazeer, Niki Parmar, et al. 2017. 'Attention Is All You Need'. *Advances in Neural Information Processing Systems 30 (NeurIPS)*.

Yao, Shunyu, Jeffrey Zhao, Dian Yu, et al. 2023. 'ReAct: Synergizing Reasoning and Acting in Language Models'. *International Conference on Learning Representations (ICLR)*.

Zadrozny, Bianca, and Charles Elkan. 2002. 'Transforming Classifier Scores into Accurate Multiclass Probability Estimates'. *Proceedings of the Eighth ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 694–99.

Index

A
Act, review, escalate, 60, 61
Agent, 31, 36, 66, 73
Attention, 25, 26
AUC, 14
Audit (random), 64

B
Bake-off, 58
Base rate, 7, 9
Bayes' rule, 10, 49

C
Calibration, 13, 18, 48, 53
Capacity, 62, 65
Choice (question type), 43, 47
Cost-based threshold, 21, 23

D
Decision layer, 35, 40, 65, 66, 74, 88
Decision record, 84
Doom demo, 41
Drift, 60

E
ECE (expected calibration error), 13, 17
Embedding, 25, 26
Escape option, 44
Extract, then decide, 66, 68

G
Gradient descent, 11
Guardrail gate, 68, 73

H
Harbor Pharma, 49
Hybrid agent, 69, 72

I
Isotonic regression, 17

J
Jev-Mem, 68
Jevons, William Stanley, 89
JSON mode, 28, 29

K
Kahneman, Daniel, 5, 37

L
LLM (large language model), 25, 29
LLM-as-judge, 54, 55
Log loss, 10, 11
Logistic regression, 10
Loss, 10, 12

M
Molas, Alex, 48

N
Noul, 43, 44

O
Overconfidence, 25, 29
Overfitting, 11

P
Phishing campaign, 15, 64, 74, 85
Platt scaling, 16, 17
Policy, 60, 65
Prompt injection, 33
Proper scoring rule, 11

R
RAG (retrieval-augmented generation), 31
Reasoning models, 68
Reliability diagram, 13, 16
RLCD, 40
Router, 68, 72

S
Score (question type), 43, 45
Shadow mode, 74, 79, 83, 87
State, 40
System 1, System 2, 37, 42
System One model, 37, 42

T
Temperature (calibration), 17
Threat intel, 70
Three-zone policy, 63

Threshold, 2, 3, 19, 24
TinyJev, 86
Token, 28, 29
Trace, 69
Transformer, 26
Transport, 81, 82
Typed question, 47
TypeSafe AI, 5, 37, 44, 48, 82

V

Vendor-reported, 40

About the Author

Sridhar Mukkandi is an applied AI engineer who builds agents and the decision systems behind them. He writes for engineers who want AI systems they can trust.

sridhar.authorhub@gmail.com